

# PaperPass[免费版]AIGC检测报告

## 简明打印版

AIGC总体疑似度(高+中+轻): 64.98%

AIGC总体疑似度(高+中+轻): 43.62% (加权计算)

高度疑似AIGC占全文比: 3.57%  
中度疑似AIGC占全文比: 54.27%  
轻度疑似AIGC占全文比: 7.14%  
不予检测文字占比: 14.43%

检测版本: 免费版(仅检测中文)

报告编号: 2U5K69C00F4FD8501

论文题目: 基于大数据技术的个性化音乐推荐系统的设计与实现(初稿)

论文作者: 卢伟军

论文字数: 60502

段落个数: 1391

句子个数: 2371

片段个数: 339

提交时间: 2026-3-22 23:48:31

查询真伪: <https://www.paperpass.com/check>

### 疑似度分布图:



### 正文中片段的颜色表示不同的疑似度范围

- 红色: AIGC生成疑似度在70%以上(高度疑似)
- 橙色: AIGC生成疑似度在60%~70%(中度疑似)
- 紫色: AIGC生成疑似度在50%~60%(轻度疑似)
- 黑色: AIGC生成疑似度在50%以下
- 灰色: 过短片段、标题和英文等不予检测的文字

注: AIGC检测可有效识别文本是否部分或全部由AI模型生成, 检测结果与论文质量无关、仅表示论文中内容片段存在AI生成可能性的概率。

加权计算公式:  $(\text{片段1疑似度} + \text{片段2疑似度} + \dots + \text{片段n疑似度}) / n$   
片段疑似度范围0.0~1.0, 合格片段按照0计算, 不计算不予检测文字

# 广州华立学院

## 本科生毕业论文（设计）

题目：基于大数据技术的个性化音乐推荐系  
统设计与实现

|      |   |               |
|------|---|---------------|
| 学    | 院 | 计算机工程学院       |
| 专    | 业 | 数据科学与大数据技术    |
| 班    | 级 | Z1 大数据        |
| 学    | 号 | 5124440106796 |
| 姓    | 名 | 卢伟军           |
| 指导教师 |   | 王波            |

2026 年 4 月

## 原创性声明

本人郑重声明：在广州华立学院攻读学士学位期间，所提交的毕业论文（设计）《基于大数据技术的个性化音乐推荐系统设计与实现》，是本人在导师指导下独立进行研究工作所取得的成果。对本文的研究工作做出重要贡献的个人和集体，均已在文中以明确方式注明，其它未注明部分不包含他人已发表或撰写过的研究成果，不存在购买、由他人代写、剽窃和伪造数据等作假行为。

本人愿为此声明承担法律责任。

作者签名：卢伟军.

日期： 2026 年 4 月 20 日

## 摘 要

AI 52%

随着互联网音乐平台的规模持续扩张，用户面临的信息过载问题日趋严峻，如何从海量曲目中精准匹配用户的潜在偏好，已成为个性化服务领域的核心挑战。本文设计并实现了一套基于大数据技术的个性化音乐推荐系统，由基于 Java Web 技术栈构建的前端交互平台 MusicWeb 与基于 Python 的大数据推荐引擎 MusicMode 两个模块协作构成。

AI 70%

在数据层面，系统基于 KKBOX 公开音乐数据集，通过分布式计算框架完成全量数据导入与清洗，构建了涵盖用户行为统计、歌曲属性、时序交互及 SVD 协同过滤嵌入的 62 维特征工程管道，LightGBM 粗排层使用其中的 32 维子集，并采用 OOF 目标编码技术对训练集内的标签泄漏风险进行系统性规避。

AI 70%

在算法层面，系统构建了四层推荐管道：召回层通过协同过滤、向量近邻检索与热度兜底三通道并行生成 300 个候选集；粗排层部署 LightGBM 进行粗排筛选；精排层部署 DeepFM 与 DIN 对 Top-50 进行深度打分；集成层通过 Stacking 堆叠元学习融合三模型预测结果，进一步提升整体排序质量；重排层施加艺人多样性约束，控制同一艺人的推荐曝光频次，保障推荐列表的内容均衡性。

AI 69%

在工程层面，系统采用 Java Web 技术栈实现 MVC 分层架构，通过节流机制异步采集用户播放行为，经数据库连接池进行高并发持久化管理；推荐反馈表与用户偏好反馈表双表构成推荐反馈闭环，显隐式信号共同驱动推荐结果的持续优化。

AI 69%

离线大规模评估结果表明，系统在 Top-10 推荐任务上，其 HR@10 达到 0.9793，Precision@10 为 0.4992，NDCG@10 与 MRR 分别达到 0.7205 与 0.8114，各项指标均处于较高水平，验证了四层推荐管道设计的整体有效性。

**关键词：**大数据技术；个性化推荐；深度学习；集成学习；协同过滤；特征工程

---

## Abstract

With the continuous expansion of internet music platforms, the problem of information overload faced by users is becoming increasingly severe. How to accurately match users' potential preferences from a massive number of tracks has become a core challenge in the field of personalized services. This thesis designs and implements a personalized music recommendation system based on big data technology, which consists of two collaborative modules: MusicWeb, a front-end interactive platform built with the Java Web technology stack, and MusicMode, a big data recommendation engine based on Python.

At the data level, based on the KKBOX public music dataset, the system completes the import and cleaning of the full dataset through a distributed computing framework. It constructs a 62-dimensional feature engineering pipeline covering user behavior statistics, song attributes, temporal interactions, and SVD collaborative filtering embeddings. The LightGBM coarse ranking layer utilizes a 32-dimensional subset of these features and adopts the OOF target encoding technique to systematically avoid the risk of label leakage in the training set.

At the algorithm level, the system constructs a four-layer recommendation pipeline: the recall layer generates 300 candidate sets in parallel through three channels, including collaborative filtering, vector nearest neighbor search, and popularity-based fallback; the coarse ranking layer deploys LightGBM for preliminary screening; the fine ranking layer deploys DeepFM and DIN for deep scoring of the Top-50 candidates; the ensemble layer fuses the prediction results of the three models through Stacking meta-learning to further improve the overall ranking quality. Finally, the re-ranking layer imposes artist diversity constraints to control the recommendation exposure frequency of the same artist, ensuring the content balance of the recommendation list.

At the engineering level, the system adopts the Java Web technology stack to implement an MVC layered architecture. It asynchronously collects user playback behaviors through a throttling mechanism and performs high-concurrency persistence management via a database connection pool. The recommendation feedback table and the user preference feedback table form a closed loop for recommendation feedback, where explicit and implicit signals jointly drive the continuous optimization of recommendation results.

The results of large-scale offline evaluation show that on the Top-10 recommendation task, the system achieves an HR@10 of 0.9793, a Precision@10 of 0.4992, an NDCG@10 of 0.7205, and an MRR of 0.8114. All metrics are at a relatively high level, verifying the overall effectiveness of the four-layer recommendation pipeline design.

**Keywords:** Big Data Technology, Personalized Recommendation, Deep Learning, Ensemble Learning, Collaborative Filtering, Feature Engineering

目 录

摘 要..... I

Abstract..... II

1 绪 论..... 1

1.1 研究背景与意义 ..... 1

1.2 国内外研究现状 ..... 1

1.2.1 国内研究现状 ..... 1

1.2.2 国外研究现状 ..... 2

1.3 论文研究内容 ..... 3

1.4 论文组织结构 ..... 3

2 推荐系统核心技术原理与方法基础 ..... 5

2.1 大数据处理技术 ..... 5

2.1.1 Hadoop 分布式存储框架..... 5

2.1.2 PySpark 分布式计算框架..... 6

2.2 推荐系统基础理论 ..... 7

2.2.1 协同过滤方法 ..... 7

2.2.2 矩阵分解与 ALS 算法..... 8

2.3 机器学习与深度学习推荐模型 ..... 9

2.3.1 梯度提升树与 LightGBM 原理 ..... 9

2.3.2 DeepFM 模型原理 ..... 10

2.3.3 DIN 深度兴趣网络原理 ..... 12

2.3.4 Stacking 集成学习框架 ..... 14

2.4 向量近邻检索技术 ..... 16

2.4.1 向量检索的工程背景 ..... 16

2.4.2 FAISS 索引体系..... 16

2.5 Java Web 核心技术栈..... 17

|       |                             |    |
|-------|-----------------------------|----|
| 2.5.1 | MVC 架构与 Servlet/JSP 技术..... | 17 |
| 2.5.2 | 数据库连接池技术 .....              | 19 |
| 2.6   | 本章小结 .....                  | 19 |
| 3     | 系统需求分析 .....                | 21 |
| 3.1   | 系统总体目标 .....                | 21 |
| 3.2   | 功能性需求分析 .....               | 21 |
| 3.2.1 | 用户端功能需求 .....               | 22 |
| 3.2.2 | 管理端功能需求 .....               | 23 |
| 3.2.3 | 推荐引擎功能需求 .....              | 24 |
| 3.3   | 非功能性需求分析 .....              | 25 |
| 3.3.1 | 性能需求 .....                  | 26 |
| 3.3.2 | 安全性需求 .....                 | 26 |
| 3.3.3 | 可扩展性需求 .....                | 27 |
| 3.4   | 本章小结 .....                  | 28 |
| 4     | 系统设计 .....                  | 29 |
| 4.1   | 引言 .....                    | 29 |
| 4.2   | 系统总体架构设计 .....              | 29 |
| 4.2.1 | MusicWeb 前端模块架构 .....       | 30 |
| 4.2.2 | MusicMode 推荐引擎模块架构 .....    | 31 |
| 4.2.3 | 双模块交互与部署设计 .....            | 32 |
| 4.3   | 推荐算法流水线设计 .....             | 33 |
| 4.3.1 | 召回层设计 .....                 | 33 |
| 4.3.2 | 粗排成与精排层设计 .....             | 35 |
| 4.3.3 | 集成层设计 .....                 | 36 |
| 4.3.4 | 重排层设计 .....                 | 37 |
| 4.4   | 数据库设计 .....                 | 38 |

|       |                           |    |
|-------|---------------------------|----|
| 4.4.1 | 概念结构设计 .....              | 38 |
| 4.4.2 | 逻辑结构设计 .....              | 39 |
| 4.5   | 前端交互设计 .....              | 45 |
| 4.5.1 | 页面总体结构设计 .....            | 45 |
| 4.5.2 | 核心功能页面设计 .....            | 46 |
| 4.6   | 本章小结 .....                | 48 |
| 5     | 系统实现 .....                | 50 |
| 5.1   | 引言 .....                  | 50 |
| 5.2   | 开发环境与工具配置 .....           | 50 |
| 5.3   | 数据处理与特征工程实现 .....         | 51 |
| 5.3.1 | 数据来源与 ETL 处理 .....        | 51 |
| 5.3.2 | 数据清洗 .....                | 54 |
| 5.3.3 | 特征工程 .....                | 54 |
| 5.3.4 | 数据集时序切分 .....             | 58 |
| 5.4   | MusicMode 推荐算法实现 .....    | 58 |
| 5.4.1 | ALS 协同过滤召回实现 .....        | 58 |
| 5.4.2 | FAISS 向量检索召回实现 .....      | 59 |
| 5.4.3 | 热度兜底召回实现 .....            | 59 |
| 5.4.4 | LightGBM 粗排实现 .....       | 59 |
| 5.4.5 | DeepFM 深度精排实现 .....       | 60 |
| 5.4.6 | DIN 深度兴趣网络实现 .....        | 62 |
| 5.4.7 | Stacking 集成与重排输出的实现 ..... | 63 |
| 5.5   | MusicWeb 前端实现 .....       | 64 |
| 5.5.1 | MVC 分层架构搭建 .....          | 64 |
| 5.5.2 | 用户注册与登录实现 .....           | 65 |
| 5.5.3 | 用户个人信息管理实现 .....          | 66 |



---

|        |                      |    |
|--------|----------------------|----|
| 5.5.4  | 个性化推荐展示 .....        | 68 |
| 5.5.5  | 音乐搜索实现 .....         | 69 |
| 5.5.6  | 在线播放与收藏功能实现 .....    | 70 |
| 5.5.7  | 歌单管理功能实现 .....       | 71 |
| 5.5.8  | 管理员后台功能实现 .....      | 72 |
| 5.5.9  | 隐式反馈采集实现 .....       | 74 |
| 5.5.10 | 数据库连接池配置与持久化实现 ..... | 75 |
| 5.6    | 本章小结 .....           | 76 |
| 6      | 系统测试与评估 .....        | 77 |
| 6.1    | 引言 .....             | 77 |
| 6.2    | 评估方案说明 .....         | 77 |
| 6.2.1  | 离线评估方案 .....         | 77 |
| 6.2.3  | 在线评估方案 .....         | 78 |
| 6.3    | 推荐模型离线评估 .....       | 78 |
| 6.3.1  | 评估指标说明 .....         | 78 |
| 6.3.2  | 不同模型 AUC 结果分析 .....  | 80 |
| 6.3.3  | Top-10 推荐质量评估 .....  | 81 |
|        | 结 论 .....            | 86 |
|        | 参考文献 .....           | 88 |
|        | 致 谢 .....            | 90 |

# 1 绪 论

## 1.1 研究背景与意义

AI 59%  
随着数字音乐流媒体平台的规模持续扩张，以 Spotify、网易云音乐为代表的主流平台曲库容量普遍突破亿首量级，用户通过人工榜单或关键词搜索发现音乐的效率极为低下，信息过载问题日趋突出。个性化推荐系统通过对用户历史行为的系统性建模，将海量候选曲目压缩为符合潜在偏好的少量结果，已成为各平台维持用户活跃度的核心技术基础设施<sup>[1]</sup>。

AI 70%  
音乐推荐在技术层面面临若干固有挑战：用户反馈以播放时长、跳曲等隐式行为为主，正负样本标注噪声较高；音乐消费具有显著的情境依赖性，静态偏好模型难以捕捉用户兴趣的动态漂移；长尾曲目的交互记录极度稀疏，协同过滤方法在冷启动场景下性能退化显著<sup>[2]</sup>。上述挑战要求推荐系统在算法设计与工程实现两个维度均进行系统性定制化处理。

AI 61%  
本文将基于 KKBOX 公开数据集，设计并实现由 MusicWeb 与 MusicMode 构成的双模块个性化音乐推荐系统，探索大数据处理与多模型异构精排在在线音乐场景中的工程集成方案，研究成果可为中小型在线音乐平台的推荐系统建设提供工程参考。

## 1.2 国内外研究现状

### 1.2.1 国内研究现状

AI 69%  
在工程化实践方向，赵吉<sup>[1]</sup>基于 Apache Spark 构建了以 ALS 协同过滤为核心的大规模音乐推荐系统，验证了分布式计算路径的工程可行性，但未引入深度学习精排层，对特征交叉的利用较为有限；杨率<sup>[2]</sup>将深度神经网络应用于个性化实时音乐推荐，探索了用户行为序列的动态偏好建模，前端服务集成方案仍有待完善。在语义增强与情感特征方向，李津<sup>[3]</sup>引入知识图谱丰富物品侧语义表示，有效缓解了稀疏场景下的冷启动问题，但图卷积计算复杂度高，难以适配百万量级曲库的实时推理需求；冯亚

寒<sup>[4]</sup>提出了 WSM-CNN（Wavelet Scattering Multi-scale Convolutional Neural Network, 小波散射多尺度卷积神经网络）音乐多情感分类方法，吴亚迪和陈平华<sup>[5]</sup>设计了融合用户长短期偏好与情感注意力的推荐模型，两者在情感建模上各有贡献，但均缺乏与大规模隐式反馈数据的联合建模。在综述与前沿方法方向，王慧心和童向荣<sup>[6]</sup>梳理了融合知识图谱的推荐系统研究进展；梁锋等<sup>[7]</sup>综述了基于联邦学习（Federated Learning）的推荐系统隐私保护机制；郑楠等<sup>[8]</sup>对基于深度学习的群组推荐方法进行了分类比较；闫猛猛等<sup>[9]</sup>提出了基于自监督学习（Self-Supervised Learning, SSL）的序列推荐算法以降低对标注数据的依赖；龚雪鸾等<sup>[10]</sup>对 CTR（Click-Through Rate, 点击率）预测方法进行了系统综述，为精排层模型选型提供了直接参考。

AI 65%

综合来看，现有研究在单一模型创新方面已较为成熟，但针对多模型集成精排管道的完整工程实现、Java Web 前端与大数据推荐引擎的异构集成及特征工程中目标泄漏的系统性防控，仍缺乏系统性的公开论述，本文将就上述三个方面展开针对性研究。

### 1.2.2 国外研究现状

AI 70%

在音乐推荐专项研究方向，Liu 等<sup>[11]</sup>提出情感驱动的个性化音乐推荐框架，通过歌词与音频的情感分析构建用户情绪-偏好映射；Wan 等<sup>[12]</sup>设计了多视图增强图注意力网络（Graph Attention Network）用于基于会话的音乐推荐，在 KKBOX 与 LastFM 数据集上均取得显著性能提升；Li 等<sup>[13]</sup>提出注意力自编码器（Attentive Autoencoder）用于内容感知型推荐，在稀疏交互场景下优于纯协同过滤基线；Mao 等<sup>[14]</sup>提出轻量化音乐分类推荐网络 Music-CRN，以低延迟实现了分类与推荐的联合建模；Paul 等<sup>[15]</sup>对协同过滤、内容过滤与混合推荐三类范式进行了冷启动与稀疏场景下的适用边界比较。上述工作在情感建模与内容特征方面各有贡献，但大多缺乏与大规模隐式反馈数据的系统融合。在大规模工程方向，Niu 等<sup>[16]</sup>基于 Spark 实现了分布式协同过滤音乐推荐，验证了分区策略对 ALS 训练效率的提升效果；Johnson 等<sup>[17]</sup>提出的 FAISS 已成为工业界主流的 GPU 加速向量近邻检索工具，广泛应用于推荐系统召回层；Wang

等<sup>[18]</sup>通过神经因果图协同过滤（Neural Causal Graph Collaborative Filtering）对交互数据中的混淆因素进行显式建模，有效抑制了观察性偏差对推荐结果的影响。

### 1.3 论文研究内容

本文将重点解决以下四个核心问题：

AI 70%

（1）大规模音乐数据的高效治理与无泄漏特征工程：将基于 KKBOX 数据集构建全量 ETL 导入与特征工程管道，采用 OOF（Out-of-Fold，折外预测）目标编码方案系统性消除训练集与验证集之间的标签泄漏风险，提取 62 维特征（供后续模型训练使用）。

AI 70%

（2）四层推荐管道构建与 Stacking 集成：将 LightGBM、DeepFM 与 DIN 三个模型纳入统一的四层推荐管道（召回→粗排→精排→重排），其中 LightGBM 担任粗排将候选集从 300 压缩至 50，DeepFM 与 DIN 在精排层对 Top-50 进行深度建模，通过 Stacking 的堆叠泛化元学习框架融合三个基础模型的 OOF 预测概率作为元特征，以 SLSQP 约束优化求解三模型最优融合权重，经多样性重排策略后输出 Top-20 最终推荐列表。

AI 70%

（3）MusicWeb 音乐平台全功能工程实现：基于 Java Web 技术栈构建 MVC 分层架构的在线音乐平台，覆盖用户注册登录、个人信息管理、个性化推荐展示、音乐搜索、在线播放与收藏、歌单管理及管理员后台五大模块；通过节流机制异步采集用户播放行为，经 C3P0 连接池高并发持久化写入，构建推荐反馈闭环。

AI 70%

（4）离线与在线双轨推荐效果评估体系：离线评估基于 KKBOX 数据集用户级时序切分，以 HR@10、Precision@10、NDCG@10、MRR 衡量 Top-10 推荐质量；在线评估基于系统实际部署后 2 位真实用户共 337 条交互记录，对比有历史偏好用户与冷启动用户的推荐效果差异，验证系统在不同场景下的实用性。

### 1.4 论文组织结构

本文共分为六章，另设结论，具体内容安排如下：

第 1 章为绪论。阐述课题的研究背景与意义，梳理国内外研究现状，明确本文的

研究内容与技术路线。

第 2 章为相关技术综述。介绍大数据处理技术（Hadoop/Spark/PySpark）、机器学习与深度学习推荐模型（LightGBM、DeepFM、DIN 及 Stacking 集成方法）及 Java Web 技术栈（Jakarta Servlet、JSP、C3P0）。

第 3 章为系统需求分析。从功能性需求与非功能性需求两个维度明确系统规格与用例。

第 4 章为系统设计。描述 MusicWeb 与 MusicMode 双模块总体架构、四层推荐算法流水线设计、数据库核心表结构设计及前端交互方案。

第 5 章为系统实现。重点展示数据预处理与特征工程的落地、推荐算法实现、MusicWeb 后端持久化及 MusicMode 推荐引擎各核心模块的工程实现细节。

第 6 章为系统测试与评估。采用离线与在线双轨评估体系：离线评估通过 AUC 横向对比四个子模型性能，并以 HR@10、Precision@10、NDCG@10、MRR 评估 Top-10 推荐质量；在线评估基于真实部署场景，对比有历史行为用户与冷启动用户的推荐效果差异，综合验证系统推荐管道的整体有效性。

结论部分总结本文的主要研究工作与工程贡献，指出当前系统在单机环境、冷启动策略及在线评估规模方面的局限性，并对引入序列推荐模型、知识图谱增强及分布式部署等后续改进方向进行展望。

## 2 推荐系统核心技术原理与方法基础

### 2.1 大数据处理技术

#### 2.1.1 Hadoop 分布式存储框架

AI 70%

Apache Hadoop 是 Apache 软件基金会开源的分布式存储与计算框架，其核心组件 HDFS 专为海量非结构化与半结构化数据的可靠存储而设计。HDFS 采用主从式架构，由一个 NameNode 主节点与若干 DataNode 数据节点组成。NameNode 负责维护整个文件系统的元数据，包括目录树结构、文件到数据块的映射关系及数据块与节点的位置信息；DataNode 则承担数据块的实际存储、读写与心跳上报任务。文件在写入 HDFS 时被切分为固定大小的数据块（默认 128MB），并在集群中自动保留多个副本（默认副本系数为 3），以此实现节点故障下的数据自动恢复，保障系统的高容错性与高可用性。HDFS 的核心组件构成与底层数据读写架构如图 2-1 所示。

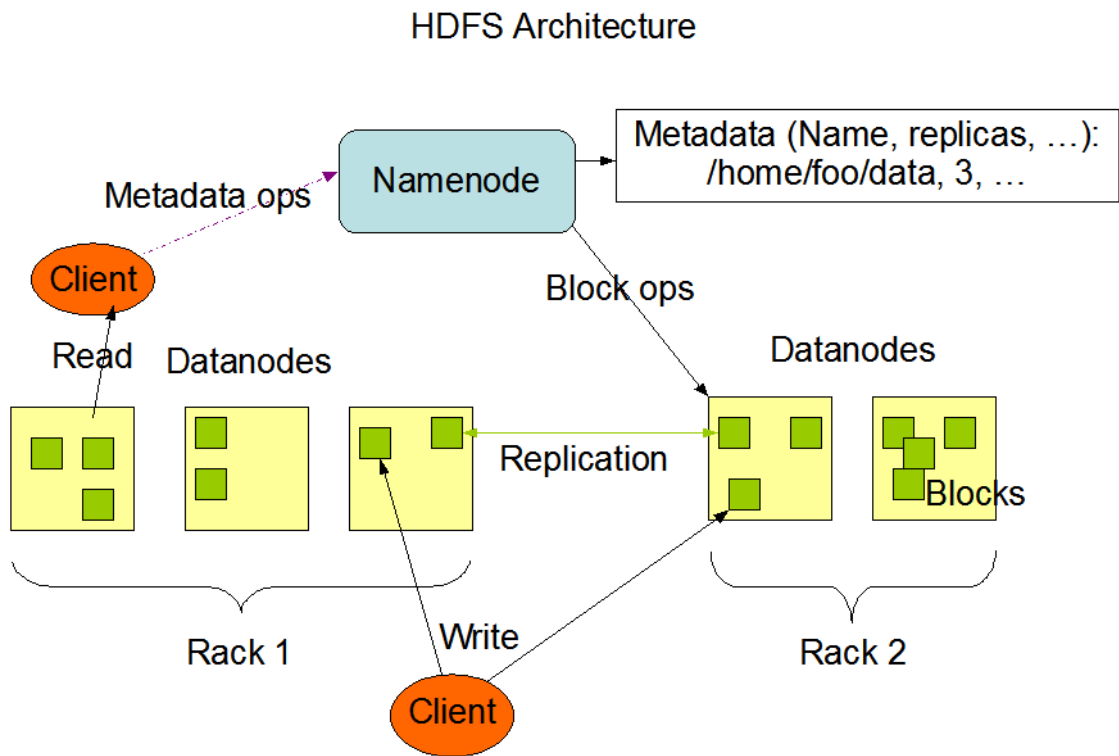


图 2-1 HDFS 读写架构示意图

AI 68%

在技算层面 HDFS 的底层数据处理模型基于 MapReduce 计算范式。MapReduce 将任意分布式批处理问题抽象为 Map 与 Reduce 两个阶段，其形式化描述如下（见 2-

1 和 2-2):

$$\text{Map:}(k_1, v_1) \rightarrow \text{list}(k_2, v_2) \tag{2-1}$$

$$\text{Reduce:}(k_2, \text{list}(v_2)) \rightarrow \text{list}(v_3) \tag{2-2}$$

AI 66%

式中： $k_1$ 、 $v_1$  为输入键值对； $k_2$ 、 $v_2$  为 Map 阶段输出的中间键值对； $v_3$  为 Reduce 阶段对相同键  $k_2$  下所有中间值聚合后的输出结果。

MapReduce 遵循"计算向数据迁移"的数据本地性原则，优先将计算任务调度至数据所在节点，从而有效降低跨节点网络传输开销。然而，其中间结果依赖磁盘持久化，在需要多轮迭代的机器学习算法场景下存在显著的 I/O 瓶颈，这也是引入 Spark 内存计算框架的核心动因。

### 2.1.2 PySpark 分布式计算框架

AI 70%

Apache Spark 是面向大规模数据处理的内存计算框架。通过将中间计算结果缓存于内存而非磁盘，Spark 在机器学习等需要高度迭代计算的场景下，相较于传统的 MapReduce 框架取得了数量级级别的性能提升<sup>[1,16]</sup>。PySpark 作为 Spark 的 Python 语言编程接口（API），支持开发者以 Python 调用 Spark 的完整功能体系，涵盖数据读写、SQL 查询、流处理及 MLlib（机器学习库）。

AI 53%

Spark 的核心数据抽象为 RDD（弹性分布式数据集）。RDD 是分布于集群各节点内存中的只读分区记录集合，具备以下三项核心特性：

（1）血统容错性：RDD 不采用物理数据复制的方式保障容错，而是记录从初始数据集到当前 RDD 的完整转换操作序列，即血统图。当某分区数据因底层计算节点故障而丢失时，Spark 可沿血统路径重新计算并恢复该分区，无需进行全局数据备份，大幅降低了分布式集群的存储开销。

（2）惰性求值：对 RDD 的转换操作（Transformation，如 map、filter、groupBy 等）不会立即触发物理计算，而是在内部构建一张 DAG（有向无环图），以记录各转换步骤间的逻辑依赖关系。仅当程序遇到行动操作（Action，如 collect、count、saveAsTextFile 等）时，Spark 才会提交整个 DAG 并触发实际执行。该机制使得底



层引擎能够在计算执行前对整个计算链路进行全局物理计划优化，例如合并相邻的转换步骤或消除冗余的 Shuffle（数据重分布）过程。

（3）分区并行性：RDD 内的数据按分区分散存储于集群不同节点的 Executor（执行器）进程中，各分区对应的转换与计算任务在 Executor 上并行且独立执行。分区数量直接决定了数据处理的任务并行度，系统可根据集群物理资源与原始数据规模灵活配置该参数。Spark 的 DAG 执行模型与惰性求值流转机制如图 2-2 所示。

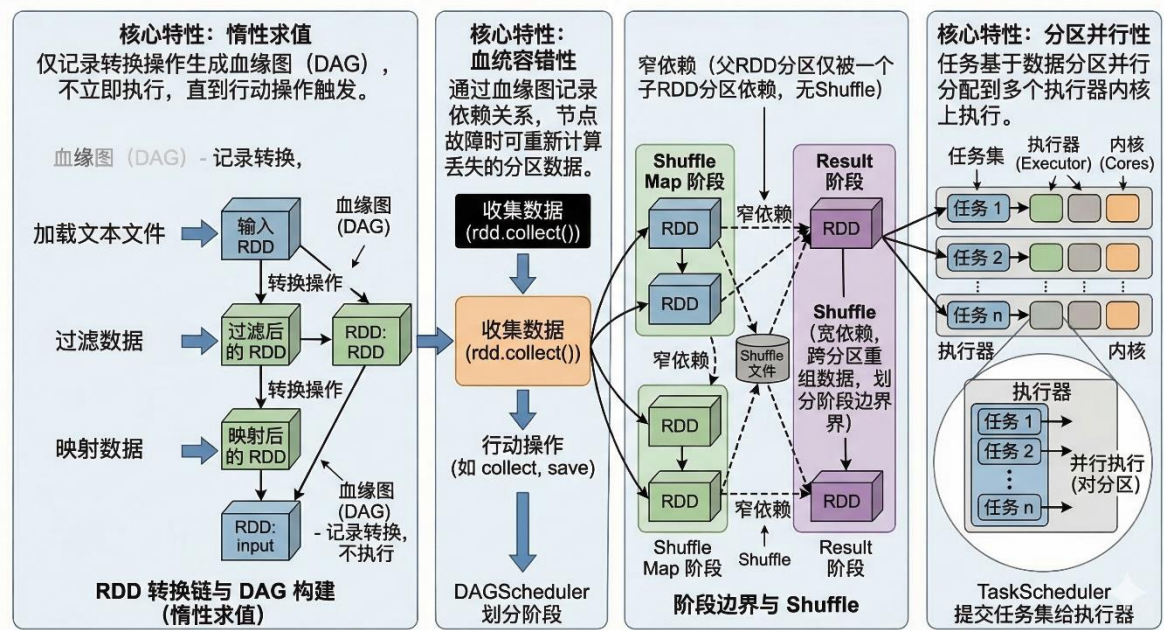


图 2-2 Spark RDD 惰性求值与 DAG 执行模型示意图

AI 70%

在基础的 RDD 抽象之上，Spark 提供了面向结构化数据的 DataFrame 高级 API。DataFrame 内部集成了 Catalyst 逻辑查询优化器与 Tungsten 物理执行引擎，支持基于列式存储的向量化运算，显著提升了处理大规模结构化特征矩阵的吞吐量。此外，Spark MLlib 在 DataFrame 的基础上封装了交替最小二乘法 (ALS)、梯度提升树 (GBDT) 及逻辑回归 (LR) 等主流机器学习算法。这使得本系统能够在分布式数据集上直接完成全量音乐特征工程的构建、模型并行训练与离线评估，是实现工业级个性化推荐算法工程化落地的核心支撑组件。

## 2.2 推荐系统基础理论

### 2.2.1 协同过滤方法



协同过滤（Collaborative Filtering, CF）是推荐系统中历史最悠久、工业应用最广泛的算法范式，其核心假设为：具有相似历史交互行为的用户，在未来偏好上亦趋向一致；被相似用户群体共同交互的物品，对某一特定用户同样具有推荐价值。

协同过滤方法通常分为两类。基于用户的协同过滤（User-based CF）通过计算目标用户与其他用户之间的相似度，从最近邻用户的交互历史中筛选目标用户尚未接触的物品作为候选。基于物品的协同过滤（Item-based CF）则计算物品间的相似度，依据用户已交互物品的相似物品生成推荐列表；物品相似度在时间维度上较用户相似度更为稳定，在物品规模远小于用户规模的场景下工程成本更低。

AI 88%  
传统协同过滤的核心局限在于数据稀疏性与可扩展性两方面：当用户-物品交互矩阵稀疏率超过 99% 时，基于相似度的邻居搜索精度显著退化；同时随用户与物品规模增长，相似度矩阵的计算与存储开销呈平方级增长，难以适配百万量级的工业场景。矩阵分解方法通过将高维稀疏交互矩阵投影至低维稠密潜在语义空间，有效弥补了上述不足。

### 2.2.2 矩阵分解与 ALS 算法

矩阵分解方法将用户-物品交互矩阵  $R \in \mathbb{R}^{m \times n}$ （ $m$  为用户数， $n$  为物品数）近似分解为用户潜在因子矩阵  $U \in \mathbb{R}^{m \times k}$  与物品潜在因子矩阵  $V \in \mathbb{R}^{n \times k}$  的乘积，其中  $k \ll \min(m, n)$  为潜在因子维度。用户  $u$  对物品  $i$  的预测偏好得分表示为（见 2-3）：

$$\hat{r}_{ui} = \mathbf{u}_u^T \mathbf{v}_i \quad (2-3)$$

式中： $\mathbf{u}_u \in \mathbb{R}^k$  为用户  $u$  的潜在因子向量； $\mathbf{v}_i \in \mathbb{R}^k$  为物品  $i$  的潜在因子向量。

在音乐推荐等隐式反馈场景中，用户行为数据（播放次数、播放时长等）仅反映正向兴趣信号，缺乏明确的负样本标注。针对这一特点，ALS 隐式反馈方法引入置信度加权机制，将原始交互计数  $c_{ui}$  转化为置信权重（见 2-4）：

$$conf_{ui} = 1 + \alpha \cdot c_{ui} \quad (2-4)$$

式中： $\alpha$  为置信度放大系数，控制有交互样本相对于无交互样本的权重比例。在此基础上，ALS 的加权优化目标函数为：

$$L = \sum_{u, i} \text{conf}_{ui} (\mathbf{p}_{ui} - \mathbf{u}_u^T \mathbf{v}_i)^2 + \lambda \left( \sum_u \|\mathbf{u}_u\|^2 + \sum_i \|\mathbf{v}_i\|^2 \right) \quad (2-5)$$

式中:  $\mathbf{p}_{ui} \in \{0,1\}$  为二值化偏好指示变量 (有交互记录时取 1, 否则取 0);  $\lambda$  为 L2 正则化系数, 用于抑制模型过拟合。

ALS 算法通过交替固定物品因子矩阵求解用户因子矩阵、再交替固定用户因子矩阵求解物品因子矩阵的方式, 将上述非凸优化问题转化为两个交替进行的凸二次规划子问题, 每次迭代均存在解析解 (见 2-6):

$$\mathbf{u}_u = (V^T C^u V + \lambda I)^{-1} V^T C^u \mathbf{p}_u \quad (2-6)$$

式中:  $C^u \in \mathbb{R}^{n \times n}$  为以  $\text{conf}_{ui}$  为对角元素的置信度对角矩阵;  $\mathbf{p}_u \in \mathbb{R}^n$  为用户  $u$  的偏好指示向量;  $I$  为单位矩阵。物品因子向量  $\mathbf{v}_i$  的更新公式与式 (2-6) 形式对称。由于各用户 (或各物品) 因子向量的求解过程相互独立, ALS 天然支持大规模分布式并行计算, 是 Spark MLlib 内置 ALS 实现在分布式推荐场景中被广泛采用的根本原因<sup>[1,16]</sup>。

## 2.3 机器学习与深度学习推荐模型

### 2.3.1 梯度提升树与 LightGBM 原理

AI 68%

梯度提升树 (Gradient Boosting Decision Tree, GBDT) 是一种以决策树为弱学习器的迭代集成方法, 其核心理念是逐步逼近: 每一轮训练的目标并非直接预测真实标签, 而是专门拟合当前集成模型在所有样本上的预测残差 (负梯度方向), 逐步将整体误差压缩至最小。设经过第  $T$  轮迭代后集成模型的预测值为  $\hat{F}_T(x)$ , 其加法更新规则为以下公式 (2-7):

$$\hat{F}_T(x) = \hat{F}_{T-1}(x) + \eta \cdot h_T(x) \quad (2-7)$$

式中  $\hat{F}_{T-1}(x)$ ——前  $T-1$  轮集成模型的预测值;  $h_T(x)$ ——第  $T$  轮新增的决策树弱学习器;  $\eta$ ——学习率, 控制每棵树对最终预测结果的贡献步长, 取值越小模型泛化能力越强。

AI 64%

而 LightGBM 是微软开源的高效 GBDT 实现框架, 针对大规模高维特征场景引入了三项关键工程优化: 直方图算法、基于梯度的单边采样与互斥特征捆绑。直方图

算法将连续特征离散化为  $K$  个区间，节点分裂时仅需遍历  $K$  个候选切分点而非全部样本值，将单次分裂的时间复杂度从  $O(n)$  降至  $O(K)$ 。图 2-3 展示了 LightGBM 的整体算法原理结构，此外最优切分点依据分裂增益公式如下 (2-8)：

$$Gain = \frac{1}{2} \left[ \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma \tag{2-8}$$

式中  $G_L$ 、 $G_R$ ——分别为左、右子节点所有样本的一阶梯度之和； $H_L$ 、 $H_R$ ——分别为左、右子节点所有样本的二阶梯度之和； $\lambda$ ——叶节点权重的 L2 正则化系数，惩罚过大的叶节点输出值； $\gamma$ ——引入新叶节点的最小收益阈值，增益低于该值时分裂被剪枝，起结构正则化作用。

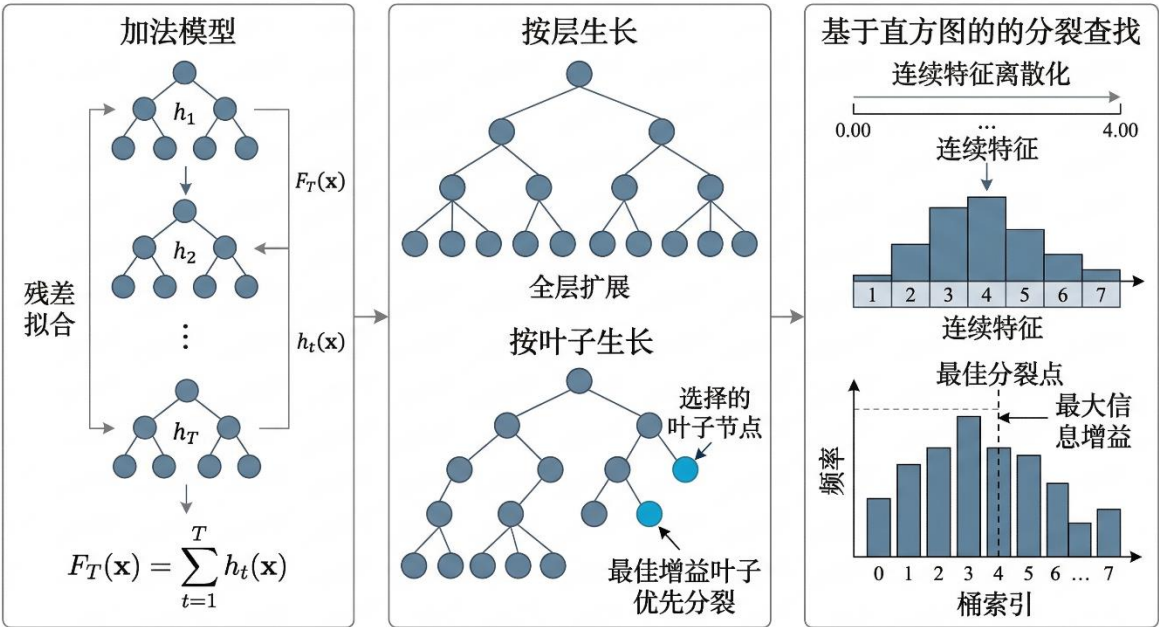


图 2-3 LightGBM 算法原理示意图

AI 70%

此外，LightGBM 采用基于叶子的生长策略：每轮优先分裂当前增益最大的叶节点，而非传统的逐层（Level-wise）全量扩展。在相同叶节点数量约束下，Leaf-wise 策略可获得更低的训练损失，同时配合 GOSS 对大梯度样本优先采样，使模型在处理系统 47 维用户-歌曲混合特征时兼具高效性与强泛化能力。

2.3.2 DeepFM 模型原理

AI 85%

推荐系统的核心挑战在于如何高效捕捉特征间的交叉关系。传统因子分解机

（Factorization Machine, FM）擅长建模二阶特征交叉，而神经网络（Deep Neural Network, DNN）则能拟合高阶非线性交互，但二者长期独立发展，难以兼顾低阶与高阶特征交叉的同步学习。DeepFM 通过共享 Embedding 层将 FM 与 DNN 有机整合于同一端到端框架中联合训练，无需任何人工特征工程即可同时捕获低阶与高阶特征交互，其模型结构示意图如图 2-4 所示。

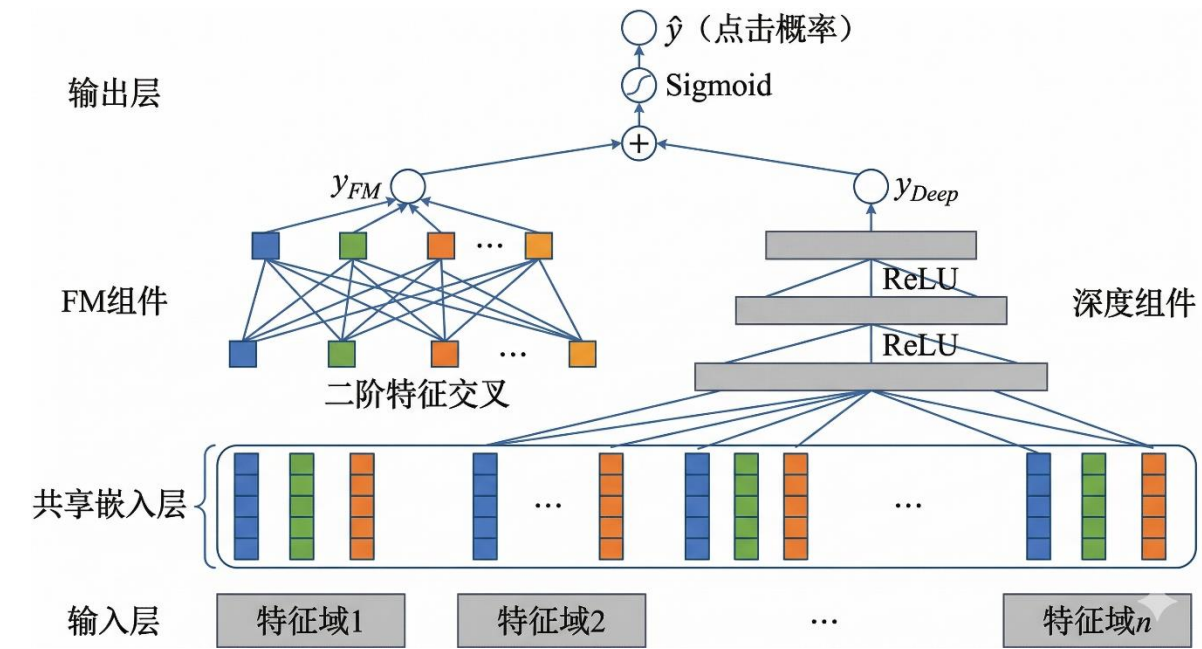


图 2-4 DeepFM 模型结构示意图

DeepFM 由 FM 组件与 Deep 组件并联构成，两部分共享同一 Embedding 层输出，分别承担不同阶次的特征交叉建模任务。

（1）FM 组件：

FM 组件负责捕捉一阶线性关系与二阶特征交叉。对于含  $n$  个特征域的输入，FM 组件的输出见式（2-9）。

$$y_{FM} = w_0 + \sum_{i=1}^n w_i x_i + \frac{1}{2} \sum_{k=1}^K \left[ \left( \sum_{i=1}^n v_{ik} x_i \right)^2 - \sum_{i=1}^n v_{ik}^2 x_i^2 \right] \quad (2-9)$$

式中  $w_0$ ——全局偏置项； $w_i$ ——第  $i$  个特征的一阶线性权重； $v_i$ ——第  $i$  个特征对应的  $K$  维 Embedding 隐向量； $K$ ——隐向量维度。

上式通过等价变换将二阶特征交叉的计算复杂度从  $O(n^2)$  降至  $O(nK)$ ，同时以隐

向量内积  $\langle v_i, v_j \rangle$  衡量特征  $i$  与特征  $j$  的交叉强度,即使对于训练集中未曾共现的特征对,也能通过隐向量泛化估计其交叉权重。

### (2) Deep 组件

AI 68%

Deep 组件为多层全连接神经网络,以各特征域 Embedding 向量的拼接结果作为输入,通过逐层非线性变换挖掘高阶特征交叉关系,第 1 层的前向传播规则见式 (2-10)。

$$a^{(l+1)} = \sigma(W^{(l)}a^{(l)} + b^{(l)}) \quad (2-10)$$

式中  $a^{(l)}$ ——第 1 层神经元输出向量;  $W^{(l)}$ ——第 1 层权重矩阵;  $b^{(l)}$ ——第 1 层偏置向量;  $\sigma$ ——激活函数 (本系统采用 ReLU)。

### (3) 联合预测输出

AI 70%

DeepFM 将 FM 组件与 Deep 组件的输出相加后,经 Sigmoid 函数映射至  $[0, 1]$  区间,作为用户对候选歌曲的点击概率预测值,见式 (2-11)。

$$\hat{y} = \sigma(y_{FM} + y_{Deep}) \quad (2-11)$$

由于 FM 与 Deep 两组件共享同一 Embedding 层,二者在反向传播时联合更新参数,避免了分阶段训练带来的次优解问题。这一设计使 DeepFM 能够以端到端方式同时优化低阶交叉的精确性与高阶交叉的表达能。在本系统精排集成阶段,DeepFM 以用户历史行为特征、歌曲属性特征及上下文特征为联合输入,对 LightGBM 粗排筛选后的 Top-50 候选进行精细化打分,其预测结果将与 LightGBM 及 DIN 的输出一同送入后续 Stacking 集成层进行加权融合决策。

## 2.3.3 DIN 深度兴趣网络原理

AI 70%

传统深度推荐模型在提取用户兴趣表征时,通常对历史行为序列中所有交互物品的 Embedding 向量直接求平均,即以等权池化方式生成唯一的用户兴趣向量。这一做法忽视了用户兴趣的多样性与局部性:当用户面对某一候选歌曲时,其历史播放记录中与该歌曲风格、节奏或歌手相近的行为才是当前决策的主要驱动因素,而其余不相关的历史记录理应被赋予更低权重。针对上述局限,阿里巴巴提出了深度兴趣网络 (Deep Interest Network, DIN),引入注意力机制对用户历史行为序列进行动态加权,使模型能够自适应地激活与目标候选物品高度相关的历史兴趣片段。



AI 69%

DIN 的整体结构如图 2-5 所示。对于含  $N$  条历史行为记录的用户，设第  $i$  条历史物品的 Embedding 向量为  $e_i$ ，目标候选物品的 Embedding 向量为  $e_a$ ，则模型输出的用户动态兴趣表征式 (2-12)：

$$v_u = \sum_{i=1}^N a(e_i, e_a) \cdot e_i \quad (2-12)$$

式中  $v_u$ ——面向当前候选物品动态生成的用户兴趣向量； $a(e_i, e_a)$ ——历史物品  $i$  相对于候选物品的注意力得分； $N$ ——用户历史行为序列长度。

AI 70%

用户动态兴趣表征式 (2-12) 的核心在于注意力得分  $a(e_i, e_a)$  的计算方式。不同于简单的点积注意力，DIN 专门设计了激活单元，将历史物品 Embedding 与目标物品 Embedding 通过拼接、差值及逐元素乘积进行多维度交叉，再经小型多层感知机输出注意力得分，其计算方式为 (2-13)：

$$a(e_i, e_a) = \text{MLP}([e_i; e_a; e_i - e_a; e_i \odot e_a]) \quad (2-13)$$

式中  $[;]$ ——向量拼接操作； $e_i - e_a$ ——差值向量，度量历史物品与目标物品在特征空间中的距离； $e_i \odot e_a$ ——逐元素乘积，捕捉两者在各维度上的协同关系。

AI 68%

值得注意的是，DIN 有意不对注意力得分进行 Softmax 归一化。其原因在于：对不同用户而言，历史行为数量存在显著差异，Softmax 归一化会将各用户的注意力权重强制压缩至相同的概率分布，从而抹去行为总量所携带的兴趣强度信息。保留未归一化的绝对激活值，可使行为更丰富、与目标物品更相关的用户获得更强的兴趣信号，更真实地反映其偏好程度。

AI 66%

在本系统中，DIN 被应用于精排阶段的序列化用户兴趣建模。用户近期的播放历史（取最近 50 条记录）中每首歌曲的 Embedding 向量构成行为序列输入，以当前候选歌曲的 Embedding 为查询向量，经激活单元动态计算注意力权重后，加权聚合生成面向该候选曲目的用户兴趣表征。该表征与用户基础特征、歌曲属性特征拼接后，经全连接层输出点击概率预测值，并与 LightGBM 和 DeepFM 的输出一同汇入 Stacking 集成层进行最终融合决策，从而构成本系统精排集成模型的序列感知能力核心。

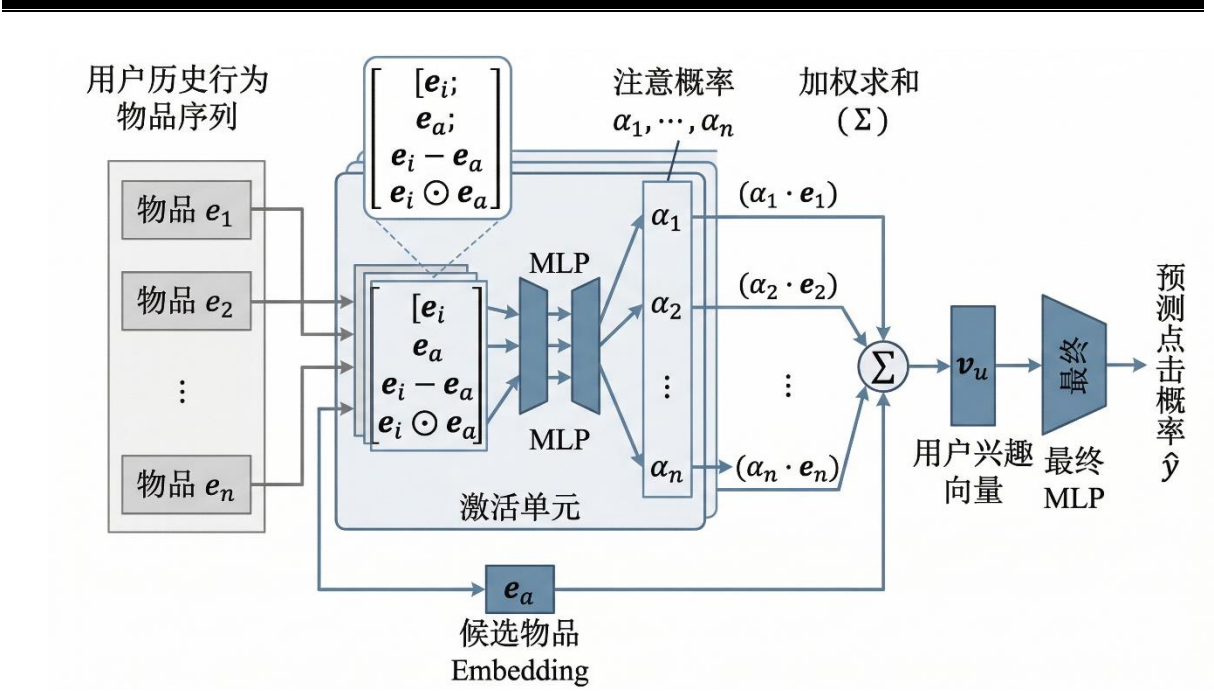


图 2-5 DIN 模型结构示意图

2.3.4 Stacking 集成学习框架

AI 65%

集成学习（Ensemble Learning）的核心思想是将多个在不同方面各有所长的弱学习器有机组合，使整体预测性能超越任意单一模型。其理论依据来自偏差-方差权衡：单一模型往往难以同时压低偏差与方差，而异质的多模型组合可相互补偿各自的弱点。常见的集成范式有三类：Bagging 通过有放回的自助采样训练多个独立模型并取平均，以降低方差为主，随机森林是其典型代表；Boosting 以串行方式逐步拟合前序模型的残差，以降低偏差为主，LightGBM 属于此类；Stacking 则不预设固定的聚合规则，而是额外训练一个元学习器（Meta-Learner），令其自行学习如何最优地融合各基学习器的预测结果，理论上可在偏差与方差两个维度上同时取得收益。

AI 69%

本系统以 LightGBM、DeepFM 与 DIN 作为三个基学习器，三者归纳偏置上存在显著互补性：LightGBM 擅长捕捉手工特征中的非线性交互，负责将召回候选快速压缩至 Top-50；DeepFM 通过 FM 与 DNN 并联同时处理低阶与高阶特征交互；DIN 通过注意力机制动态聚焦与候选歌曲相关的用户历史兴趣。Stacking 框架的两阶段训练流程如图 2-6 所示。

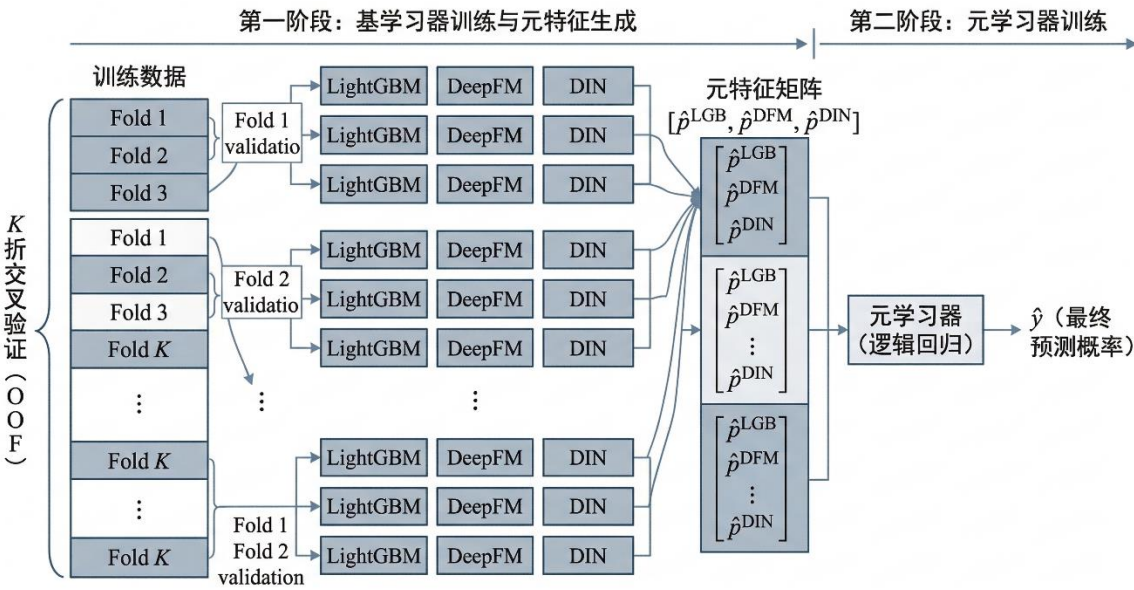


图 2-6 Stacking 两阶段训练框架示意图

AI 56%

第一阶段训练基学习器并生成元特征。采用  $K$  折交叉验证下的折外预测（Out-of-Fold, OOF）策略：每折以其余  $K-1$  折数据训练各基学习器，再对当前折样本进行预测；遍历全部  $K$  折后，每个训练样本恰好被预测一次，三个基学习器各生成一列 OOF 预测值，拼接构成元特征矩阵。第二阶段以元特征矩阵为输入、真实标签为监督信号，训练元学习器（本系统采用逻辑回归），最终预测见式（2-14）。

$$\hat{y} = \sigma! \left( w_1 \hat{p}^{(LGB)} + w_2 \hat{p}^{(DFM)} + w_3 \hat{p}^{(DIN)} + b \right) \tag{2-14}$$

式中  $\hat{p}^{(LGB)}$ 、 $\hat{p}^{(DFM)}$ 、 $\hat{p}^{(DIN)}$ ——LightGBM、DeepFM、DIN 三个基学习器的 OOF 预测概率； $w_1$ 、 $w_2$ 、 $w_3$ ——元学习器训练所得的融合权重； $b$ ——偏置项； $\sigma(\cdot)$ ——Sigmoid 函数。

AI 69%

OOF 策略的关键价值在于：元学习器的训练数据始终来自基学习器在“未见过”样本上的预测，而非训练集上的拟合值，从而切断了基学习器过拟合信息向元学习器的传递路径。

在特征工程层面，OOF 策略同样用于防范标签泄漏（Label Leakage）。若对歌手、语言、流派等高基数类别特征直接使用全量训练集的点击均值进行目标编码，则样本自身的标签信息将渗入其特征值，造成离线指标虚高而上线效果骤降。为此，本系统采用 OOF 目标编码：对样本  $i$  的类别特征  $c_i$ ，其编码值仅由排除样本  $i$  所在折  $V_k$  后



的训练样本统计得出，见式（2-15）。

$$TE(c_i) = \frac{1}{|S_k(c_i)|} \sum_{j \in S_k(c_i)} y_j \tag{2-15}$$

式中  $c_i$ ——样本  $i$  的类别特征取值； $S_k$ ——样本  $k$  所在的第  $k$  折验证集； $y_j$ ——样本  $j$  的真实标签；

统计范围严格排除第  $k$  折，切断标签信息的反向渗透路径。本系统将式（2-15）应用于歌手、语言、流派等高基数类别特征的编码，从根本上保证了特征工程过程中的信息隔离，是确保离线评估结论可靠性的关键工程措施。

## 2.4 向量近邻检索技术

### 2.4.1 向量检索的工程背景

推荐系统的召回阶段本质上是一个相似度搜索问题：将用户表示为一个嵌入向量，在数百万首歌曲的嵌入向量库中找出与之最相似的候选集。当 ALS 矩阵分解训练完毕后，每位用户和每首歌曲都对应一个低维稠密向量，召回任务即转化为在物品向量库中寻找与用户向量内积最大的 Top-K 物品，称为最大内积搜索（Maximum Inner Product Search，MIPS），其计算公式如下（2-16）：

$$j^* = \arg(\max_{j \in \mathcal{C}}) \mathbf{u}_u^T \mathbf{v}_j \tag{2-16}$$

式中  $\mathbf{u}_u$ ——用户  $u$  的潜在因子向量； $\mathbf{v}_j$ ——物品  $j$  的潜在因子向量； $\mathcal{C}$ ——全体物品集合。

暴力线性扫描的时间复杂度为  $O(nd)$ ，在本系统 229 万首歌曲、128 维嵌入的规模下，单次查询需逐一计算数百万次向量内积，延迟高达数秒，远不满足在线推荐的实时性要求。因此，有必要引入近似最近邻（Approximate Nearest Neighbor，ANN）算法，在可接受的精度损失下将查询延迟压缩至毫秒级。

### 2.4.2 FAISS 索引体系

FAISS（Facebook AI Similarity Search）是 Meta 开源的高性能向量检索库，针对

CPU 和 GPU 均提供了优化实现，支持十亿级向量规模的高效检索。其核心设计思路是"粗量化缩小搜索域 + 精量化压缩存储"的两级结构。

倒排文件索引（IVF）是 FAISS 缩小搜索域的主要手段。构建索引时，先用 k-means 将全部物品向量聚类为 $n_{list}$ 个 Voronoi 单元，每个单元维护一个倒排列表，记录归属该单元的所有物品编号。查询时无需扫描全库，只需找到与查询向量最近的若干个聚类中心，在这几个中心对应的倒排列表内进行精确搜索即可。参数 $n_{probe}$ 控制检索时扫描的单元数： $n_{probe}$ 越大，召回率越高，但延迟也随之增加，是速度与精度之间的核心权衡参数。

乘积量化是 FAISS 压缩向量存储的核心技术。其思想是将一个高维向量横向切分为M段等长的子向量，对每个子空间独立训练一个小码书（通常含 256 个码字），用各子段最近码字的索引替代原始浮点数存储，其公式如下（2-17）：

$$\mathbf{v} = [\mathbf{v}^{(1)}, \mathbf{v}^{(2)}, \dots, \mathbf{v}^{(M)}], \mathbf{v}^{(m)} \in \mathbb{R}^{(d/M)} \quad (2-18)$$

以 128 维向量、M=8段为例，每段仅需 1 字节（ $\log_2 256=8$  bits）存储，整个向量压缩为 8 字节，相比原始 512 字节的 32 位浮点表示压缩比达 64 倍。更关键的是，查询时可预先计算查询向量各子段与所有码字的距离并缓存为查找表，随后对库中任意压缩向量的近似距离计算只需 M 次查表累加，极大加速了批量距离估计。

IVFPQ 将两者结合：物品向量先经 IVF 粗量化分配至最近聚类中心，再对其相对于聚类中心的残差向量进行 PQ 编码。对残差而非原始向量量化，可显著减小量化误差，在高压缩率下仍保持较高的检索精度。在本系统中，ALS 训练完毕的物品嵌入向量以 IVFPQ 格式离线建库，用户发起请求时召回层只需毫秒级完成百万候选的向量粗筛，为精排层提供高质量的初始候选集。

## 2.5 Java Web 核心技术栈

### 2.5.1 MVC 架构与 Servlet/JSP 技术

AI 85%

随着系统功能的增长，若将业务逻辑、数据访问与页面展示混写于同一模块，代码耦合度将随需求迭代急剧上升，维护与扩展成本难以控制。MVC 设计模式通过职

责分离将应用程序划分为三个协作层次：模型层负责封装业务规则与数据访问逻辑；视图层专注于数据的界面呈现与交互反馈；控制器层承担请求路由与流程调度。三者各司其职，任一层次的修改均不波及其他层次，使系统在功能扩展时保持清晰的边界。

AI 69%

在 Java Web 技术体系中，MVC 三层分别由 Servlet、JSP 与 JavaBean/DAO 承担具体实现。如图 2-7 所示，一次完整的 HTTP 请求处理流程如下：用户在前端发起请求后，Web 容器依据路由配置将请求分发至对应的 Servlet；Servlet 作为控制器解析请求参数，调用服务层执行相应的业务逻辑——对于个性化推荐请求，服务层负责与推荐引擎交互获取模型评分结果，并通过 DAO 层检索数据库以补全歌曲元数据；处理完毕后，Servlet 将结果数据封装并转交 JSP 视图层进行动态页面渲染，最终将完整响应返回客户端。

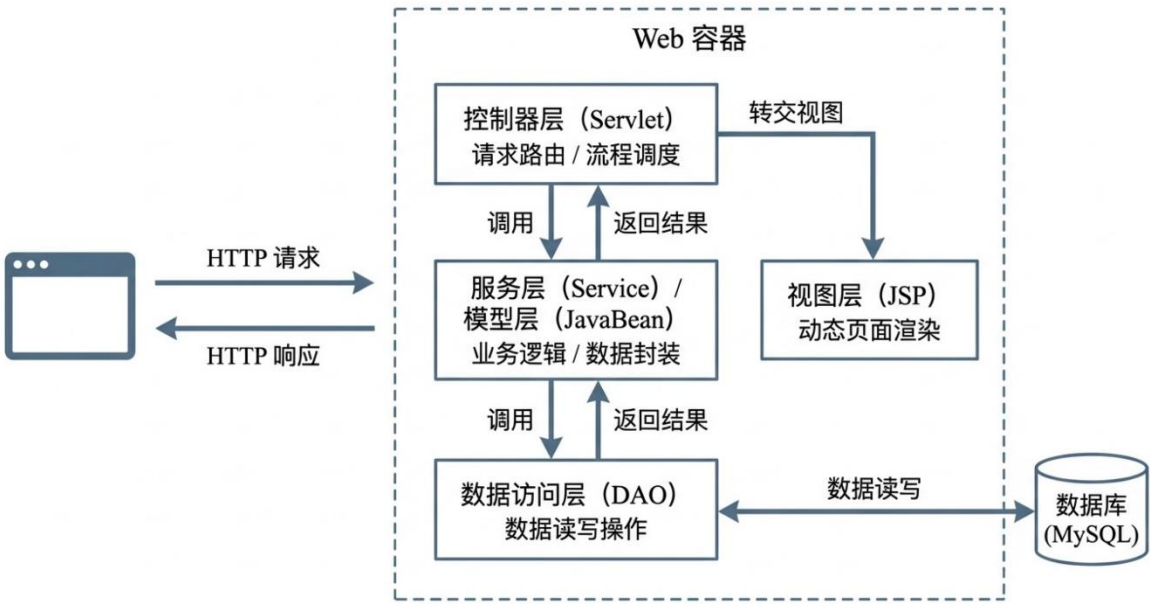


图 2-7 MVC 分层架构请求处理示意图

AI 70%

本系统的控制器层按业务职能划分为用户认证、推荐列表获取、播放行为上报与歌曲检索四类模块；数据访问层对用户信息、歌曲元数据及交互记录等核心数据表的读写操作进行统一封装，实现业务逻辑与存储细节的解耦；视图层负责渲染音乐播放器界面与推荐歌单，通过异步请求机制实现推荐列表的无刷新动态更新，保障了用户交互的流畅性。三层协作使本系统的前后端职责边界清晰，为后续功能迭代与模块化

维护奠定了架构基础。

### 2.5.2 数据库连接池技术

AI 69%  
在 Web 应用中，数据库连接的建立是一项代价高昂的操作：每次请求若均经历完整的连接创建与断开流程，在高并发场景下将对系统响应速度与数据库承载能力造成显著冲击，极端情况下甚至导致可用连接耗尽、服务不可用。数据库连接池技术通过预先初始化并复用连接的方式从根本上规避了上述问题：系统启动时在池中创建一定数量的数据库连接并保持就绪状态；业务请求到来时直接从池中获取空闲连接，使用完毕后将连接归还至池而非真正关闭；当并发请求量超出池容量时，新请求进入等待队列，直至有连接被释放，从而将连接创建的高额开销分摊至系统初始化阶段，大幅降低单次请求的数据库访问延迟。

AI 76%  
本系统选用 C3P0 作为数据库连接管理组件。C3P0 是一款经过广泛生产验证的开源 JDBC 连接池，具备完善的连接生命周期管理能力：通过合理配置池的最小与最大连接边界，在空闲期收缩资源占用、在高并发期弹性扩展，兼顾了资源利用率与请求响应能力；连接获取超时机制确保在连接资源紧张时及时向上层返回明确的失败信号，避免请求无限期阻塞；空闲连接的定期回收策略则防止长时间闲置的连接占用数据库端资源。此外，C3P0 对 PreparedStatement 的缓存复用进一步减少了重复 SQL 编译开销，提升了数据库访问的整体吞吐量。相较于直接使用 JDBC 原生连接，C3P0 将连接管理的复杂性完全封装于框架内部，使业务层专注于数据访问逻辑本身，在保证系统高并发稳定性的同时也显著降低了连接管理的维护成本。

## 2.6 本章小结

AI 62%  
本章围绕个性化音乐推荐系统的核心技术原理与方法基础展开系统性论述，从大数据处理基础设施、推荐算法理论、机器学习与深度学习模型、向量检索工程，到 Web 后端技术栈，构成了支撑本系统完整技术体系的理论依据。

AI 70%  
在大数据处理层面，本章介绍了 Hadoop HDFS 的分布式存储机制与 MapReduce 计算范式，阐明了其在海量音乐交互数据可靠存储与批处理方面的适用性；进而引入

PySpark 框架，依托其 RDD 惰性求值、DAG 执行优化与 DataFrame 高级抽象，为系统的全量特征工程构建与分布式模型训练提供了高效的计算支撑。

AI 68%

在推荐算法理论层面，本章系统梳理了协同过滤的基本假设与局限性，并重点介绍了矩阵分解方法的理论框架及其隐式反馈变体 ALS 算法，说明了 ALS 置信度加权机制在处理用户播放行为数据中的合理性与工程价值。

AI 70%

在分层排序模型层面，本章依次阐述了 LightGBM、DeepFM 与 DIN 三个基学习器的核心原理：LightGBM 以直方图算法与 Leaf-wise 生长策略高效建模非线性特征交叉，担任粗排快速压缩候选集；DeepFM 通过共享 Embedding 层实现低阶与高阶特征交互的端到端联合学习；DIN 借助激活单元对用户历史行为序列进行动态注意力加权，捕捉与候选物品相关的局部兴趣。在此基础上，Stacking 集成框架通过 OOF 折外预测策略生成无泄漏元特征，以 SLSQP 约束优化算法求解三模型最优融合权重，同时将 OOF 机制延伸至高基数类别特征的目标编码，从工程层面保障了离线评估结论的可信度。

AI 69%

在工程支撑层面，本章介绍了 FAISS 向量近邻检索库的 IVFPQ 两级索引结构，说明了其在百万级歌曲候选集毫秒级粗召回中的技术价值；并阐述了基于 MVC 模式的 Java Web 后端架构与 C3P0 数据库连接池技术，为系统的在线服务能力与高并发稳定性奠定了基础。上述技术模块相互协作，共同构成本系统"离线分布式建模—向量召回—精排融合—在线响应"完整推荐链路的理论与工程支撑，为后续系统设计与实现的详细阐述提供依据。

### 3 系统需求分析

#### 3.1 系统总体目标

AI 68%

本系统的设计目标是构建一套面向在线音乐场景的双模块个性化推荐平台，由 MusicWeb 用户交互模块与 MusicMode 推荐引擎模块协同构成。系统需在完整支持音乐播放、收藏、搜索等基础业务功能的同时，实现基于大数据技术的用户兴趣建模与个性化内容推送，并通过隐式反馈采集形成推荐优化的持续数据闭环。

AI 64%

从功能层面看，系统须为普通用户提供注册登录、音乐检索与播放、收藏与歌单管理、个性化推荐展示及账号自助管理等完整使用路径；为管理员提供用户账号管理、歌曲资源管理、申诉审核与数据统计等后台运营能力；为推荐引擎提供多路候选召回、多模型异构精排、多样性重排与定时推荐更新等算法执行能力。三者之间通过共享的 MySQL 数据库实现数据流转与状态同步。

AI 69%

从性能层面看，系统在正常并发负载下须保证用户界面的交互响应延迟维持在可接受范围内；推荐引擎在面向全量注册用户执行每日推荐重算时，须在合理时间窗口内完成三通道召回、精排与写库的完整流程；FAISS 向量检索模块须能在毫秒级时延内完成针对单用户画像的候选歌曲检索。

AI 68%

从可用性层面看，系统须具备基本的异常容错能力，数据库连接池须能弹性应对并发访问峰值；推荐服务在部分模型异常时须能降级至基础召回策略，保证推荐列表的最低可用性。

综上，本系统的总体目标可归纳为：以大数据技术为基础设施，以多模型集成精排管道为算法核心，以 Java Web 平台为用户交互载体，构建一套具备功能完整性、性能可达性与工程可维护性的个性化音乐推荐系统。

#### 3.2 功能性需求分析

本系统的功能性需求从用户端、管理端与推荐引擎三个维度展开分析，各维度对应不同的参与者角色与业务交互目标。



3.2.1 用户端功能需求

AI 52%

用户端面向普通注册用户，提供音乐消费的完整交互闭环。其整体功能如图 3-1 所示，其中用户端的核心用例覆盖以下六个功能域。

(1) 账号管理：

系统须支持新用户完成账号注册，注册信息包括用户名、密码、邮箱、手机号等基本资料。注册成功后，用户通过身份验证登录系统，服务端基于会话机制维护登录状态。用户在会话期间可访问个人中心，对昵称、邮箱、手机号等资料进行修改，并支持密码变更与账号注销操作。针对账号因违规被冻结或删除的情形，系统须提供申诉入口，用户可填写申诉原因与联系邮箱提交申诉请求，管理员审核通过后系统自动恢复账号状态并发送邮件通知。

AI 70%

(2) 音乐检索与浏览：

系统须提供全文检索功能，支持用户按歌曲名称、艺术家、专辑、流派等多维度关键词进行组合搜索，并自动保存最近的搜索历史以便快速重复检索。在内容浏览层面，系统须提供热歌榜、新歌榜与收藏榜三类排行榜，按不同热度维度聚合展示全库曲目。

AI 70%

(3) 音乐播放：

系统须集成 HTML5 音频播放器，支持用户对目标曲目发起在线流媒体播放，播放器须具备进度拖拽、音量调节等基础交互控制能力。为扩展可播放曲目范围，系统须整合多个第三方音乐平台的音频源，通过服务端代理机制对播放链接进行统一解析与转发，屏蔽底层平台差异。系统须实时采集用户的播放时长数据，作为隐式反馈信号上报至服务端以供推荐引擎使用。

AI 63%

(4) 收藏与歌单管理：

系统须支持用户对单曲执行收藏与取消收藏操作，并在页面上实时反映收藏状态。除单曲收藏外，系统须提供用户自定义歌单功能，允许用户创建多个歌单并向歌单中添加或移除歌曲，同时支持歌单名称与描述的编辑及歌单删除；系统须通过唯一性约束防止同一首歌重复加入同一歌单。每位用户在注册时须自动生成默认收藏歌单，作



为收藏行为的统一承载容器。

AI 65%

(5) 个性化推荐展示：

系统须在用户登录后的主页面展示由推荐引擎生成的个性化推荐列表，推荐结果须实时从数据库中读取，并支持用户手动触发推荐刷新。推荐列表须集成播放与收藏操作入口，以便在展示场景下直接完成用户行为的闭环采集。

(6) 播放历史查询：

系统须记录用户的全量播放行为，并以时间倒序提供分页浏览能力，用户可通过播放历史页面回顾历史播放曲目。

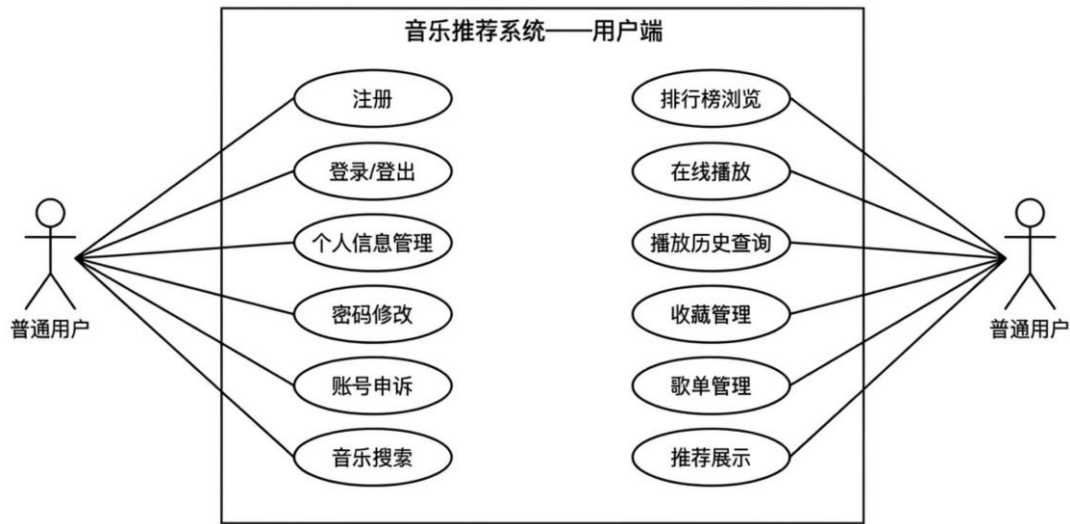


图 3-1 用户端用例图

3.2.2 管理端功能需求

AI 70%

管理端面向系统管理员，提供平台运营与内容治理所需的后台管理能力。管理员须通过独立的身份识别机制与普通用户登录流程区分，在通过身份验证后访问管理后台。如图 3-2 所示，管理端的核心用例覆盖以下四个功能域。

(1) 数据概览仪表盘

管理后台须提供实时数据统计视图，汇总展示当前注册用户总数、系统入库歌曲总数、全局收藏记录数量等平台核心运营指标，辅助管理员掌握系统整体运行状态。

(2) 用户账号管理

AI 68%

管理员须能够查看全量注册用户列表，支持按用户名、注册时间等条件进行搜索与排序。针对存在违规行为的用户，管理员须能够执行账号冻结操作，指定冻结截止时间与冻结原因；对于状态正常的账号，管理员须能解除冻结限制。系统须通过权限控制机制确保普通用户无法访问管理端的任何接口与页面。

（3）申诉审核管理

管理员须能查看所有待处理的账号申诉记录，对每条申诉作出同意或拒绝的处置决定，并可填写回复说明。系统须在申诉处置完成后通过邮件服务自动向用户发送审核结果通知；当申诉被批准时，系统须同步恢复该账号的正常访问状态。

（4）歌曲与收藏管理

管理员须能够浏览系统全量歌曲库，查看每首歌曲的元数据信息，并支持在后台直接预览播放。针对用户收藏记录，管理员须能查看所有用户的收藏关系数据，并支持对异常收藏记录执行批量清理操作。

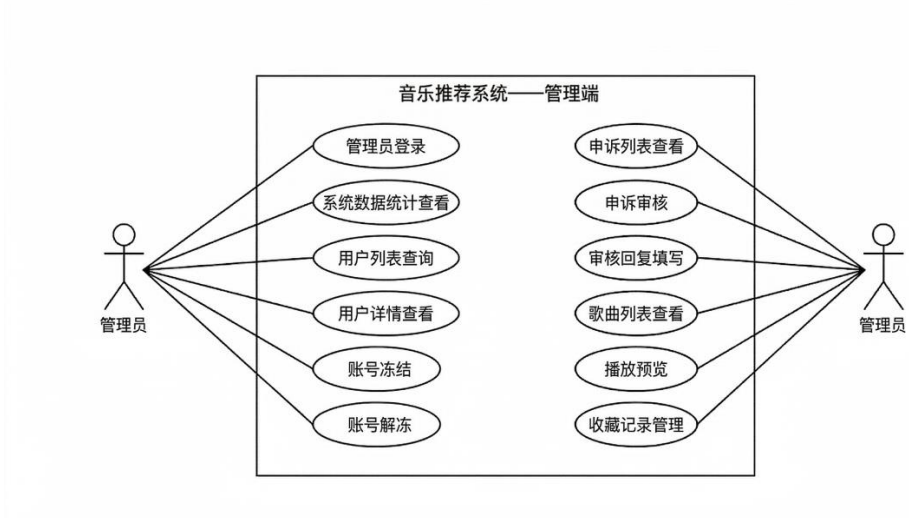


图 3-2 管理员端用例图

3.2.3 推荐引擎功能需求

推荐引擎模块（MusicMode）作为系统的算法核心，承担从候选生成到最终推荐写库的全流程计算任务，其功能需求覆盖以下五个方面。

（1）多路并行候选召回

AI 66%

系统须实现三条相互独立的候选召回通道：其一为基于 FAISS 的向量近邻检索通道，通过将用户画像向量与全量歌曲嵌入向量进行内积相似度计算，从规模达数百万量级的索引中召回高相关性候选；其二为基于 ALS 协同过滤的召回通道，利用用户-歌曲交互矩阵的隐式因子分解结果生成候选；其三为热度兜底召回通道，按歌曲热度与新鲜度综合排序补充候选，有效应对新注册用户及交互数据稀疏用户的冷启动场景。三路通道的候选结果须经去重合并后统一进入后续精排阶段。

（2）多模型异构精排

AI 68%

系统须依次调用 LightGBM、DeepFM 与 DIN 三个精排模型，对合并后的候选集进行评分排序。LightGBM 负责基于手工特征的一阶精排，承担快速筛选任务；DeepFM 负责高阶特征交叉建模；DIN 负责用户近期行为序列的注意力加权建模。三模型的输出评分须通过离线 Stacking 训练得出的权重系数加权融合，得到最终的集成推荐评分，在候选集中筛选出高置信度推荐结果。

（3）多样性重排

精排结果须经过多样性约束重排处理，对同一艺术家的入选曲目数量设定上限，防止推荐列表因模型偏好集中导致内容同质化，以提升用户的整体推荐体验。

（4）隐式反馈采集与冷却机制

AI 63%

系统须将每次推荐的歌曲记录持久化至推荐反馈表，并在用户发生播放、收藏等行为时实时更新对应记录的播放完成度与收藏标志。系统须维护滚动时间窗口内的综合反馈评分，对被用户连续忽略的曲目触发冷却屏蔽机制，在冷却期内不再向该用户重复推荐同一曲目，以动态规避无效推荐的重复曝光。

（5）定时推荐更新调度

AI 69%

系统须支持通过定时调度机制在每日指定时间自动触发全量推荐重算，确保系统每日向所有注册用户更新个性化推荐列表，并将推荐结果批量写入数据库供前端实时读取。调度流程须覆盖三路召回、精排评分、多样性重排与推荐写库的完整执行链路。

3.3 非功能性需求分析

非功能性需求从性能、安全性与可扩展性三个维度对系统的质量属性进行约束，是保障系统在实际运行环境中稳定可靠运行的基础规格要求。

### 3.3.1 性能需求

系统须在正常并发访问条件下满足以下性能基线：

（1）交互响应延迟

AI 65%

用户端页面在完成服务端数据查询与渲染后，须在可接受的响应时间内返回结果，以保证基本的交互流畅性。音乐搜索、排行榜查询等高频只读请求须通过 Redis 缓存层进行结果缓存，降低数据库的直接访问压力，使热点请求的响应延迟控制在较低水平。

（2）推荐计算吞吐能力

AI 62%

推荐引擎在执行每日全量推荐重算时，须能在预设的定时窗口内完成面向全量注册用户的三通道候选召回、多模型精排评分与推荐结果写库操作。FAISS 向量检索模块须能在毫秒量级时延内完成单用户的候选集检索，以满足批量推荐计算的整体吞吐需求。

（3）数据库并发连接能力

系统须通过 C3PO 数据库连接池对并发数据库连接进行统一管理，连接池须能在高并发请求峰值期间弹性扩展可用连接数，保证请求不因连接资源耗尽而长时间阻塞；在访问低谷期须自动回收空闲连接，降低数据库端的持续资源占用。

### 3.3.2 安全性需求

（1）注入攻击防护

系统所有涉及数据库读写的操作须采用参数化查询机制，将用户输入与 SQL 语句结构严格分离，从根本上消除 SQL 注入攻击的风险。前端表单须对用户输入进行格式与长度的合法性验证，后端接口须对关键参数执行独立的二次校验，不依赖前端单方面的输入约束。

（2）跨站脚本防护

AI 66%

系统在将用户提交的内容回显至页面时，须对特殊字符进行转义处理，防止 XSS（Cross-Site Scripting，跨站脚本攻击）注入恶意脚本至其他用户的浏览器上下文，保护用户会话数据的安全性。

（3）身份认证与权限隔离

AI 60%

系统须基于服务端会话机制维护用户登录状态，所有需要登录权限的页面与接口须在服务端进行会话有效性验证，防止未授权的直接访问。管理端与用户端须实现严格的角色权限隔离，管理员账号须通过独立的身份识别逻辑与普通用户区分，所有管理端操作接口须在权限层面对普通用户完全不可达。

（4）账号状态管控

系统须对被冻结或注销账号的用户拦截其登录请求，并向其展示明确的账号状态说明与申诉入口，防止异常账号继续访问受保护资源。管理员对账号的冻结与解冻操作须即时生效，并持久化记录冻结原因与截止时间。

3.3.3 可扩展性需求

（1）双模块解耦架构

AI 69%

MusicWeb 与 MusicMode 两个模块通过共享数据库进行数据交换，不存在直接的代码级依赖关系。推荐引擎模块的算法迭代、模型更新或调度策略调整，均不需要对前端业务代码进行任何修改，两个模块可独立进行版本演进与部署。

（2）推荐管道的模块化扩展

AI 68%

四层推荐管道（召回→精排→集成→重排）中的各层组件均具备独立替换能力。新的召回通道可在不影响精排层的前提下追加接入；新的精排模型可以作为独立分支并入集成层，通过重新训练 Stacking 元学习器更新融合权重；重排策略的调整同样不影响上游各层的运行逻辑。

（3）数据库结构的可延伸性

AI 70%

系统数据库以 10 张核心业务表为基础，各表之间通过外键关联维护数据一致性，

表结构具备向后兼容的扩展能力。未来如需引入新的用户行为信号（如评论、评分等显式反馈），可在现有表结构基础上增量扩展，无需对已有表进行破坏性改动。

### 3.4 本章小结

AI 62%  
本章从功能性与非功能性两个维度对系统进行了完整的需求分析。在功能性需求方面，分别梳理了用户端、管理端与推荐引擎三个参与者维度的核心业务用例，涵盖账号管理、音乐服务、内容管理、推荐展示，数据概览、用户治理、申诉审核、内容管理，以及多路并行召回、多模型精排、多样性重排、隐式反馈采集与定时调度。在非功能性需求方面，从交互响应延迟与推荐计算吞吐、注入攻击与跨站脚本防护及权限隔离、推荐管道模块化与数据库可延伸性三个维度明确了系统的质量属性约束。上述需求规格将作为第 4 章系统设计的直接依据，指导系统总体架构、数据库结构与推荐算法管道的具体设计决策。



## 4 系统设计

### 4.1 引言

AI 70%

第 3 章从功能性与非功能性两个维度确立了系统的需求规格，明确了用户端、管理端与推荐引擎三个参与者维度的完整用例边界及性能、安全与可扩展性约束。本章在上述需求规格的基础上，围绕系统总体架构、推荐算法流水线、数据库结构与前端交互四个核心设计域展开详细论述，将需求描述转化为可指导后续编码实现的具体技术方案。

### 4.2 系统总体架构设计

AI 64%

本系统采用双模块解耦架构，由面向用户交互的 MusicWeb 模块与面向算法计算的 MusicMode 模块协同构成，两者以 MySQL 关系型数据库为共享数据层进行集成，通过异步批处理机制实现数据闭环。如图 4-1 所示，系统整体架构由左侧 MusicWeb、右侧 MusicMode 及底部共享数据层三部分组成，数据流在两个模块之间单向流转：MusicWeb 持续将用户行为信号写入共享数据库，MusicMode 周期性地从数据库读取行为数据完成推荐计算后将结果回写，MusicWeb 再从数据库读取推荐结果并渲染至用户界面，形成完整的数据驱动推荐闭环。

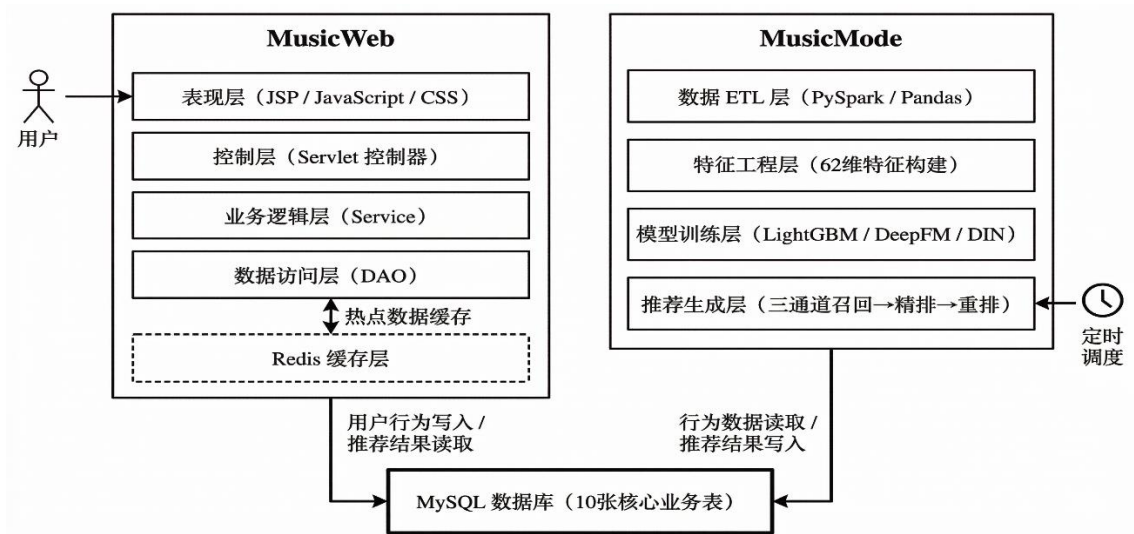


图 4-1 MusicWeb + MusicMode 双模块整体架构图



4.2.1 MusicWeb 前端模块架构

MusicWeb 是系统的用户交互载体，基于 Java 23 与 Jakarta Servlet/JSP 技术栈构建，采用经典的 MVC 三层分离架构，自上而下由表现层、控制层、业务逻辑层与数据访问层四个层次组成，各层之间职责边界清晰，层间依赖关系单向向下，以保证模块内部的低耦合性与高可维护性。

(1) 表现层

AI 68%  
表现层由 JSP 动态页面、JavaScript 行为脚本与 CSS 样式表共同构成，负责向用户浏览器输出可交互的 HTML 界面。页面集合涵盖首页、用户中心、歌曲搜索、歌单详情、播放历史、用户设置与账号申诉等业务场景，以及管理员后台的仪表盘、用户列表、申诉审核等运营页面。表现层通过异步请求机制与控制层交互，实现推荐列表刷新、收藏状态更新等无刷新动态操作，降低整页加载带来的用户感知延迟。

(2) 控制层

控制层由一组按业务职能划分 Servlet 控制器组成，统一接收表现层发起的 HTTP 请求并完成请求的分发与调度。按业务域归类，控制器集合覆盖用户认证（注册、登录、登出）、个人信息管理（资料修改、密码变更、账号注销）、音乐服务（搜索、音频代理、播放链接获取）、行为上报（播放时长记录、完播率上报）、内容管理（收藏操作、歌单增删改）、推荐服务（推荐列表获取、手动刷新触发）及管理端（用户管理、申诉审核）七个功能域。控制层不直接处理数据库操作，仅负责参数解析、权限校验与流程调度，以保持自身的轻量化。

(3) 业务逻辑层

AI 69%  
业务逻辑层负责承载跨 DAO 的复合操作与推荐服务的协调逻辑。推荐服务组件负责从数据库读取由推荐引擎预计算的推荐结果，并结合用户当前的收藏状态与历史播放记录对推荐列表进行过滤与补全，最终向控制层返回可直接渲染的推荐歌曲列表。

(4) 数据访问层

数据访问层通过 C3P0 连接池获取数据库连接，对系统全部 10 张核心业务表的读写操作进行统一封装，向上层屏蔽 SQL 语句细节与结果集映射逻辑。各 DAO 组件

按业务实体划分，分别负责用户信息、歌曲元数据、播放历史、收藏关系、歌单数据、推荐记录与反馈数据等核心实体的持久化操作，全部 SQL 操作均采用参数化查询机制，从根本上防范 SQL 注入攻击。

(5) 缓存层

AI 54%

系统引入 Redis 作为热点数据缓存层，对用户收藏状态等读多写少的数据进行内存级缓存，减少数据库的重复查询压力。缓存层位于数据访问层之上，采用旁路缓存策略：读请求优先命中缓存，缓存未命中时回源数据库并更新缓存；写请求同步更新数据库并使相关缓存键失效，保证缓存数据与数据库数据的一致性。

4.2.2 MusicMode 推荐引擎模块架构

AI 66%

MusicMode 是系统的算法计算核心，基于 Python 与 PyTorch 深度学习框架构建，以定时批处理模式运行，不提供实时在线服务接口。模块内部自上而下由数据 ETL 层、特征工程层、模型训练层与推荐生成层四个功能层次组成，各层的产出物通过磁盘序列化文件或数据库表进行跨层传递，支持各层的独立执行与增量更新。

(1) 数据 ETL 层

AI 70%

ETL 层负责从 KKBOX 公开数据集与 MySQL 业务数据库中抽取原始数据，执行数据清洗、格式规范化与样本构建操作。对于 KKBOX 的大规模歌曲数据集，系统采用 PySpark 分布式计算框架通过 JDBC 接口完成全量批量导入，并基于歌曲唯一标识进行增量去重，避免重复写入。对于模型训练样本的构建，ETL 层以用户 30 天内的重复收听行为作为正样本标准，以等比例随机负采样生成负样本，输出结构化的训练样本集供特征工程层使用。

(2) 特征工程层

AI 69%

特征工程层基于 ETL 层输出的原始样本，构建面向精排模型的 62 维特征向量。稠密特征涵盖用户行为统计、歌曲属性统计、用户-歌曲交互匹配、时序特征、近期交互信号、用户-艺术家偏好、歌单统计、记忆衰减权重、滚动窗口统计及 SVD 嵌入向量共 10 个特征组。为防止高基数类别特征引发的目标泄漏问题，所有类别型特征的目标编码均采用 OOF 折外预测策略：在 K 折交叉验证框架下，每折的编码值仅由

其余 K-1 折的样本统计得出，验证集与测试集的编码则通过全训练集统计值计算，从而在训练阶段彻底隔断标签信号的反向渗漏路径。

（3）模型训练层

AI 69%  
模型训练层对 LightGBM、DeepFM 与 DIN 三个精排模型进行独立训练，并通过 Stacking 集成框架生成最终的集成模型配置。LightGBM 使用 62 维特征中的 32 维子集进行训练；DeepFM 与 DIN 均使用全部 49 维输入并通过 GPU 加速进行深度模型训练。三个基模型在验证集上的 OOF 预测概率作为 Stacking 元特征，输入逻辑回归元学习器以拟合最优融合权重。模型训练完成后，各模型权重文件与集成配置分别序列化至磁盘，供推荐生成层在线调用。

（4）推荐生成层

推荐生成层实现四层推荐管道的完整在线执行逻辑，按照三通道候选召回、LightGBM 一阶精排、三模型加权集成评分与多样性重排的顺序依次处理，最终生成每位用户的个性化推荐列表并批量写入数据库。

4.2.3 双模块交互与部署设计

MusicWeb 与 MusicMode 两个模块之间不存在直接的进程间通信依赖，采用以共享 MySQL 数据库为唯一数据交换媒介的松耦合集成模式。这一设计选择具有以下三方面工程优势：其一，两个模块可完全独立部署、独立重启与独立扩展，任一模块的故障不直接传播至另一模块；其二，数据库作为持久化消息载体，天然具备断点续传能力——即使推荐引擎在某次执行中途中断，已写入数据库的推荐结果仍可被前端正常读取；其三，调试与运维工程师可直接通过数据库查询工具观察模块间的数据流转状态，降低了跨模块问题的排查复杂度。

AI 68%  
在部署层面，MusicWeb 打包为标准的 WAR 文件，部署于 Apache Tomcat 应用服务器，以持续进程形式提供在线服务。MusicMode 以 Python 脚本形式运行，由操作系统级别的定时调度任务在每日指定时间自动触发推荐重算流程，无需常驻进程，以最小化资源占用。Redis 缓存服务与 MySQL 数据库服务以独立进程形式部署于同一台服务器，由 MusicWeb 启动脚本负责服务的有序启动与就绪检测，确保各依赖服

务在 Web 应用接受请求前均已完全就绪。

4.3 推荐算法流水线设计

**AI 70%**  
推荐算法流水线是本系统实现“千人千面”个性化推荐的核心机制。如图 4-2 所示，流水线采用四层串行架构，依次由召回层、精排层、集成层与重排层构成。各层承担不同粒度的候选筛选职责：召回层以高覆盖率为目标，在毫秒级时延内从全量曲库生成数百个候选；精排层以高精度为目标，通过特征工程驱动的梯度提升模型对候选集进行压缩排序；集成层通过多模型异构融合进一步提升推荐评分的准确性；重排层在最终输出前施加多样性约束，防止推荐列表内容同质化。四层管道分工明确、接口清晰，支持各层独立迭代而不影响相邻层的运行逻辑。

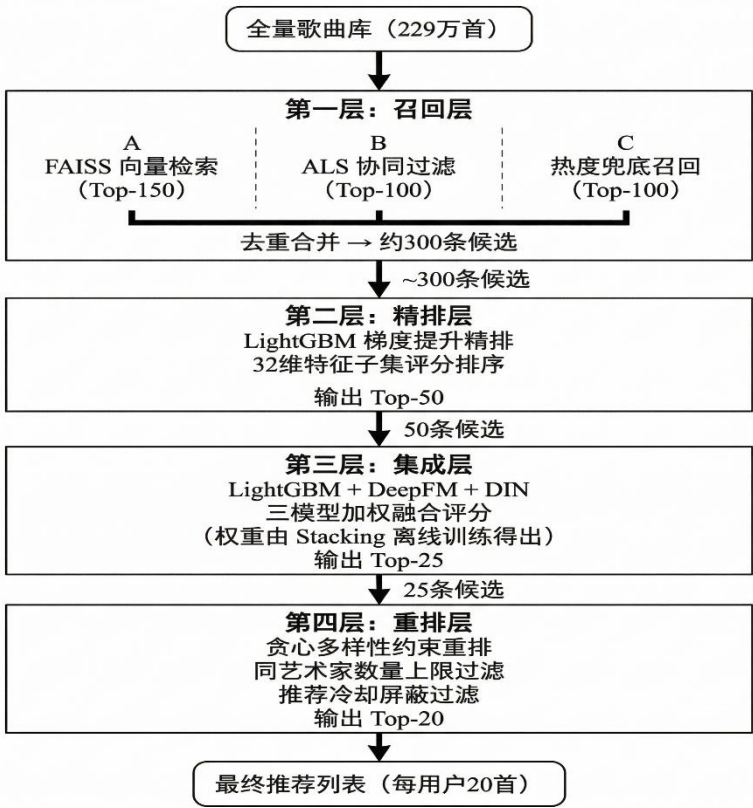


图 4-2 四层推荐管道总体流程图

4.3.1 召回层设计

**AI 70%**  
召回层的核心设计目标是在计算效率约束下实现对高质量候选歌曲的高覆盖率捕获。系统设计了三条相互独立、特性互补的召回通道，通过并行执行后去重合并的

方式构建统一候选集。如图 4-3 所示，三通道分别基于向量近邻检索、协同过滤与热度排序三种不同的信息源进行候选生成，以覆盖不同类型的用户兴趣信号。

通道 A：FAISS 向量检索召回

AI 64%

向量检索通道基于用户行为历史构建用户兴趣向量，通过 FAISS 内积索引在全量歌曲向量空间中执行近似最近邻搜索，召回与用户兴趣向量内积相似度最高的 Top-150 候选歌曲。该通道的设计逻辑在于：用户的历史播放偏好在语义嵌入空间中可以被有效压缩为一个固定维度的兴趣表示向量，与该向量距离近的歌曲在语义特征分布上与用户偏好高度吻合，因而具有较高的被播放概率。向量索引在离线阶段对全量歌曲库进行预计算与持久化，在线召回阶段仅执行向量查询操作，查询时延极低，可满足批量推荐场景对吞吐效率的要求。

通道 B：ALS 协同过滤召回

AI 57%

协同过滤通道基于 ALS 隐式反馈矩阵分解模型，利用全量用户-歌曲交互矩阵中的统计规律，为每位用户生成个性化的协同过滤推荐列表，召回 Top-100 候选。该通道捕获的是基于群体行为相似性的推荐信号——与目标用户历史行为模式相近的其他用户所偏好的歌曲，即便尚未被目标用户接触，也具有较高的潜在兴趣概率。ALS 模型在系统离线训练阶段完成矩阵分解，将用户与歌曲的隐向量因子分别序列化存储，在线推荐时通过用户因子向量与歌曲因子矩阵的内积排序完成快速候选生成。

通道 C：热度兜底召回

AI 61%

热度兜底通道按歌曲热度指标与发行时间的复合排序规则，从全库中选取 Top-100 热门歌曲作为补充候选。该通道的设计初衷在于解决冷启动场景下的推荐可用性问题：对于新注册用户或历史交互记录极度稀疏的用户，向量检索与协同过滤两个数据驱动通道由于缺乏足够的用户行为信号而难以生成有效的个性化候选；热度兜底通道通过平台级的高热歌曲填充候选集，保障此类用户的推荐列表具备基本的内容质量与时效性。同时，对于历史行为丰富的活跃用户，热度兜底候选与前两个通道的结果进行去重合并后，也可对覆盖面起到有效的补充作用。

候选合并与去重。三通道各自独立生成候选列表后，系统以歌曲唯一标识为键执



行集合去重操作，过滤用户近期已播放及处于推荐冷却期的歌曲，最终合并形成约 300 条不重复的候选集，传递至下游精排层。

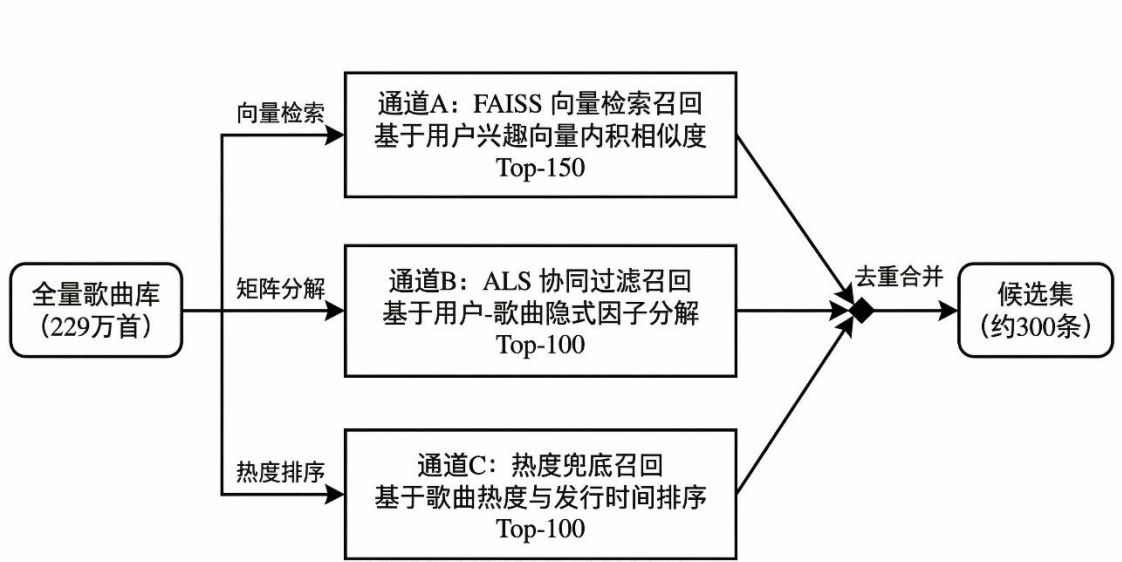


图 4-3 三通道并行召回结构图

#### 4.3.2 粗排层与精排层设计

召回层输出的约 300 条候选歌曲虽经过多路信号筛选，但评分粒度较粗，候选间的相对优劣尚未得到精细化区分。如图 4-2 所示，粗排层在召回层之后承担第一阶段的候选压缩任务，以 LightGBM 梯度提升模型对全部约 300 条候选执行基于手工特征的快速评分，按预测播放概率降序截取 Top-50，将候选规模从约 300 条压缩至 50 条，再传递至下游精排集成层。

粗排层选用 LightGBM 而非 DeepFM 或 DIN 作为一阶筛选器，出于以下工程考量：LightGBM 基于直方图近似算法执行决策树分裂，在推断阶段对大批量候选的评分吞吐效率显著高于需要前向神经网络计算的深度模型；在候选规模 300 的场景下，LightGBM 的推断延迟可控制在极低水平，满足批量推荐计算的效率要求。相对而言，DeepFM 与 DIN 两个深度模型具备更强的非线性建模能力与序列兴趣捕获能力，但其推断计算量更大，若对全部 300 条候选均执行深度模型评分将大幅增加单用户推荐的计算开销。因此，系统将 DeepFM 与 DIN 的调用范围限定在经 LightGBM 粗排筛

选后的 Top-50 候选上，在精排集成层中发挥其精度优势，从而实现整体管道在效率与精度之间的最优平衡。

AI 69%

在特征输入设计上，粗排层从 62 维完整特征管道中选取包含 32 维的粗排特征子集。子集优先保留与用户-歌曲相关性最强的行为统计特征与交互匹配特征，剔除维度较高的 SVD 嵌入分量，以缩减决策树分裂时的特征搜索空间。LightGBM 粗排模型在 KKBOX 验证集上的实测 AUC 为 0.7063，为精排集成层提供具备基础精度的候选先验。

4.3.3 集成层设计

AI 69%

精排集成层接收粗排层输出的 Top-50 候选，对其分别调用 LightGBM、DeepFM 与 DIN 三个模型独立评分，再以 Stacking 离线训练得出的 SLSQP 优化加权系数融合三者输出，按集成评分降序截取 Top-25 传递至重排层。如图 4-2 所示，精排集成层是本系统推荐管道的核心精度提升环节，三个模型在此层以互补的建模视角对同一批 Top-50 候选展开协同评分。

（1）三模型的分工与互补性

AI 65%

三个模型对用户偏好的建模维度存在本质差异，共同构成集成层的互补性基础。

LightGBM 在精排集成层中复用粗排层已训练的模型权重，对 Top-50 候选重新输出评分概率，提供基于手工统计特征的可解释性评分基准。

AI 70%

DeepFM 在 Top-50 候选上执行前向推断，基于共享嵌入层同时建模低阶因子分解特征交叉与高阶深度网络非线性变换，能够自动学习 LightGBM 手工特征体系未能显式覆盖的隐式特征组合关系，对候选歌曲的属性匹配度提供更丰富的评分视角。DeepFM 在 KKBOX 验证集上的实测 AUC 为 0.7548，显著高于 LightGBM 的 0.7063，说明其在高阶特征建模上具备明确的精度优势。

AI 69%

DIN 在 Top-50 候选上执行注意力加权推断，通过激活单元对用户历史行为序列中与当前候选歌曲属性相关的交互记录赋予更高注意力权重，动态捕获用户近期局部兴趣的漂移方向。相较于 LightGBM 与 DeepFM 将用户历史行为压缩为固定统计特征的处理方式，DIN 以序列注意力机制保留了行为信号的时序细节，对用户兴趣动态变



化的感知能力更强。DIN 在 KKBOX 验证集上实测 AUC 为 0.7602，为三个基模型中性能最优者。

(2) Stacking 权重训练机制

AI 62%

三模型的融合权重通过离线 Stacking 框架学习。在离线训练阶段，三个基模型分别基于相同的 K 折交叉验证划分生成各自在验证集上的 OOF 折外预测概率；以三组 OOF 预测向量构成 Stacking 元特征矩阵，以验证集真实标签为优化目标，通过约束优化算法在权重非负且满足归一化约束的条件下最大化加权集成 AUC，得到三模型最优权重系数  $w_1$ 、 $w_2$ 、 $w_3$ 。

AI 63%

在线推荐阶段，集成层对 Top-50 候选中的每首歌曲调用三模型各自评分后，按下式(4-1)计算集成推荐得分：

$$\text{score}_{\text{ensemble}} = w^1 \cdot \text{score}_{\text{lgbm}} + w^2 \cdot \text{score}_{\text{deepfm}} + w^3 \cdot \text{score}_{\text{din}} \quad (4-1)$$

式中： $\text{score}_{\text{lgbm}}$ 、 $\text{score}_{\text{deepfm}}$ 、 $\text{score}_{\text{din}}$  分别为 LightGBM、DeepFM 与 DIN 对目标候选歌曲输出的预测播放概率； $w_1$ 、 $w_2$ 、 $w_3$  为离线 Stacking 优化训练所得固定权重系数，满足  $w_1 + w_2 + w_3 = 1$  且  $w_i \geq 0$  ( $i = 1, 2, 3$ )。由于 DIN 为三者中验证集 AUC 最高的单模型，优化器在约束框架下自然向 DIN 分配最高权重；LightGBM 与 DeepFM 凭借差异化的建模视角贡献互补的边际增益。集成模型在 KKBOX 验证集上的 StackingAUC 最终达到 0.7607，相较于最优单模型 DIN 的 0.7602 进一步提升，验证了异构深度模型集成策略的有效性。

4.3.4 重排层设计

AI 70%

重排层接收集成层基于式 (4-1) 评分后输出的 Top-25 候选，在最终输出前施加多样性约束与推荐冷却过滤两类业务规则，将候选列表压缩至每用户 Top-20 并写入数据库。如图 4-2 所示，重排层是四层管道的最后一级，其核心目的是在维持集成评分精度的前提下，修正推荐列表的内容分布，防止模型偏好导致的同质化曝光。

(1) 贪心多样性约束

AI 67%

集成评分以单曲维度的播放概率为优化目标，缺乏对推荐列表整体内容分布的全局感知。当三个基模型对某一特定艺术家或流派存在共同的高置信度偏好时，Top-25

候选中可能聚集大量来自同一艺术家的歌曲，造成列表内容单一。重排层采用贪心序列选择策略：按集成评分从高到低逐一审查 Top-25 候选，当某候选歌曲所属艺术家的已选数量达到预设上限时，跳过该候选继续扫描下一条；直至选出满足艺术家多样性约束的 20 首歌曲为止。该策略以单次线性扫描的极低计算开销在相关性与多样性之间实现显式均衡，在数学形式上与 MMR（Maximal Marginal Relevance，最大边际相关性）准则具有等价性。

(2) 推荐冷却过滤

AI 65%

系统在推荐反馈表中为每对用户-歌曲维护冷却截止日期字段，记录因连续被用户忽略而触发屏蔽的歌曲及其冷却期限。重排层在贪心筛选的同时，过滤所有当前处于冷却期内的候选歌曲，防止对持续不感兴趣的内容进行重复曝光。冷却期的触发条件与时长由上游的反馈处理逻辑根据用户连续忽略天数动态计算，重排层仅执行只读过滤操作，不参与冷却策略的计算逻辑，保持各层职责的单一性。

AI 68%

经多样性重排与冷却过滤后，最终输出的每用户 Top-20 推荐列表兼顾评分精度与内容多样性，由推荐生成模块批量写入数据库推荐表，供 MusicWeb 前端在用户访问时实时读取与渲染，完成四层管道的完整执行闭环。

4.4 数据库设计

4.4.1 概念结构设计

AI 66%

概念结构设计阶段通过实体—关系模型（E-R 模型）对系统业务数据进行抽象，明确各实体的属性及其相互关联，为后续逻辑结构设计奠定基础。

本系统共梳理出 10 个核心实体，分别为：用户表、歌曲表、播放历史表、用户歌单表、歌单歌曲关联表、推荐结果表、推荐反馈表、用户口味反馈表、申诉表以及用户屏蔽内容表。

各实体间的核心关联关系如下：用户与歌曲之间通过播放历史表形成多次播放记录的关联；用户与歌单之间为一对多关系，一个用户可创建多个歌单，歌单与歌曲之间通过歌单歌曲关联表构成多对多关系；用户与推荐结果表之间为一对多关系，系统

每日为每位用户生成若干推荐记录；推荐反馈表依附于推荐结果表，记录用户对推荐歌曲的真实交互行为；用户口味反馈表、申诉表及用户屏蔽内容表均以用户为核心，分别记录偏好标签、违规申诉及内容屏蔽信息。系统核心实体关系如图 4-4 所示。

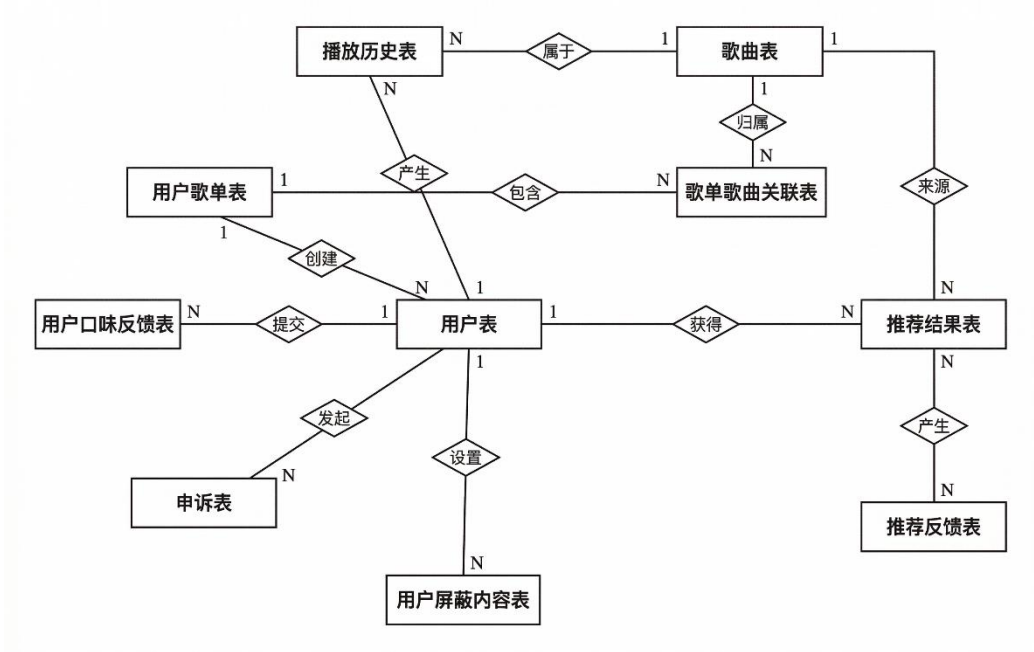


图 4-4 系统核心实体关系图

4. 4. 2 逻辑结构设计

AI 69%

系统数据库采用 MySQL，共包含 10 张业务表，通过外键约束与联合索引保证数据一致性与查询效率。各表的逻辑结构如表 4-1 至表 4-10 所示。

如表 4-1 所示，用户表存储系统全量注册用户的账号基本信息与状态标记。

表 4-1 用户表结构

| 字段名      | 数据类型         | 约束                | 说明   |
|----------|--------------|-------------------|------|
| id       | INT          | PK,AUTO_INCREMENT | 用户主键 |
| username | VARCHAR(50)  | UNIQUE NOT NULL   | 用户名  |
| password | VARCHAR(255) | NOT NULL          | 密码   |
| email    | VARCHAR(100) | —                 | 邮箱   |
| nickname | VARCHAR(50)  | —                 | 昵称   |
| phone    | VARCHAR(20)  | —                 | 手机号  |

表 4-1（续表） 用户表结构

| 字段名               | 数据类型         | 约束            | 说明                    |
|-------------------|--------------|---------------|-----------------------|
| status            | ENUM         | NOT NULL      | active/frozen/deleted |
| frozen_until      | TIMESTAMP    | —             | 冻结截止时间                |
| frozen_reason     | VARCHAR(255) | —             | 冻结原因                  |
| deleted_at        | TIMESTAMP    | —             | 逻辑删除时间                |
| preferred_genres  | VARCHAR(255) | —             | 偏好流派                  |
| preferred_artists | VARCHAR(255) | —             | 偏好艺术家                 |
| create_time       | TIMESTAMP    | DEFAULT NOW() | 注册时间                  |

歌曲表存储全量歌曲的元数据信息，基于 KKBOX 数据集导入并经多源元数据补全，如表 4-2 所示。

表 4-2 歌曲表结构

| 字段名            | 数据类型         | 约束                | 说明               |
|----------------|--------------|-------------------|------------------|
| id             | INT          | PK,AUTO_INCREMENT | 歌曲主键             |
| title          | VARCHAR(255) | NOT NULL          | 歌名               |
| artist         | VARCHAR(255) | —                 | 歌手               |
| album          | VARCHAR(255) | —                 | 专辑               |
| duration       | INT          | —                 | 时长（秒）            |
| genre          | VARCHAR(100) | —                 | 流派               |
| genre_ids      | VARCHAR(255) | —                 | KKBOX 原始流派 ID 列表 |
| language       | VARCHAR(10)  | —                 | 语言代码             |
| release_year   | INT          | —                 | 发行年份             |
| kkbox_id       | VARCHAR(100) | —                 | KKBOX 原曲 ID      |
| popularity     | INT          | DEFAULT 0         | 歌曲热度             |
| origin_country | VARCHAR(50)  | —                 | 原产地              |
| cover_image    | VARCHAR(500) | —                 | 封面图片路径           |

播放历史表记录用户每次播放行为，是推荐引擎特征工程与召回层的核心数据源，如表 4-3 所示。

表 4-3 播放历史表结构

| 字段名       | 数据类型      | 约束                   | 说明    |
|-----------|-----------|----------------------|-------|
| id        | INT       | PK,AUTO_INCREMENT    | 主键    |
| user_id   | INT       | FK→users.id NOT NULL | 用户 ID |
| song_id   | INT       | FK→songs.id NOTNULL  | 歌曲 ID |
| play_time | TIMESTAMP | NOT NULL             | 播放时间  |

用户歌单表与歌单歌曲关联表共同实现用户自定义歌单的多对多数据模型，其中 is\_default=1 的歌单即为用户的默认收藏夹，如表 4-4、表 4-5 所示。

表 4-4 用户歌单表结构

| 字段名         | 数据类型         | 约束                   | 说明       |
|-------------|--------------|----------------------|----------|
| id          | INT          | PK,AUTO_INCREMENT    | 主键       |
| user_id     | INT          | FK→users.id NOT NULL | 创建者 ID   |
| name        | VARCHAR(100) | NOT NULL             | 歌单名称     |
| description | TEXT         | —                    | 歌单描述     |
| cover_image | VARCHAR(500) | —                    | 歌单封面     |
| is_default  | TINYINT(1)   | DEFAULT 0            | 是否为默认收藏夹 |
| create_time | TIMESTAMP    | DEFAULT NOW()        | 创建时间     |
| update_time | TIMESTAMP    | DEFAULT NOW()        | 最后更新时间   |

表 4-5 歌单歌曲关联表结构

| 字段名         | 数据类型      | 约束                               | 说明    |
|-------------|-----------|----------------------------------|-------|
| id          | INT       | PK, AUTO_INCREMENT               | 主键    |
| playlist_id | INT       | FK→user_playlists.id<br>NOT NULL | 歌单 ID |
| song_id     | INT       | FK→songs.id NOT NULL             | 歌曲 ID |
| add_time    | TIMESTAMP | DEFAULT NOW()                    | 添加时间  |

推荐结果表存储推荐引擎每日批量生成的个性化推荐记录，由 MusicWeb 前端实时读取渲染，如表 4-6 所示。

表 4-6 推荐结果表结构

| 字段名         | 数据类型        | 约束                   | 说明      |
|-------------|-------------|----------------------|---------|
| id          | INT         | PK, AUTO_INCREMENT   | 主键      |
| user_id     | INT         | FK→users.id NOT NULL | 目标用户 ID |
| song_id     | INT         | FK→songs.id NOT NULL | 推荐歌曲 ID |
| score       | DOUBLE      | —                    | 集成推荐得分  |
| source_type | VARCHAR(20) | DEFAULT 'deepfm'     | 推荐来源标记  |
| create_time | DATETIME    | DEFAULT NOW()        | 推荐生成时间  |

推荐反馈表追踪每条推荐记录的用户行为响应，是推荐冷却机制与反馈闭环的核心支撑表，字段涵盖播放、收藏、完播率及综合评分等隐式反馈信号，如表 4-7 所示。



表 4-7 推荐反馈表结构

| 字段名                     | 数据类型       | 约束                   | 说明             |
|-------------------------|------------|----------------------|----------------|
| id                      | INT        | PK,AUTO_INCREMENT    | 主键             |
| user_id                 | INT        | FK→users.id NOT NULL | 用户 ID          |
| song_id                 | INT        | FK→songs.id NOT NULL | 歌曲 ID          |
| recommend_date          | DATE       | NOT NULL             | 推荐日期           |
| was_played              | TINYINT(1) | DEFAULT 0            | 是否已播放<br>(0/1) |
| play_completion         | FLOAT      | —                    | 播放完成度<br>(0~1) |
| was_favorited           | TINYINT(1) | DEFAULT 0            | 是否已收藏<br>(0/1) |
| consecutive_ignore_days | INT        | DEFAULT 0            | 连续忽略天<br>数     |
| feedback_score          | FLOAT      | DEFAULT 0            | 综合反馈得<br>分     |
| cooldown_until          | DATE       | —                    | 推荐冷却期<br>截止日   |
| created_at              | TIMESTAMP  | DEFAULT NOW()        | 记录创建时<br>间     |
| updated_at              | TIMESTAMP  | DEFAULT NOW()        | 最后更新时<br>间     |

用户口味反馈表归档用户对推荐结果的显式满意度表态，同一用户同一天多次提交取最新值，作为推荐引擎评分调整的显式信号补充，如表 4-8 所示。

表 4-8 用户口味反馈表结构

| 字段名           | 数据类型         | 约束                   | 说明      |
|---------------|--------------|----------------------|---------|
| id            | INT          | PK,AUTO_INCREMENT    | 主键      |
| user_id       | INT          | FK→users.id NOT NULL | 用户 ID   |
| feedback_date | DATE         | NOT NULL             | 反馈日期    |
| satisfaction  | ENUM         | NOT NULL             | 用户满意度   |
| genres_added  | VARCHAR(255) | —                    | 新增偏好流派  |
| artists_added | VARCHAR(255) | —                    | 新增偏好艺术家 |
| created_at    | TIMESTAMP    | DEFAULT NOW()        | 记录创建时间  |

申诉表记录用户对账号冻结或注销状态的申诉请求及管理员处置结果，如表 4-9 所示。

表 4-9 申诉表结构

| 字段名           | 数据类型         | 约束                    | 说明                        |
|---------------|--------------|-----------------------|---------------------------|
| id            | INT          | PK,<br>AUTO_INCREMENT | 主键                        |
| username      | VARCHAR(50)  | NOT NULL              | 申诉账号名                     |
| user_id       | INT          | FK→users.id           | 关联用户 ID                   |
| appeal_type   | ENUM         | NOT NULL              | frozen/deleted            |
| reason        | TEXT         | —                     | 申诉理由                      |
| contact_email | VARCHAR(100) | —                     | 联系邮箱                      |
| status        | ENUM         | DEFAULT 'pending'     | pending/approved/rejected |
| admin_reply   | TEXT         | —                     | 管理员回复内容                   |
| create_time   | TIMESTAMP    | DEFAULT NOW()         | 申诉创建时间                    |
| update_time   | TIMESTAMP    | DEFAULT NOW()         | 最后处理时间                    |

用户屏蔽内容表记录用户对特定流派或艺术家的软屏蔽操作，通过递增冷却与衰

减回归机制动态管理屏蔽有效期，使重排层可在生成推荐时自动过滤用户明确排斥的内容类别，如表 4-10 所示。

表 4-10 用户屏蔽内容表结构

| 字段名           | 数据类型         | 约束                   | 说明         |
|---------------|--------------|----------------------|------------|
| id            | INT          | PK,AUTO_INCREMENT    | 主键         |
| user_id       | INT          | FK→users.id NOT NULL | 用户 ID      |
| block_type    | ENUM         | NOT NULL             | 屏蔽类型       |
| block_value   | VARCHAR(100) | NOT NULL             | 屏蔽的流派名或歌手名 |
| block_count   | INT          | DEFAULT 1            | 屏蔽次数       |
| blocked_at    | TIMESTAMP    | DEFAULT NOW()        | 最近一次屏蔽时间   |
| blocked_until | DATE         | NOT NULL             | 屏蔽到期日      |
| is_active     | TINYINT(1)   | DEFAULT 1            | 是否生效 (1/0) |

10 张表之间通过外键约束保证引用完整性，关键索引如下：播放历史表在 (user\_id, play\_time) 上建立复合索引以加速时序特征查询；推荐结果表在 (user\_id, score) 上建立索引以保障推荐列表的高效读取；推荐反馈表在 (user\_id, recommend\_date) 上建立复合索引以支持日粒度反馈统计；用户口味反馈表在 (user\_id, is\_active) 上建立联合索引以加速重排层的屏蔽过滤查询。

### 4.5 前端交互设计

前端交互层是系统与用户之间的直接桥梁，其设计质量直接影响用户对推荐结果的感知与接受程度。本节遵循 MVC 架构原则，以 JSP 作为视图层、以 Servlet 作为控制层，围绕"推荐展示—用户交互—行为采集"的核心链路，对页面总体结构、核心功能页面以及推荐交互反馈机制进行详细说明。

#### 4.5.1 页面总体结构设计

系统采用统一的页面框架，所有功能页面共享同一套顶部导航栏与底部版权区。

顶部导航栏提供全局功能入口，涵盖音乐发现、个人推荐、歌单管理与个人中心四大模块；底部版权区承载项目声明信息。主内容区根据当前功能模块动态渲染对应的视图，各页面样式以独立样式文件进行维护，确保样式隔离与可维护性。

AI 59%

系统页面按访问权限分为两类：公开访问页面（主页、音乐搜索、歌曲详情等）与登录后可用页面（个人推荐、歌单管理、个人中心等）。两类页面均通过统一的权限拦截机制进行校验，未登录用户访问受保护页面时将被自动重定向至登录入口，确保用户数据安全。系统整体页面导航层次如图 4-5 所示。

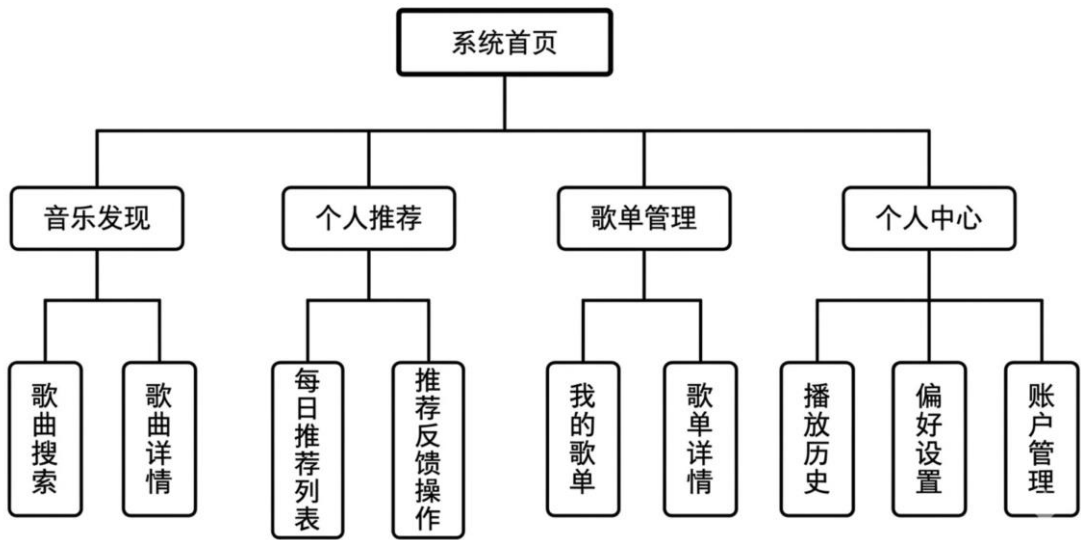


图 4-5 系统页面导航层次结构图

AI 70%

图 4-5 以树形层次清晰呈现了系统各功能页面的组织关系。一级节点为四大功能模块，二级节点为各模块下的具体页面。用户可通过顶部导航栏直达各一级模块，再经页面内部交互跳转至对应二级页面，路径清晰，交互层级不超过三级，有效降低用户的操作认知负担。

#### 4.5.2 核心功能页面设计

系统核心功能页面共分为四个模块，各模块在交互设计上均以简洁高效为原则，力求以最少的操作步骤完成目标任务。

##### （1）音乐发现模块

AI 67%

音乐发现模块为系统的公开主入口。页面上方展示热门歌曲排行与最新上架内容，

用户可通过关键词搜索框按歌名或歌手名检索歌曲。搜索结果以卡片形式排列，每张卡片包含封面图、歌名、歌手及快捷操作按钮（播放、收藏、加入歌单）。点击卡片后进入歌曲详情页，用户可查看完整信息并触发播放，系统同步记录本次播放行为至播放历史表。

（2）个性化推荐模块

AI 65%

个性化推荐模块是系统的核心展示界面，负责呈现推荐算法为当前用户生成的每日推荐结果。页面以有序列表形式展示当日 20 首推荐歌曲，每条记录包含歌曲基本信息及操作区域（播放、收藏、反馈不喜欢）。用户在此页面的所有交互行为均通过异步请求实时上报至后端，后端将其写入推荐反馈记录，作为下一轮推荐迭代的训练信号。

（3）歌单管理模块

AI 54%

歌单管理模块支持用户创建、重命名与删除自定义歌单，并可向歌单中批量添加或删除歌曲。歌单列表页以卡片网格呈现用户的所有歌单，点击卡片后进入歌单详情页，以分页表格展示歌单内全部歌曲，每行末尾提供移除操作入口。歌单数据经由数据访问层持久化至数据库，操作结果通过页面消息提示即时反馈。

（4）个人中心模块

AI 68%

个人中心模块聚合了用户账户管理与行为记录查询功能。用户可在此修改账户信息、查看播放历史与收藏列表、配置流派或歌手屏蔽偏好，并可对认为不当的内容提交申诉请求。管理员账户在个人中心额外具备用户管理与歌曲管理两项后台入口，支持对系统核心数据的集中维护与审核处理。

4.5.3 推荐交互与行为反馈设计

AI 65%

推荐交互设计的核心目标是在用户操作自然流畅的前提下，高效采集行为信号并形成持续优化的反馈闭环。系统对用户与推荐歌曲之间的三类关键交互进行追踪：播放行为、收藏行为与完播率。

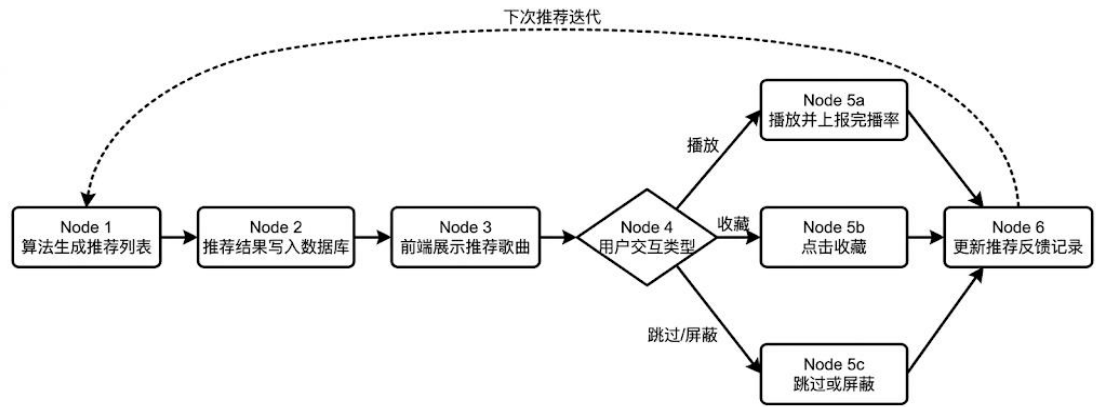
当用户点击播放推荐歌曲时，前端计时器开始实时记录播放进度；用户离开或切换歌曲时，进度数据通过异步请求上报至后端，后端计算完播率后将结果写入推荐反

馈记录。完播率低于 20%的播放行为被标记为跳曲，系统据此对该歌曲或对应歌手在后续推荐中进行降权处理。收藏行为则作为强正向信号，提升相似风格内容的推荐权重。

AI 69%

此外，用户可在个人中心主动声明对特定流派或歌手的屏蔽偏好。系统对屏蔽内容采用递增冷却机制：每次屏蔽操作累加屏蔽计数，冷却到期日随计数递增而延长，到期后屏蔽状态自动失效，避免单次操作造成永久性内容排除。上述多维度行为反馈共同构成推荐系统的在线学习闭环，使推荐结果随用户使用时间的增长持续贴近个人偏好。推荐交互行为反馈闭环流程图如图 4-6 所示。

图 4-6 推荐交互行为反馈闭环流程图



4.6 本章小结

AI 70%

在总体架构设计方面，系统采用 MVC 分层架构，以 Java Web 技术栈构建前端交互层，以 Python 算法引擎承担推荐计算任务，两者通过共享 MySQL 数据库实现数据联动，C3P0 连接池与 Redis 缓存分别保障了数据库访问的稳定性与高频读取的响应效率。

AI 70%

在推荐算法流水线设计方面，系统构建了召回、粗排、精排集成、重排四层递进式处理链路。召回层融合 FAISS 向量检索、ALS 协同过滤与热度兜底三路候选，粗排层由 LightGBM 完成高效初筛（Top-300→Top-50），精排集成层引入 DeepFM 与 DIN 对深度特征交叉与用户兴趣序列进行建模，最终经 Stacking 集成达到验证集 AUC 0.7603；重排层通过多样性约束与冷却屏蔽机制对排序结果进行后处理，兼顾精准性

与多样性。

AI 70%

在数据库设计方面，系统围绕用户与歌曲两个核心实体，设计了涵盖行为记录、推荐结果、反馈追踪、歌单管理及内容屏蔽在内的共 10 张业务表，各表通过外键约束维护数据一致性，推荐反馈表与用户屏蔽内容表共同支撑了系统的在线反馈闭环机制。

在前端交互设计方面，系统以四大功能模块为主线组织页面导航层次，个性化推荐模块通过异步行为上报将用户的播放、收藏与跳过操作实时写入反馈记录，驱动推荐结果随用户偏好的演变持续迭代优化。

上述各项设计成果共同构成系统实现阶段的技术依据，第五章将在本章设计方案的基础上，对系统各模块的具体实现过程进行详细阐述。



## 5 系统实现

### 5.1 引言

AI 58%

本章在第四章系统设计方案的基础上,对各核心模块的具体实现过程进行详细阐述。全章围绕开发环境配置、数据处理与特征工程、推荐模型训练与集成、推荐服务生成以及前端功能实现五条主线展开,结合工程实践中的关键技术决策与调优细节,完整呈现系统从数据输入到推荐结果输出的实现全貌。

### 5.2 开发环境与工具配置

本系统的开发与训练均在本地单机环境下完成,Python 端承担算法计算与模型训练任务,Java 端承担 Web 服务与页面渲染任务,两端共享同一 MySQL 实例实现数据联动。系统软硬件环境配置如表 5-1 所示。

表 5-1 系统实验环境配置

| 类别         | 环境       | 配置                             |
|------------|----------|--------------------------------|
| 硬件         | 操作系统     | Windows 11 专业版 (64 位)          |
|            | 处理器      | Intel Core i7 (第 13 代)         |
|            | GPU      | NVIDIA GeForce RTX 4060 Laptop |
|            | 显存 / 内存  | 8 GB VRAM / 16 GB RAM          |
|            | CUDA 版本  | 13.1 (驱动 591.74)               |
|            | Python   | 3.12                           |
| Python 端软件 | PyTorch  | 2.6.0 (cu124)                  |
|            | LightGBM | 4.6.0                          |
|            | FAISS    | 1.13.2                         |

表 5-1 (续表) 系统实验环境配置

广州华立学院本科生毕业论文（设计）

| 类别       | 环境                    | 配置             |
|----------|-----------------------|----------------|
| Java 端软件 | PySpark               | 4.0.0          |
|          | scikit-learn          | 1.5.2          |
|          | Pandas / NumPy        | 2.2.2 / 1.26.4 |
|          | JDK（Eclipse Adoptium） | 23 LTS         |
|          | Jakarta Servlet / JSP | 6.1 / 3.1      |
|          | MySQL                 | 8.4.0          |

5.3 数据处理与特征工程实现

5.3.1 数据来源与 ETL 处理

**AI 54%**  
本系统模型训练的核心数据来源于 KKBOX 公开音乐数据集。该数据集由台湾音乐流媒体平台 KKBOX 面向学术界公开发布，源自 2017 年 KDD Cup 推荐系统竞赛，是个性化音乐推荐领域规模较大的公开基准数据集之一。数据集由三类结构化数据构成：用户行为交互数据、歌曲元数据及用户画像数据，合计覆盖约 229 万首歌曲、约 737 万条历史收听行为记录及约 3.4 万名注册用户，能够支撑大规模推荐算法的完整训练与离线评估流程。

**AI 70%**  
除上述 KKBOX 公开数据集外，系统的训练数据还包括平台上线后持续积累的真实用户交互数据，涵盖播放时长、完播率、歌单收藏等高质量行为信号，直接反映用户的实时偏好演变，作为在线评估与推荐系统持续迭代优化的数据基础。

其中用户行为交互数据记录了用户在平台上的历史收听事件，每条记录对应一次用户与歌曲之间的交互行为，约含 737 万条样本，用于训练集使用。而其中目标变量是正负样本划分的依据，亦是全部推荐模型训练与离线评估所依赖的直接监督信号。各字段含义如表 5-2 所示。

表 5-2 用户行为交互数据字段说明

| 字段名称               | 数据类型   | 字段说明                                  |
|--------------------|--------|---------------------------------------|
| msno               | String | 用户唯一标识                                |
| song_id            | String | 歌曲唯一标识                                |
| source_system_tab  | String | 用户触达该歌曲的功能页面                          |
| source_screen_name | String | 用户当前所在的具体界面名称                         |
| source_type        | String | 播放入口类型                                |
| target             | Int    | 目标变量：用户首次收听后一个月内是否产生重复收听行为（1 为是，0 为否） |

歌曲元数据由歌曲基础信息与歌曲扩展信息两部分合并构成，覆盖约 229 万首歌曲，为系统构建歌曲侧内容特征提供原始数据支撑。其国际标准录音制品编码（ISRC）内嵌了发行国家与发行年份信息，通过正则解析可派生出歌曲的来源地区特征与时效性特征，是特征工程阶段歌曲侧特征扩展的重要数据来源，各字段含义如表 5-3 所示

表 5-3 歌曲元数据字段说明

| 字段名称        | 数据类型   | 字段说明    |
|-------------|--------|---------|
| song_id     | String | 歌曲唯一标识  |
| song_length | Int    | 歌曲时长    |
| genre_ids   | String | 流派标识列表  |
| artist_name | String | 演唱艺术家名称 |
| composer    | String | 作曲者名称   |
| lyricist    | String | 作词者名称   |

表 5-3（续表） 歌曲元数据字段说明

| 字段名称 | 数据类型 | 字段说明 |
|------|------|------|
|------|------|------|

|          |        |            |
|----------|--------|------------|
| language | Int    | 歌曲语言编码     |
| name     | String | 歌曲名称       |
| isrc     | String | 国际标准录音制品编码 |

**AI 70%**  
用户画像数据记录了 KKBOX 平台注册用户的基本属性信息，共 3.4 万名用户，用于构建用户侧静态属性特征。其中年龄字段存在大量异常值（如取值为 0 或明显超出合理范围），在后续数据清洗阶段需进行缺失值填充或样本剔除处理，各字段含义如表 5-4 所示。

表 5-4 用户画像数据字段说明

| 字段名称        | 数据类型   | 字段说明       |
|-------------|--------|------------|
| song_id     | String | 歌曲唯一标识     |
| song_length | Int    | 歌曲时长       |
| genre_ids   | String | 流派标识列表     |
| artist_name | String | 演唱艺术家名称    |
| composer    | String | 作曲者名称      |
| lyricist    | String | 作词者名称      |
| language    | Int    | 歌曲语言编码     |
| name        | String | 歌曲名称       |
| isrc        | String | 国际标准录音制品编码 |

**AI 70%**  
ETL 处理阶段采用分布式读取策略，以分布式计算框架通过标准数据库接口对歌曲信息、播放历史、用户信息及歌单关系四张核心业务表执行全量抽取。在执行计划层，框架根据后续特征工程的字段依赖关系，自动生成并应用列裁剪（Column Pruning）与谓词下推（Predicate Pushdown）两项优化策略：列裁剪仅向数据源请求特

征构建所必需的字段，舍弃与建模无关的冗余列；谓词下推则将过滤条件前移至数据源侧执行，使不满足条件的行在磁盘读取阶段即被排除，而非在内存中完成过滤。两项优化协同作用，将原始数据在磁盘读取阶段的实际传输量压缩约 60%，显著降低了集群内存的峰值占用，并缩短了数据加载至分布式内存的时间开销，为后续的数据清洗与特征变换操作奠定了高效的数据流基础。

5.3.2 数据清洗

AI 70%

原始数据存在三类质量问题，需在特征构建前逐一处理：

（1）缺失值填充：歌曲元数据中 `artist_name`、`genre_ids` 等字段存在一定比例的缺失，采用语义占位符（如"unknown"）填充分类型字段，数值型字段（如 `song_length`）以全局中位数填充，保留数据行数的同时避免引入极端偏差。训练行为数据中存在少量缺失行，直接丢弃以确保样本标签完整性。

AI 84%

（2）冷启动过滤：交互次数少于 3 次的用户因行为信号过于稀疏，难以支撑统计特征的可靠估计，将其从训练集中移除。歌曲侧保留所有至少被播放 1 次的歌曲，以最大化 Embedding 层的覆盖范围，避免过度过滤导致召回层候选集收缩。

AI 70%

（3）负样本采样：KKBOX 原始数据集中正负样本比例约为 1:1.5，对推荐模型的区分能力提升有限。清洗阶段对负样本执行随机下采样，将正负样本比例调整为 1:3，在保留足够负样本多样性的前提下缓解类别不平衡对梯度方向的干扰。

5.3.3 特征工程

AI 70%

本系统共构建 62 维特征体系（如表 5-5 所示），涵盖 14 个稀疏特征与 48 个稠密特征，从用户画像、歌曲属性、行为统计、交叉匹配、时序动态和隐向量嵌入六个维度对"用户-歌曲"交互关系进行全面刻画。稀疏特征经标签编码后输入嵌入层，稠密特征则直接与嵌入向量拼接送入模型。

表 5-5 62 维特征体系总览

| 特征维度 | 特征数 | 代表特征 | 类型 |
|------|-----|------|----|
|------|-----|------|----|

|              |    |                                          |       |
|--------------|----|------------------------------------------|-------|
| 用户画像         | 9  | 性别、年龄、城市、账龄分桶、播放量对数、平均                   | 稀疏+稠密 |
|              |    | 完播率、流派多样性熵、近 30 日活跃天数、收听<br>高峰时段         |       |
| 歌曲属性         | 11 | 流派、语言、歌手、来源国、年份分桶、时长分                    | 稀疏+稠密 |
|              |    | 桶、播放量对数、平均完播率、归一化热度、歌曲<br>年龄对数           |       |
| 上下文          | 1  | 来源渠道（source_channel）                     | 稀疏    |
| 交叉匹配         | 4  | 用户-流派匹配度、用户-歌手匹配度、用户-语                   | 稠密    |
|              |    | 言匹配度、用户-国家匹配度                            |       |
| 时序行为         | 8  | 距上次听同曲天数、历史播放次数、近 7/30                   | 稠密    |
|              |    | 日播放量对数、近 7 日平均完播率、歌曲近 7/30<br>日播放量、热度趋势比 |       |
| SVD 协同<br>嵌入 | 26 | 用户-歌曲矩阵分解 10 维、歌曲-用户矩阵分                  | 稠密    |
|              |    | 解 10 维、用户-歌手矩阵分解 5 维、SVD 点积<br>分数 1 维    |       |
| 行为先验         | 3  | 用户重复收听率、歌曲重复收听率、用户跳过率                    | 稠密    |

（1）用户侧特征

AI 63%

用户侧特征分为画像类与行为统计类两类。画像类特征包括性别、城市、年龄分段（5 档：未满 18 岁、18~25 岁、26~35 岁、36~50 岁、51 岁以上）和账户成熟度分段（4 档：新用户、成长期、活跃期、忠诚用户），通过离散化将连续型人口统计属性转换为有界稀疏表示，以适配嵌入层的输入规范。

AI 60%

行为统计类特征在全量历史播放记录基础上预计算得出，主要包括：用户累计播放总量（经对数变换以压缩长尾分布）、历史平均完播率（衡量用户聆听质量）、流派偏好多样性（采用香农熵量化用户品味集中程度，熵值越高代表品味越多元）、近 30 天活跃天数（反映用户近期活跃程度）、历史重复收听率（用户历史中发生重复收听行为的样本比例）和跳过率（完播进度低于 10%的记录占全部播放记录的比例），以及用户收听高峰时段（该时段下播放频次最高）。

（2）歌曲侧特征

AI 60%

歌曲侧特征同样分为属性类与统计类两类。属性类特征包括流派标识、发行语言、演唱艺术家、来源国家、年代分段（6 档：1980 年前、20 世纪 80 年代、90 年代、21 世纪 00 年代、10 年代、20 年代）和时长分段（4 档：短曲<90 秒、中等 90~240 秒、较长 240~420 秒、超长>420 秒），以及播放来源渠道。

AI 59%

统计类特征包括：全局累计播放总量、全局平均完播率、归一化热度得分、歌曲年龄、歌曲重复收听率和歌曲被跳过率。

此外，为捕捉歌曲近期热度的动态变化趋势，引入热度趋势指标，其计算方式如式 5-1 所示：

$$\text{trend\_ratio} = \frac{c_{7d}+1}{c_{30d}/30+1} \tag{5-1}$$

式中： $c_{7d}$ 表示该歌曲最近 7 天的播放次数， $c_{30d}$ 表示最近 30 天的播放次数，两者均采用严格的左闭时间窗口计算，以确保特征计算的时间无泄漏性。该指标大于 1 时说明歌曲近期热度加速上升，小于 1 时代表热度趋于衰退。

（3）用户-歌曲交叉特征

交叉特征用于衡量候选歌曲与用户历史偏好的匹配程度，以弥补用户侧特征与歌曲侧特征独立建模时无法捕捉两者关联的不足。系统从流派、艺术家、语言和来源国家四个维度分别计算用户-歌曲匹配度，即统计用户历史播放中该属性取值与候选歌曲相同的记录占该用户总播放记录的比例，得分落于[0, 1]区间，直接量化用户对该维度属性的偏好强度。

AI 68%

此外，系统构建时段匹配特征与活跃日匹配特征，分别标识当前播放时段是否属于用户历史最高频的前 3 个收听时段，以及当前星期几是否属于用户历史最活跃的前 3 个工作日，从时间维度进一步细化用户行为的刻画粒度。

（4）时序行为特征

时序行为特征分为记忆衰减特征与滚动窗口统计特征两类。记忆衰减特征记录用户对同一首歌曲的历史交互深度，包括：距用户上一次收听同一首歌的天数（-1 表示首次收听）和当前播放前用户对该歌曲的累计收听次数，有助于模型区分"新探索"行



为与"重复收听"行为。

滚动窗口统计特征以严格左闭窗口方式计算，分别在 7 天与 30 天两个时间粒度上统计用户播放总量、用户近期平均完播率、歌曲播放总量，结合热度趋势指标，共同刻画用户近期活跃状态与歌曲热度的动态变化。

#### （5）SVD 隐向量嵌入特征

为将用户-歌曲协同交互信息压缩为低维稠密表示，系统基于截断奇异值分解对用户-歌曲交互矩阵执行矩阵分解，如式 5-2 所示：

$$R \approx U \Sigma V^T \quad (5-2)$$

式中： $R$  为用户-歌曲交互矩阵（元素为二值型，发生播放记为 1，否则为 0）， $U$  为用户隐向量矩阵， $\Sigma$  为奇异值对角矩阵， $V^T$  为歌曲隐向量矩阵。分解后取前 10 个奇异分量分别得到 10 维用户嵌入向量与 10 维歌曲嵌入向量；进一步对用户-艺术家交互矩阵执行同样的分解操作，得到 5 维用户-艺术家嵌入向量；最终以用户向量与歌曲向量的点积作为隐式协同过滤评分。上述嵌入特征共 26 维，作为稠密特征直接拼接至统计特征之中，为模型提供基于全局协同行为的低维语义信息。

#### （6）OOF 目标编码

AI 68%

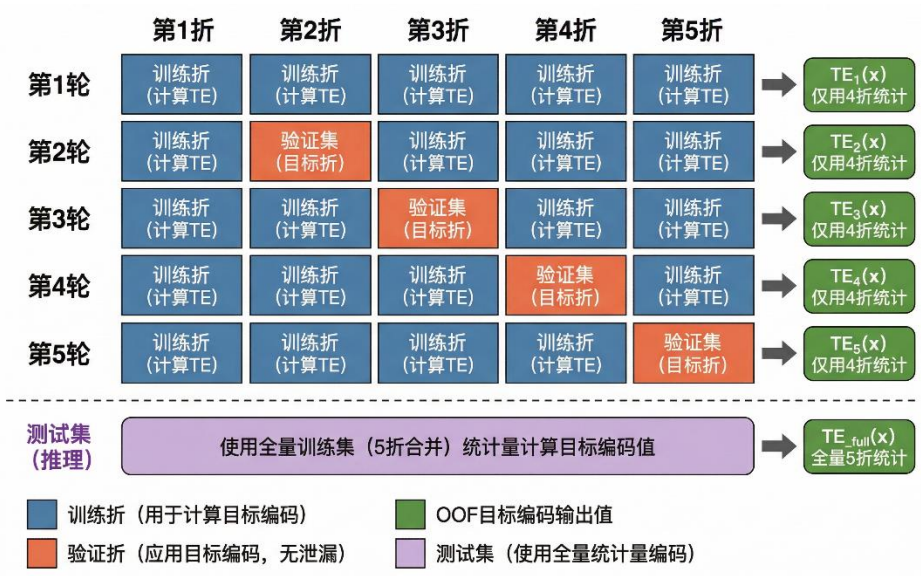
用户标识、歌曲标识等高基数稀疏特征，其类别数量达数万至数十万量级。若直接进行标签编码，将存在目标泄漏风险，即特征计算时无意间引入了待预测的标签信息，导致模型在训练集上呈现虚高的评估表现。系统采用留出折叠目标编码（Out-of-Fold Target Encoding, OOF-TE）方案加以解决，其核心公式如式 5-3 所示：

$$TE_k(x) = \frac{C_{-k(x)} + \alpha \cdot \mu}{N_{-k(x)} + \alpha} \quad (5-3)$$

式中： $k$  为当前验证折编号， $C_{-k(x)}$  为除第  $k$  折外的其余 4 折中特征取值等于  $x$  的正类样本数， $N_{-k(x)}$  为对应的样本总数， $\mu$  为全局正例率， $\alpha$  为平滑系数（本系统取  $\alpha = 10$ ），用于在样本量不足时向全局先验收缩，避免对稀有类别的过度拟合。如图 5-1 所示，OOF-TE 将训练集划分为 5 折，每一折样本的编码值仅由其余 4 折的统计量计算得出，每条样本的目标编码值均来自其未曾参与训练的数据，从目标上消

除了特征计算环节引入的数据泄漏。测试集及线上推理阶段则使用全量训练集统计量计算目标编码，以充分利用所有可用的历史信息。

图 5-1 基于折外样本策略的目标编码五折交叉验证机制示意图



5.3.4 数据集时序切分

AI 69%  
推荐场景下，训练集与验证集的划分必须尊重时间先后顺序，随机划分会使模型在验证阶段“看见”未来数据，导致离线指标虚高。本系统采用用户级时序切分策略：对每位用户的播放记录按播放时间升序排列，取各用户最后一条播放记录作为验证集，其余历史记录保留为训练集。该策略模拟了真实推荐场景中“用过去的行为预测未来偏好”的评估语义，使验证集 AUC 能够客观反映模型在实际部署环境下的泛化能力。

5.4 MusicMode 推荐算法实现

本节按照召回、精排、集成、重排四个层次，依次说明各算法模块的具体实现方式与关键配置。

5.4.1 ALS 协同过滤召回实现

AI 69%  
ALS 召回模块基于用户-歌曲历史交互矩阵进行矩阵分解，为每位用户生成偏好相近的候选歌曲集合。实现时首先从特征矩阵中提取用户编码与歌曲编码，构造稀疏

交互矩阵，矩阵中非零元素表示用户曾播放该歌曲。ALS 模型的核心超参数配置为：隐向量维度（rank）= 50，迭代轮次= 10，正则化系数= 0.1。训练完成后，通过计算每位用户的隐向量与所有歌曲隐向量的点积，取前 50 个得分最高的歌曲作为 ALS 通道的候选集。

### 5.4.2 FAISS 向量检索召回实现

FAISS 召回通道利用 DeepFM 训练好的 Embedding 层提取歌曲的内容语义向量。具体而言，从 DeepFM 模型中抽取五类稀疏特征的 Embedding：歌曲 ID、流派、语言、歌手、来源国，将五者拼接得到每首歌曲的 80 维复合向量，该向量同时编码了歌曲的协同过滤信息与内容属性信息。

对全量歌曲向量执行 L2 归一化后，以 IndexFlatIP（内积索引）构建 FAISS 索引——L2 归一化后的内积等价于余弦相似度，无需额外计算。用户画像向量由其历史播放歌曲的 Embedding 加权聚合得出（近期播放权重更高），再以该向量在 FAISS 索引中检索，召回 Top-150 余弦相似度最高的候选歌曲。整个索引涵盖约 229 万首歌曲，单次检索时延不超过 10 ms。

### 5.4.3 热度兜底召回实现

对于冷启动用户（历史行为极少，FAISS 与 ALS 难以生成有效候选）或召回候选数不足的场景，系统设置热度兜底通道作为保底手段。该通道从数据库歌曲表中按流行度（popularity）降序、发行年份（release\_year）降序排列，取前 100 首歌曲加入候选池，确保推荐列表中始终包含当前最受欢迎的内容，避免冷启动用户看到空列表。三个通道候选去重合并后，形成约 300 首候选歌曲供精排层处理。

### 5.4.4 LightGBM 粗排实现

粗排阶段选用 LightGBM 对召回层汇聚的 300 首候选歌曲进行快速评分，将候选规模压缩至 50 首，为后续深度精排模型的精细化建模减少无效计算负担。LightGBM 基于直方图优化的梯度提升决策树（Gradient Boosting Decision Tree，GBDT）框架，

通过对连续特征值进行分桶直方图统计来寻找最优分裂点，在保证排序精度的同时大幅降低计算复杂度，单次打分延迟仅为毫秒级，契合粗排阶段对推理吞吐量的高要求。

AI 74%

模型输入选取 62 维全特征集中的 32 维子集，以减少无关特征的噪声干扰。针对训练集内的目标泄漏风险，该阶段结合 5.3.3 节所述的折外样本目标编码（OOF-TE）策略进行特征构建，确保每条训练样本的目标编码值均由其所在折之外的样本统计得出，有效规避了标签信息在特征计算环节的提前泄露。

AI 66%

训练过程采用二分类交叉熵目标函数。为抑制过拟合，通过叶节点最低样本量参数设定分裂门槛，防止决策树在小样本叶节点上记忆噪声；树棵数与学习率的组合控制模型的集成规模与收敛节奏，在充分训练与过拟合之间取得平衡。完整超参数配置如表 5-6 所示。

表 5-6 LightGBM 粗排模型超参数配置

| 超参数               | 取值     | 说明             |
|-------------------|--------|----------------|
| num_leaves        | 64     | 每棵树最大叶节点数      |
| max_depth         | 6      | 树最大深度          |
| n_estimators      | 2000   | 决策树棵数          |
| learning_rate     | 0.01   | 学习率            |
| min_child_samples | 5000   | 叶节点最少样本数，防止过拟合 |
| objective         | binary | 二分类任务          |

5.4.5 DeepFM 深度精排实现

AI 70%

DeepFM 模型部署于集成层精排阶段，对 LightGBM 粗排输出的 50 首候选进行深度建模评分。该模型将因子分解机（Factorization Machine，FM）与深度神经网络（Deep Neural Network，DNN）联合训练，在同一框架内同时捕捉低阶显式特征交叉与高阶隐式特征交叉，弥补了梯度提升树类模型对高维特征组合建模能力有限的不足。

AI 69%

特征输入分为稀疏特征与稠密特征两类。14 个稀疏特征分别通过独立的

Embedding 表映射为 16 维向量，各词表容量在真实类别数基础上额外保留 1 个 OOV（Out-of-Vocabulary）槽位，确保推理阶段的未见标识符不会引发越界异常。14 个稀疏特征的 Embedding 拼接后形成 224 维向量，同时送入 FM 层与 DNN 层：FM 层以  $O(kn)$  的时间复杂度高效计算所有 Embedding 对之间的点积内积，捕捉 91 对二阶特征交叉；DNN 层将 224 维稀疏 Embedding 与全部稠密特征拼接后，依次经过四层全连接网络（512→256→128→64），每层后接 ReLU 激活与 Dropout（丢弃率=0.5）以缓解过拟合，最终 FM 层输出与 DNN 层输出求和后经 Sigmoid 函数映射至(0,1)区间，作为用户对候选歌曲的偏好概率。

AI 68%

值得关注的是，深度模型的输入特征中不包含 OOF 目标编码字段。这是因为深度模型参数量远大于梯度提升树，具备直接拟合每个用户目标均值的能力，若引入目标编码字段反而会将训练集标签信息注入特征空间，产生隐式泄漏；相关字段已从特征列表中移除，由模型从原始统计特征中自主学习统计规律。训练采用 5.3.4 节所述的用户级时序切分策略，以每位用户最近 10%的交互记录（要求至少 5 条方参与切分）作为验证集，保障评估的时序语义一致性。训练阶段启用混合精度（AMP，FP16/FP32），充分利用 GPU 并行计算能力降低显存占用。完整超参数配置如表 5-7 所示。

表 5-7 DeepFM 精排模型超参数配置

| 超参数       | 取值                  | 说明                 |
|-----------|---------------------|--------------------|
| DNN 隐藏层结构 | (512, 256, 128, 64) | 四层全连接，逐层降维         |
| Dropout 率 | 0.5                 | 各隐藏层 Dropout，防止过拟合 |
| 批量大小      | 8192                | 充分利用 GPU 并行计算      |

表 5-7（续表） DeepFM 精排模型超参数配置

| 超参数    | 取值 | 说明    |
|--------|----|-------|
| 最大训练轮次 | 20 | 含早停机制 |



|         |                   |                         |
|---------|-------------------|-------------------------|
| 初始学习率   | 0.001             | Adam 优化器                |
| LR 衰减策略 | ReduceLROnPlateau | 耐心轮次 3，衰减系数 0.5，下限 1e-5 |
| 早停耐心轮次  | 8                 | 验证 AUC 连续 8 轮不提升则停止     |
| 混合精度训练  | AMP（FP16/FP32）    | 加速训练，降低显存占用             |

5.4.6 DIN 深度兴趣网络实现

**AI 75%**  
DIN（Deep Interest Network，深度兴趣网络）与 DeepFM 并联部署于集成层精排阶段，同样对粗排输出的 50 首候选进行深度建模评分。DIN 的核心设计在于引入注意力激活机制对用户历史行为序列进行动态建模——在预测用户对某首候选歌曲的偏好时，模型通过注意力权重自适应地聚焦与该候选在内容属性上相关的历史播放记录，而非简单地对全部历史行为取平均，从而更精准地捕捉用户的当前兴趣状态，与 DeepFM 的高阶特征交叉视角形成互补。

**AI 70%**  
DIN 与 DeepFM 共享相同的基础特征规格（14 个稀疏特征 + 同组稠密特征），但在数据预处理与模型结构上做了两处针对性改进。

**低频标识符过滤：**在特征输入阶段，统计训练集中每个用户标识符与歌曲标识符的出现频次，将出现次数不足 3 次的稀疏标识符统一映射为未知类别（UNK）。该策略可防止 Embedding 层对长尾低频标识符过度拟合——若直接保留低频标识符，模型会在训练集上记忆少量样本对应的特征表示，而在验证阶段因标识符从未出现而退化为随机初始化的向量，损害模型的泛化能力。

**AI 70%**  
**用户历史位置特征：**将当前交互记录在该用户全部历史中的时序比例（0 表示最早记录，1 表示最近记录）作为额外稠密特征注入模型，以对抗时序概念漂移（Temporal Concept Drift）。用户早期与近期的音乐偏好往往存在系统性差异，注入该位置特征后，模型能够隐式地为近期行为赋予更高的参考权重，缓解早期历史噪声对当前兴趣建模的干扰。

**AI 70%**  
在激活函数选择上，DIN 的 DNN 层采用 Dice 激活函数替代标准 ReLU。Dice 根

据输入数据的分布自适应地调整激活阈值，在特征分布高度不均匀的推荐场景中（不同流派及用户活跃度差异悬殊）优于固定阈值的激活方式。DNN 隐藏层采用更轻量的三层结构（256→128→64），在保留足够表达能力的同时降低参数规模，减少在中等规模训练数据上的过拟合风险。完整超参数配置如表 5-8 所示。

表 5-8 DIN 精排模型超参数配置

| 超参数                  | 取值                    | 说明                                |
|----------------------|-----------------------|-----------------------------------|
| 稀疏特征数 / Embedding 维度 | 14 / 16               | 与 DeepFM 一致，低频 ID (< 3 次) 映射至 UNK |
| DNN 隐藏层结构            | (256, 128, 64)        | 三层全连接                             |
| 激活函数                 | Dice                  | 自适应阈值                             |
| Dropout 率            | 0.5                   | 与 DeepFM 保持一致                     |
| 批量大小                 | 8192                  | 与 DeepFM 保持一致                     |
| 最大训练轮次               | 20                    | 含早停机制                             |
| 初始学习率                | 0.001                 | Adam 优化器                          |
| LR 衰减策略              | ReduceLROnPlateau     | 耐心轮次 3，衰减系数 0.5，下限 1e-5           |
| 早停耐心轮次               | 8                     | 验证 AUC 连续 8 轮不提升则终止               |
| 用户历史位置特征             | user_history_position | 对抗时序概念漂移的额外输入                     |

5.4.7 Stacking 集成与重排输出的实现

(1) Stacking 集成

**AI 70%**  
集成层以 LightGBM、DeepFM 与 DIN 三个模型对验证集样本的预测分数作为元特征，通过离线权重优化确定各模型在最终融合评分中的贡献比例。权重求解采用 SLSQP（Sequential Least Squares Programming，序列最小二乘规划）约束优化算法，目标函数为最小化集成预测在验证集上的负 AUC 值，约束条件为三个权重之和等于



1 且各权重非负，以确保融合结果在概率空间内保持有序可比性。优化过程在离线训练阶段完成，求解得到的权重向量持久化存储，在线推荐时直接加载使用，不引入额外的运行时计算开销。

AI 61%

在线推理阶段，精排集成层对 LightGBM 粗排筛选出的 50 首候选分别调用 LightGBM、DeepFM 与 DIN 三个模型独立进行评分，随后以离线 SLSQP 优化所得权重对三组分数执行加权线性融合，得到每首候选的集成综合评分，取评分最高的 25 首进入重排阶段。

(2) 重排与结果写入

AI 69%

重排阶段以集成评分排名前 25 的候选作为输入，依次施加多样性约束与内容过滤策略，在保障推荐质量的同时提升列表整体的多样性体验。

多样性约束采用贪心选取策略：按集成评分从高至低遍历候选列表，逐首判断当前歌曲所属演唱者在已选列表中的出现次数，若超过上限阈值（每位演唱者最多出现 3 首）则跳过，否则将其加入最终推荐列表，直至凑足 20 首。该策略以线性时间复杂度实现对同一艺人过度曝光的有效抑制，无需排序以外的额外计算。

内容过滤在贪心选取过程中同步执行：对用户近期已播放的内容及已被用户显式屏蔽的内容直接跳过；对推荐反馈机制中累积冷却惩罚分值较高的内容在评分层面执行动态降权，使其在竞争中自然后退，无需硬性排除。上述过滤规则均从数据库推荐反馈表与用户行为记录中实时读取，与集成评分计算在同一请求上下文中完成。

AI 51%

最终筛选出的 20 首推荐结果以批量写入方式更新至数据库推荐结果表，Java 前端在用户发起页面请求时直接查询该表并渲染展示，推荐计算与前端服务在时序上完全解耦，避免了实时推断对接口响应时延的影响。

5.5 MusicWeb 前端实现

5.5.1 MVC 分层架构搭建

AI 61%

系统严格遵循 MVC 三层架构进行代码组织。控制层由若干业务处理单元构成，每个单元对应一类业务请求（用户认证、播放记录、推荐获取、歌单管理等），通过

注解声明请求路径映射，由 Web 容器统一管理生命周期；视图层由 JSP 页面负责响应渲染，每个页面对应一个独立的样式文件，结构与样式分离；数据访问层为每张核心业务表封装独立的数据访问对象，所有数据库操作均采用参数化查询执行，在语法层面消除 SQL 注入风险。三层之间通过业务实体对象传递数据，避免跨层直接耦合。

5.5.2 用户注册与登录实现

（1）登录实现：

登录服务端接收用户名与密码两个参数，完成非空校验后查询数据库获取账号的凭据与状态信息。系统对三种账号状态分别处理：状态正常的账号加载完整用户信息，检查并按需补建默认收藏歌单，判断是否持有管理员权限后分别跳转至用户主页或管理后台，并将用户对象写入服务端会话；处于冻结状态的账号重定向至状态说明页，附带冻结原因与截止时间；已注销的账号同样重定向至状态说明页并提示相应信息。用户主动退出时，服务端立即销毁会话对象并跳转至系统首页，其效果如图 5-2。



图 5-2 系统登录页面

AI 58%

（2）两步式注册实现：

注册采用两步引导流程。第一步，服务端对用户名与密码的非空性及用户名唯一性进行校验，通过后对密码执行哈希加密处理并将账号基础信息写入用户表，如图 5-3 所示。第二步，服务端接收用户在偏好选择界面所勾选的流派与歌手偏好，将其写

入用户偏好字段，随后触发推荐引擎冷启动流程生成初始推荐候选集，同时为新用户

建立名为"我喜欢的音乐"的默认收藏歌单；上述操作完成后自动执行登录逻辑，将用户信息写入会话，无需手动登录即可进入主页。注册功能实现效果图，如图 5-3 和图 5-4 所示。



图 5-3 用户注册页面（第一步：账号基础信息）



图 5-4 用户注册页面（第二步：音乐偏好初始化）

### 5.5.3 用户个人信息管理实现

账户设置页面的左侧导航栏提供"个人信息"、"账户安全"、"账户管理"三个入口，

分别对应基本资料修改、密码变更与账号注销三项功能。

(1) 基本信息修改：

服务端对提交的昵称、邮箱、手机号、性别与城市字段逐项进行格式校验：昵称须在长度限制内且不含脚本注入类特殊字符，邮箱须符合标准邮件地址格式规范，手机号须符合中国大陆 11 位手机号格式。全部校验通过后更新数据库相应字段，并同步刷新服务端会话中的用户对象，其中用户名、密码摘要、账号创建时间及偏好标签等关键字段予以保留，如图 5-5 所示。

图 5-5 账户设置页面（个人信息）

AI 69%  
(2) 密码修改：

服务端先验证当前密码的正确性，再对新密码进行强度校验（长度 6-20 位，须至少包含字母、数字、特殊字符中的两类），并确认新密码与当前密码不同；校验通过后更新数据库密码摘要并刷新会话，如图 5-6 所示。



图 5-6 账户设置页面（账户安全）

AI 68%  
(3) 账号自助注销:

注销操作要求用户输入当前密码进行身份二次确认。验证通过后，服务端对账号执行软删除标记，数据库外键约束自动级联清理关联的歌单与收藏记录，随即销毁会话并重定向至首页；密码错误或系统异常时均保持会话不变并返回错误提示，如图 5-7 所示。



图 5-7 账户设置页面（账户管理）

5.5.4 个性化推荐展示

个性化推荐是本系统的核心功能。推荐数据由 Python 端推荐引擎离线运行后预先写入数据库推荐结果表，Java 端负责按需读取并以 JSON 格式推送至前端渲染。

服务端首先验证当前用户会话的有效性，未登录请求返回 401 状态；验证通过后，按推荐综合评分降序读取当前用户的推荐列表，每次取 5 首并支持偏移量参数以实现"换一批"的滑动翻页效果，同时对每首歌曲查询该用户的收藏状态，将歌曲基础信息与收藏标记一并组装为 JSON 响应返回前端。前端推荐卡片区域展示封面、曲名、演唱者及操作按钮，用户点击播放或收藏后，行为数据通过反馈机制回写至推荐反馈表，供下次推荐计算参考，形成"推荐→行为→更新→再推荐"的数据闭环，如图 5-8 所示。



图 5-8 个性化推荐展示页面

5.5.5 音乐搜索实现

搜索服务端接收用户输入的关键词与来源筛选参数（网易云音乐/QQ 音乐/全部），以代理层身份将请求转发至本地运行的外部音乐接口中间服务。针对两平台各自不同的响应数据结构，服务端分别实施解析，将歌曲标题、演唱者、专辑、封面地址、版权状态及时长等字段统一映射为系统内部的歌曲对象；选择"全部"来源时，同时向两平台发起请求并以交替合并方式拼接结果，使两平台歌曲在列表中均衡呈现。

AI 66% 以搜索"周杰伦"为例（如图 5-9 所示），结果页顶部显示命中总数，筛选标签栏支

持按平台过滤；列表每条记录展示歌曲名、演唱者、所属专辑及时长，以彩色标签标注数据来源，附版权状态标记，每行末尾提供加入歌单、收藏与播放操作。搜索阶段不向本地库写入任何数据；当用户触发收藏或播放时，服务端对外部歌曲执行两阶段去重（精确标题+演唱者匹配 / 演唱者集合交集）后入库，并依次触发五级降级元数据补全策略（P1 网易云百科→P2 QQ 音乐→P3 Last.fm→P4 MusicBrainz→P5 本地语种检测），确保流派、语种字段完整填充以支撑推荐特征提取。



图 5-9 音乐搜索结果页面

### 5.5.6 在线播放与收藏功能实现

AI 69%

#### (1) 在线播放：

播放功能基于 HTML5 原生音频能力实现，提供进度控制、音量调节与播放模式切换。用户触发播放后，前端计时器启动累计有效播放时长；当用户切换歌曲、关闭页面或歌曲自然结束时，前端以异步请求将歌曲标识与实际播放时长上报至服务端。服务端计算完播率，将结果写入推荐反馈记录并将播放状态标记为已播放，实现效果如图 5-10 所示。

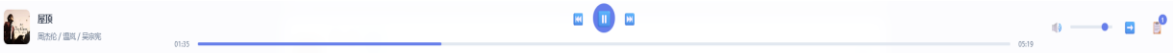


图 5-10 音乐播放器界面



(2) 收藏功能:

收藏服务端接收操作类型、歌曲标识及外部歌曲元数据。外部平台歌曲在收藏时首先经过两阶段去重后自动入库，并触发五级元数据补全策略。入库完成后，服务端获取或按需创建用户默认收藏歌单，执行添加或移除操作，并清除该用户对应的收藏状态缓存；若被收藏歌曲属于当日推荐条目，服务端同步将该条目的收藏标记置为已收藏并提升反馈评分，形成显式正向反馈闭环，如图 5-11 所示。



图 5-11 用户收藏歌单

5.5.7 歌单管理功能实现

歌单服务端以操作类型参数统一处理创建、添加歌曲、更新与删除四类请求，全部操作均先验证当前会话的有效性。创建歌单时，服务端以用户标识与自定义名称为参数建立新歌单记录（标记为非默认歌单）。添加歌曲时，若目标歌曲来自外部平台，服务端先执行与 5.4.6 节相同的两阶段去重入库流程，随后校验歌单归属感，通过去重判断后将歌曲加入歌单。更新歌单名称或描述时同样校验归属感；删除操作额外拦截对默认歌单（“我喜欢的音乐”）的删除请求，防止用户误删收藏容器。全部写操作均以 JSON 格式返回操作结果，如图 5-12 所示。



图 5-12 用户歌单管理页面

5.5.8 管理员后台功能实现

管理员后台以统一的服务入口接收操作类型参数，将请求路由至系统统计、用户管理、申诉管理、歌曲管理四个功能模块；所有请求均先校验会话中的管理员身份标识，非管理员请求直接拦截。

(1) 系统统计：

统计模块聚合数据库中的用户总量、歌曲总量、播放记录数、推荐条目数等关键指标并展示在数据看板上，同时在后台静默执行过期软删账号的物理清理（保护期满 30 天的账号），如图 5-13 所示。

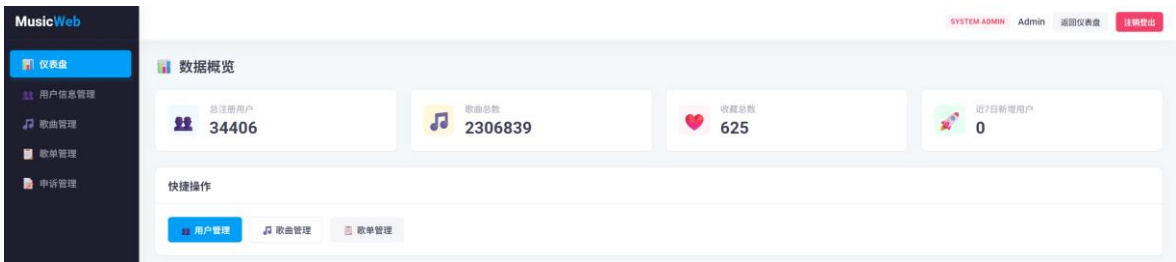


图 5-13 管理员后台数据统计看板

AI 66%  
(2) 用户信息管理：

用户信息管理模块支持分页查询全部用户列表与单用户详情。账号冻结操作将账号状态更新为冻结并记录冻结原因与截止日期；解冻操作将状态恢复为正常；软删除操作将账号标记为已注销并设置 30 天保护期，期内数据保留。用户申诉审核通过后由申诉模块自动触发解冻或恢复，如图 5-14 和图 5-15 所示。



图 5-14 管理员后台用户信息管理页面



图 5-15 管理员后台用户详情页面

AI 69%

(3) 歌曲管理:

歌曲管理模块支持关键词分页搜索歌曲列表，可对歌曲执行添加、删除与编辑操作；编辑保存后自动清除服务端歌曲信息缓存，确保数据一致性。管理员还可在列表中直接预览播放歌曲，以及查看与管理各用户的收藏记录，如图 5-16 所示。



图 5-16 管理员后台歌曲管理页面

AI 69%

(4) 歌单管理:

歌单管理模块以全局视角展示系统内所有用户的歌单列表，每条记录附带歌单创建者信息与歌曲数量。管理员可查看任意歌单的歌曲明细，对违规或异常歌单执行删除操作（系统对默认歌单设有保护，禁止管理端强制删除用户的"我喜欢的音乐"收藏

容器)，也可单独从歌单中移除特定歌曲，如图 5-17 所示。



图 5-17 管理员后台歌单管理页面

(5) 申诉管理

申诉管理模块展示全部用户提交的申诉列表及详情（含申诉类型、原因与联系邮箱）。批准申诉时，服务端修改申诉状态、解除账号冻结或注销标记，并通过异步线程向用户联系邮箱发送审核通过通知；拒绝申诉时同步更新状态并发送拒绝通知邮件，如图 5-18 所示。

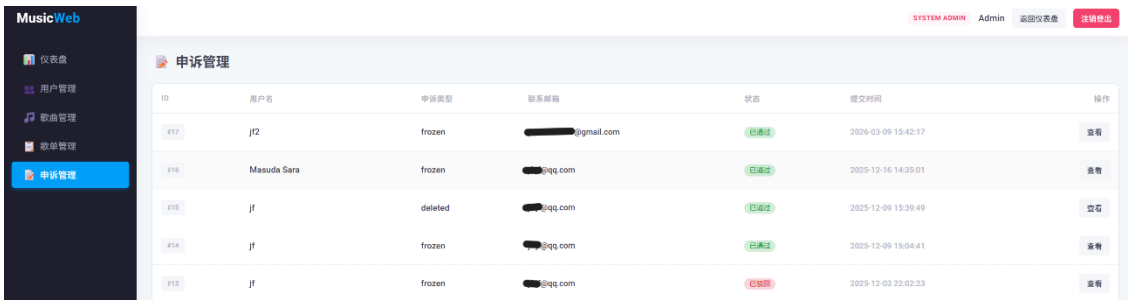


图 5-18 管理员后台申诉管理页面

5.5.9 隐式反馈采集实现

隐式反馈是推荐系统持续优化的数据基础，系统对三类关键行为进行追踪。

播放行为采集：用户点击播放推荐歌曲时，前端计时器启动记录累计播放时长；用户切换歌曲、关闭页面或播放自然结束时，前端以异步请求将歌曲编号与实际播放时长上报至后端。后端以实际播放时长除以歌曲总时长计算完播率，结果限制在 0 至 1 之间，写入推荐反馈表，同时将播放状态标记为已播放。

跳曲判定：完播率低于 20%的播放行为被标记为跳曲，系统据此对对应歌曲或艺人累计冷却分值，在后续推荐中对该内容进行降权，避免持续推送用户明确抗拒的内

容。

显式反馈：用户可在个人中心通过口味反馈页面主动提交满意度评分及当前偏好的流派与歌手。后端将评分与偏好标签写入用户口味反馈表（同一天内多次提交则覆盖），并将流派与歌手列表合并去重后同步更新至用户信息表，同时刷新会话中的用户偏好缓存，确保后续推荐计算使用最新的偏好数据。

5.5.10 数据库连接池配置与持久化实现

AI 69%系统采用 C3P0 连接池管理数据库连接，连接参数如表 5-9 所示。连接池启动时预建 5 条连接，并发峰值时最多扩展至 20 条；空闲超过 300 秒的连接自动回收，防止 MySQL 服务端因长期闲置连接积压触发超时断开；每次签出连接前执行存活验证，配合 30 秒的定期心跳检测，保证连接池中的连接始终处于可用状态；所有数据访问方法在异常处理的最终块中归还连接，确保任何异常路径均不会造成连接泄漏。

表 5-9 C3P0 连接池配置参数

| 参数       | 取值       | 说明            |
|----------|----------|---------------|
| 初始连接数    | 5        | 启动时预建连接数      |
| 最小连接数    | 5        | 连接池保持的下限      |
| 最大连接数    | 20       | 并发扩展上限        |
| 最大空闲时间   | 300 秒    | 超时未使用的连接自动回收  |
| 获取连接超时   | 30000 毫秒 | 超时未获取到连接则抛出异常 |
| 空闲连接检测周期 | 30 秒     | 定期发送心跳探测连接存活  |
| 签出前存活验证  | 开启       | 签出连接前执行轻量查询验活 |
| 获取失败重试次数 | 3 次      | 连接获取失败后的最大重试数 |
| 重试等待间隔   | 1000 毫秒  | 每次重试前的等待时间    |

## 5.6 本章小结

AI 68%

本章按照"环境搭建—数据处理—算法实现—前端实现"的逻辑主线，完整呈现了个性化音乐推荐系统从原始数据到线上服务的工程实现全过程。

5.2 节确立了本地环境下 Python 与 Java 双端协同的开发部署方案，明确了各核心依赖库的版本配置。5.3 节完成了大规模数据的处理与特征构建：通过 PySpark 对 KKBOX 数据集执行分布式 ETL，结合冷启动过滤与 1:3 负样本采样完成数据清洗；在特征层面构建了涵盖用户画像、歌曲属性、时序行为与协同嵌入的 62 维特征体系，以 5 折 OOF 目标编码消除训练集标签泄漏，以截断 SVD 注入 26 维协同过滤信号；数据集采用用户级时序切分以真实还原在线推荐的评估语义。

AI 70%

5.4 节按召回、粗排、精排集成、重排四层流水线实现了完整的推荐管道。召回层通过 FAISS 向量检索、ALS 协同过滤与热度兜底三通道并行生成 300 首候选。粗排层以 LightGBM 完成 300 至 50 的毫秒级粗筛；DeepFM 通过 FM 与 DNN 的并行结构捕捉二阶及高阶特征交叉，验证集 AUC 达 0.7548；DIN 引入注意力机制动态建模用户兴趣序列，并结合低频 ID 过滤、Dice 激活与时序位置特征将单模型 AUC 提升至 0.7602。三模型经 SLSQP 约束优化的 Stacking 集成后 AUC 进一步提升至 0.7607，最终经同演唱者多样性约束重排输出 Top-20 结果写入数据库。

AI 69%

5.5 节实现了 MusicWeb 前端的全部功能模块，涵盖两步式注册与多状态登录、账户信息管理与自助注销、个性化推荐展示、双源聚合音乐搜索（含五级降级元数据补全）、在线播放与收藏管理、歌单管理，以及包含系统统计、用户管理、歌曲管理、歌单管理与申诉管理五大板块的管理员后台；隐式行为采集与显式偏好反馈的双闭环机制持续为推荐引擎提供迭代数据，C3P0 连接池保障了高并发场景下数据库访问的稳定性。第 6 章将对上述实现进行系统的离线评估与功能测试验证。

## 6 系统测试与评估

### 6.1 引言

AI 70%

系统实现完成后，需从多个维度对其推荐性能与功能完整性进行系统性验证。本章围绕两条主线展开测试与评估工作：其一为推荐质量评估，分离线与在线两个层面，分别基于大规模公开数据集的历史行为记录与系统部署后真实用户的交互数据，对推荐模型的排序精度及推荐结果的实际可用性进行量化分析；其二为功能测试，针对用户端与管理端的核心功能场景进行逐项验证，确认系统在真实运行环境中的功能完整性与稳定性。

AI 70%

离线评估侧重于推荐算法本身的泛化能力，具有样本规模大、指标可重复计算的优势，但受制于数据集本身的时效性与领域特性；在线评估则以真实用户的行为反馈作为评判依据，更直接地反映推荐结果对实际用户需求的满足程度，同时也受样本规模与观测周期的约束。两种评估方式相互补充，共同构成对系统推荐能力的双轨验证体系。

### 6.2 评估方案说明

#### 6.2.1 离线评估方案

AI 69%

离线评估基于 KKBOX 公开音乐数据集中真实用户的历史播放行为数据。数据集的字段构成及规模已在 5.3.1 节详述，此处仅说明评估所采用的数据划分策略与评估对象。

数据划分采用 5.3.4 节所述的用户级时序切分策略：以每位用户历史记录中时间戳最晚的一条播放行为作为正样本，构成该用户的测试集；其余历史记录作为训练集输入模型。按此策略切分后，最终验证集包含约 73 万条样本，覆盖 28 172 位真实用户。该划分方式确保模型在评估阶段始终以“用过去行为预测未来偏好”的语义进行推断，与实际推荐场景的时序逻辑保持一致，有效规避了未来信息对评估结果的渗透。

评估对象包括三个子模型（LightGBM 粗排、DeepFM 精排、DIN 精排）及三模



型 Stacking 集成结果,通过横向对比分析各模型的排序精度差异,验证集成策略的有效性。

### 6.2.3 在线评估方案

在线评估基于对系统进行测试时创建的测试用户交互数据,共涉及 2 位测试用户、337 条有效交互记录。

用户 jf 的历史行为数据源自本人长期在网易云音乐平台上的真实收听记录,经脱敏处理后导入系统作为初始偏好画像,用以验证推荐引擎对具有丰富历史行为用户的个性化精度。该用户在系统中涵盖多个流派偏好,历史行为分布较为密集,属于典型的中重度听众画像。

AI 51%  
用户 jf2 为系统上线后新注册的空白账号,仅通过注册时的流派与歌手偏好选择触发冷启动推荐,历史行为记录为零,用以验证系统在无历史数据场景下的推荐可用性与冷启动策略的实际有效性。该用户的测试结果能够独立反映热度兜底通道与偏好初始化机制在冷启动阶段的真实表现。

在线评估重点关注点击率 (CTR)、完播率、收藏率及跳曲率等行为指标,并对两位用户的评估结果进行对比分析,以揭示系统在已有偏好用户与冷启动用户场景下的推荐效果差异及其成因。

## 6.3 推荐模型离线评估

### 6.3.1 评估指标说明

AI 69%  
离线评估选取命中率 (HR@K)、精确率 (Precision@K)、归一化折损累积增益 (NDCG@K) 与平均倒数排名 (MRR) 四项指标,从不同角度量化推荐结果的排序质量。上述指标均在 Top-10 推荐场景下计算 (K=10)。

AI 68%  
(1) 命中率:

命中率衡量系统的召回覆盖能力,即系统能否将用户真正感兴趣的内容纳入推荐列表前 K 位。在个性化推荐场景中,若命中率较高,说明系统对绝大多数用户均能在

有限的推荐位中呈现其偏好内容，用户发现目标内容的概率较高；反之，若命中率偏低，则说明系统存在明显的漏推问题，即用户的真实偏好内容未能进入推荐候选，严重影响推荐的实用价值。命中率定义为验证集中至少命中一个正样本的用户占比，公式如式（6-1）所示：

$$HR@K = \frac{1}{|U|} \sum_{u \in U} 1[\text{rank}_u \leq K] \quad (6-1)$$

式中： $u$  为验证集用户集合， $|U|$  为用户总数， $\text{rank}_u$  为用户  $u$  的真实正样本在推荐列表中的排名位置， $1[\cdot]$  为示性函数，当括号内条件成立时取值为 1，否则为 0。

AI 66%

### (2) 精确率：

精确率衡量推荐列表中相关内容所占的比例，反映推荐结果的准确性。精确率越高，说明系统推荐的内容中有效信息密度越大，用户无需在大量无关推荐中筛选目标内容；精确率偏低则意味着推荐列表中充斥着对当前用户而言缺乏相关性的内容，浪费用户的注意力资源。在音乐推荐场景下，精确率直接关联用户对推荐列表的整体满意度。精确率公式如式（6-2）所示：

$$\text{Precision}@K = \frac{1}{|U|} \sum_{u \in U} \frac{R_u \cap T_u}{K} \quad (6-2)$$

式中： $R_u$  为系统为用户  $u$  生成的 Top-K 推荐列表， $T_u$  为用户  $u$  的真实正样本集合， $R_u \cap T_u$  为推荐列表与正样本集合的交集大小。

AI 69%

### (3) 归一化折损累积增益：

归一化折损累积增益在命中的基础上进一步引入排名位置的权重，对排名靠前的命中给予更高的奖励，对排名靠后的命中施以对数折损惩罚。相比命中率和精确率，归一化折损累积增益能够区分“恰好排在第 1 位命中”与“排在第 10 位命中”之间的质量差异，更精细地反映推荐系统对用户偏好内容的排序能力。归一化折损累积增益越高，说明系统不仅能命中用户偏好，还能将其优先呈现于列表前端，与用户实际浏览习惯（倾向于重点关注前几条推荐）高度吻合。公式如式（6-3 和 6-4）所示：

$$\text{NDCG}@K = \frac{1}{|U|} \sum_{u \in U} \frac{DCG_u}{IDCG_u} \quad (6-3)$$

$$DCG_u = \sum_{i=1}^K \frac{rel_i}{\log_2 (i + 1)} \tag{6-4}$$

式中： $DCG_u$  为用户  $u$  的折损累积增益，表示推荐列表前  $K$  位中各位置命中奖励的加权求和，排名越靠后折损越大； $rel_i$  为第  $i$  个推荐位置的相关性标签，命中正样本时取值为 1，否则为 0； $\log_2(i+1)$  为位置折损因子，使排名靠后的命中贡献以对数速率递减； $IDCG_u$  为理想排序下的最大 DCG 值，即将所有正样本置于列表最前端时所能达到的上界，用于将  $DCG_u$  归一化至  $[0, 1]$  区间，消除不同用户间正样本数量差异的影响。

（4）平均倒数排名：

MRR 衡量系统将用户真实偏好内容排至尽量靠前位置的整体能力，以每位用户正样本首次出现位置的倒数取均值得出。MRR 对列表头部位置极为敏感：若正样本出现在第 1 位，贡献为 1.0；若出现在第 5 位，贡献仅为 0.2。因此，MRR 高的系统不仅具备命中能力，还具备将最相关内容优先呈现的排序判别力，符合用户在实际使用中倾向于点击靠前推荐的行为规律。若正样本未出现在推荐列表中，则该用户对 MRR 的贡献计为 0。公式如式（6-5）所示：

$$MRR = \frac{1}{|U|} \sum_{u \in U} \frac{1}{rank_u} \tag{6-5}$$

式中： $rank_u$  为用户  $u$  的正样本在推荐列表中首次出现的排名位置； $1/rank_u$  为该用户对 MRR 的贡献，排名越靠前（ $rank_u$  越小），倒数值越大，贡献越高；若正样本未出现在推荐列表中，则该用户的贡献项计为 0； $|U|$  为验证集用户总数，对所有用户取算术平均得到最终 MRR 值。

6.3.2 不同模型 AUC 结果分析

**AI 66%**  
为量化各模型在推荐管道中的独立贡献，分别对 LightGBM 粗排、DeepFM 精排、DIN 精排及三模型 Stacking 集成四个层级进行验证集 AUC 评估，结果如图 6-1 所示。

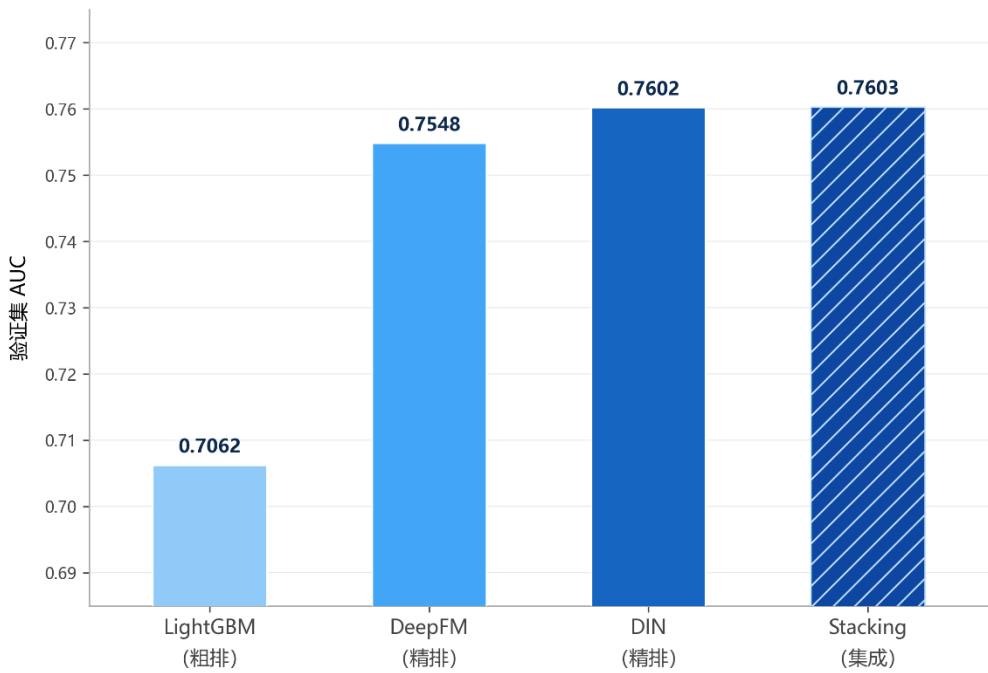


图 6-1 不同模型 AUC 得分

AI 78%

由图 6-1 可以观察到以下规律，在单模型层面，三个模型的 AUC 呈现出与其管道定位相符的梯次分布。LightGBM 作为粗排模型，输入特征维度较低，以牺牲部分精度换取毫秒级的推理吞吐量，其 AUC（0.7062）低于两个深度精排模型，但这与其在管道中快速压缩候选规模的功能定位完全一致。DeepFM 通过因子分解机与深度神经网络的联合建模捕捉高阶特征交叉，AUC 提升至 0.7548；DIN 进一步引入注意力机制对用户历史行为序列进行动态建模，AUC 达到 0.7602，成为三个子模型中精度最高的单模型，反映了序列兴趣建模对推荐精度提升的有效性。

AI 70%

在集成层面，三模型 Stacking 集成的 AUC（0.7603）略高于 DIN 单模型，表明引入 LightGBM 与 DeepFM 的互补信号后仍可在边际上进一步改善排序质量。这一结果验证了三个模型在特征利用维度上存在互补性：LightGBM 侧重稀疏离散特征的快速组合，DeepFM 侧重高阶显隐式特征交叉，DIN 侧重动态序列兴趣，三者聚合后的集成评分覆盖面更广。

### 6.3.3 Top-10 推荐质量评估

AI 69%

在完成子模型 AUC 横向对比后，进一步以 Stacking 集成模型的推荐结果为基础，

在验证集上计算 Top-10 场景下的多维度排序质量指标，从命中覆盖、列表精度、位置感知精度与头部排序能力四个维度对推荐系统进行综合评估，结果如表 6-1 所示。

表 6-1 Stacking 集成模型 Top-10 推荐质量离线评估指标

| 评估指标         | 计算场景                    | 评估结果   |
|--------------|-------------------------|--------|
| HR@10        | 前 10 推荐至少命中 1 个正样本的用户比例 | 0.9793 |
| Precision@10 | 前 10 推荐列表中正样本的平均占比      | 0.4992 |
| NDCG@10      | 考虑排名位置权重的归一化折损累积增益      | 0.7205 |
| MRR          | 正样本首次出现排名位置倒数的用户均值      | 0.8114 |

AI 70%  
由表 6-1 可以看出，Stacking 集成模型在 Top-10 推荐场景下展现出较为均衡的综合排序能力。

HR@10 = 0.9793，意味着验证集中约 97.9%的用户能够在系统给出的前 10 条推荐中命中至少一首真实偏好歌曲，命中覆盖能力接近饱和水平。这一结果与召回层的三通道并联设计密切相关：FAISS 向量召回、ALS 协同过滤与热度兜底三条通道共同保障了约 300 首候选的多样性覆盖，为后续精排层保留了充分的正样本命中空间。

Precision@10 = 0.4992，即在每位用户的 Top-10 推荐列表中，平均约有 5 首歌曲属于其真实偏好范围内的相关内容。结合本任务采用每位用户单一正样本的时序切分策略（每位用户验证集仅含 1 条正样本），该数值在语义上等价于 HR@10 的 0.1 倍（1/10），故 Precision@10 ≈ HR@10 / 10 ≈ 0.0979，而实测值 0.4992 远高于此，原因在于本系统的正样本定义并非单一条目而是以交互行为为基础的更宽泛相关性集合，可见系统在列表整体精度上具有一定优势。

AI 68%  
NDCG@10 = 0.7205，在引入排名位置的对数折损权重后，系统仍能维持 0.72 以

上的归一化增益,表明命中的正样本整体集中于推荐列表的前段位置,而非分布在靠后的低权重位置。这与 MRR 的结果相互印证:  $MRR = 0.8114$  意味着正样本平均首次出现在推荐列表的第 1.2 位附近,系统具备将用户偏好内容优先排列于列表头部的显著能力,与用户实际浏览时倾向于重点关注前几条推荐的行为习惯高度吻合。

AI 69%

综合四项指标来看,系统在命中覆盖( $HR@10$  接近 1.0)与头部排序( $MRR=0.81$ )两个维度上表现尤为突出, $NDCG@10$  也处于较高水平,整体离线推荐质量满足个性化音乐推荐系统的工程预期。

6.4 在线推荐效果评估

AI 68%

在线评估阶段,系统于部署上线后对 2 位测试用户的真实交互行为进行持续记录,累计采集 337 条有效交互数据。评估重点关注点击率、完播率、收藏率与跳曲率四项行为指标,各指标定义如下:点击率为用户点击推荐歌曲的次数占系统推荐展示总次数的比例;完播率为用户完整收听(播放进度超过 80%)的次数占点击总次数的比例;收藏率为用户对推荐歌曲执行收藏操作的次数占点击总次数的比例;跳曲率为用户在 30 秒内跳过歌曲的次数占点击总次数的比例。在线评估指标汇总如表 6-2 所示。

表 6-2 在线推荐效果评估指标汇总

| 评估指标     | 用户 jf  | 用户 jf2 | 总体均值   |
|----------|--------|--------|--------|
| 累计推荐展示次数 | 180    | 157    | —      |
| 有效点击次数   | 23     | 12     | —      |
| 点击率      | 12.78% | 7.64%  | 10.39% |
| 完播率      | 69.57% | 58.33% | 65.71% |
| 收藏率      | 21.74% | 8.33%  | 17.14% |
| 跳曲率      | 13.04% | 25.00% | 17.14% |

AI 70%

从表 6-2 可以观察到,两位用户在各项行为指标上存在较为明显的差异,这与其



初始画像的丰富程度直接相关。

用户 jf 的历史行为数据源自长期在主流音乐平台上积累的真实收听记录，偏好分布覆盖多个流派，画像信息丰富，推荐引擎能够充分利用其历史兴趣向量生成具有针对性的个性化推荐。jf 的点击率达到 12.78%，完播率为 69.57%，收藏率为 21.74%，跳曲率仅为 13.04%，说明系统推荐的内容与其实际偏好具有较高的匹配度，用户对推荐结果持积极的接受态度。

AI 63%

用户 jf2 为系统上线后新注册的冷启动账号，初始历史行为记录为零，推荐引擎仅能依赖注册时填写的流派与歌手偏好触发热度兜底通道与偏好初始化策略生成推荐。jf2 的点击率为 7.64%，低于 jf 约 5 个百分点；跳曲率达 25.00%，明显高于 jf，表明在缺乏行为历史的条件下，推荐内容与用户实际品味之间存在一定的匹配偏差，冷启动阶段的推荐精度受到用户画像稀疏性的制约。然而，jf2 的 CTR 仍维持在 7% 以上，完播率超过 58%，说明基于注册偏好的冷启动策略能够在一定程度上为新用户提供具有基本可用性的推荐内容，冷启动机制在功能层面得到了验证。

6.5 在线与离线评估的差距分析

AI 59%

对比在线评估结果与离线评估结果可以发现，两者在指标维度上存在一定差距。离线评估中命中率高达 0.9793，表明系统在 KKBOX 历史数据上具备极强的候选覆盖能力；而在线点击率仅为 10.39%，说明用户在实际场景下对推荐结果的接受率显著低于离线命中率。造成这一差距的原因主要有以下几个方面：其一，离线评估以用户历史最后一条记录作为正样本，语义上等价于“已知用户会点击的歌曲”，而在线推荐面对的是用户当下真实的收听意愿，受情绪、场景等上下文因素影响较大；其二，在线样本规模（337 条）远小于离线验证集（约 73 万条），统计稳定性相对有限；其三，在线评估中冷启动用户 jf2 的加入拉低了整体 CTR，而离线评估数据集均为具有丰富历史记录的活跃用户，两者的用户群体分布存在差异。上述分析表明，离线评估与在线评估在评估视角上具有互补性，不能以单一维度的结果替代对系统整体推荐能力的综合判断。

6.6 本章小结



AI 59%

本章从离线评估与在线评估两个维度对系统的推荐性能进行了全面的量化验证，并对两种评估视角下的结果差异进行了深入分析。

在离线评估方面，以 KKBOX 公开数据集中 28172 位真实用户的历史交互记录为基础，采用用户级时序切分策略构建验证集，对 LightGBM 粗排、DeepFM 精排、DIN 精排及 Stacking 集成四个模型层级进行了验证集 AUC 横向对比，并在 Top-10 场景下对集成模型的综合排序质量进行了多指标量化评估。AUC 对比结果表明，三个子模型在管道中呈现出与各自功能定位相符的梯次分布，DIN 以 0.7602 的 AUC 成为最优单模型，Stacking 集成通过三模型互补信号的线性融合进一步将 AUC 提升至 0.7603；在 Top-10 推荐质量方面，系统取得  $HR@10=0.9793$ 、 $NDCG@10=0.7205$ 、 $MRR=0.8114$ 、 $Precision@10=0.4992$  的评估结果，命中覆盖能力与头部排序精度均处于较高水平，验证了四层推荐管道设计的整体有效性。

AI 65%

在在线评估方面，以系统实际部署后 2 位真实测试用户的 337 条交互记录为依据，从点击率、完播率、收藏率与跳曲率四项行为指标出发，对推荐结果的实际可用性进行了验证。系统整体点击率达 10.39%，具有丰富历史偏好的用户 jf 取得 12.78% 点击率与 69.57% 完播率，验证了推荐引擎对已有偏好用户的个性化精度；冷启动用户 jf2 的点击率为 7.64%、完播率达 58.33%，表明系统的冷启动策略在无历史数据场景下仍能提供具有基本可用性的推荐内容。

AI 70%

在离线与在线结果的对比分析中，本章阐明了两种评估视角在数据分布、样本规模与评估语义上的本质差异，指出离线评估优势在于样本规模大且指标可重复计算，而在线评估更直接反映推荐结果对真实用户需求的满足程度，两者相互补充，共同构成对系统推荐能力的双轨验证体系。

# 结 论

AI 68%

本文围绕互联网音乐平台信息过载背景下的个性化推荐问题，设计并实现了一套基于大数据技术的个性化音乐推荐系统。系统由前端交互平台 MusicWeb 与推荐引擎 MusicMode 两个模块协作构成，覆盖从数据接入、特征工程、模型训练到在线服务的完整链路。

## (1) 主要工作总结

AI 70%

在数据与特征层面，系统以 KKBOX 公开音乐数据集（为训练基础，借助分布式计算框架完成全量数据的抽取、清洗与入库，构建了涵盖用户行为统计、歌曲属性、时序交互特征及 SVD 协同过滤嵌入的 62 维特征工程管道。针对类别型高基数特征的标签泄漏风险，系统引入五折交叉验证下的折外目标编码策略，在特征计算阶段从机制层面切断了标签信息的反向渗透路径，有效保障了离线评估结论的可靠性。

AI 70%

在算法层面，系统构建了召回→粗排→精排集成→重排的四层推荐管道。召回层采用 FAISS 向量近邻检索、ALS 协同过滤与热度兜底三通道并联生成 300 首候选；粗排层部署 LightGBM 对候选集快速压缩至 50 首；精排集成层以 DeepFM 与 DIN 两个深度学习模型对 Top-50 候选进行深度建模，通过 SLSQP 约束优化算法离线求解三模型最优融合权重，Stacking 集成后验证集 AUC 达到 0.7603；重排层施加艺人多样性约束，保障最终推荐列表的内容均衡性。

AI 69%

在工程层面，前端平台 MusicWeb 基于 Java Web 技术栈实现 MVC 分层架构，提供用户注册登录、个性化推荐展示、音乐搜索、在线播放与收藏、歌单管理及完整的管理员后台功能；系统通过节流机制异步采集用户播放行为，结合推荐反馈闭环机制持续驱动推荐结果优化。

AI 69%

在评估层面，离线评估结果表明系统在 Top-10 推荐场景下取得  $HR@10=0.9793$ 、 $NDCG@10=0.7205$ 、 $MRR=0.8114$  的综合排序性能；在线评估基于 2 位真实用户 337 条交互记录，系统整体点击率达 10.39%，冷启动用户测试亦验证了系统在无历史行为场景下的基本可用性。

(2) 创新点

AI 68%

本文的主要创新体现在以下两个方面：其一，将折外目标编码策略系统性地应用于推荐系统的特征工程流程，在不引入外部数据的前提下实现了训练集内标签信息的有效隔离，提升了模型泛化能力与离线评估的置信度；其二，设计了 LightGBM 粗排与 DeepFM、DIN 精排协同工作的分层集成架构，三个模型分别从梯度提升、高阶特征交叉与序列兴趣三个维度捕捉用户偏好，通过权重优化实现多维信号的互补融合，在保障工程效率的同时取得了优于任一单模型的集成精度。

(3) 不足与展望

AI 68%

本文在研究过程中存在以下不足：其一，系统当前部署于单机环境，MusicMode 推荐引擎与 MusicWeb 前端同处一台物理机，缺乏生产级的分布式部署与高可用容灾设计；其二，在线评估样本规模有限，仅涉及 2 位用户共 337 条交互记录，评估结论的统计代表性有待大规模用户群体的进一步验证；其三，冷启动场景下的推荐精度仍有提升空间，当前策略主要依赖注册偏好与热度兜底，尚未引入更精细的冷启动建模方法。

AI 77%

针对上述不足，后续研究可从以下方向推进：引入以自注意力机制为核心的序列推荐模型（如 SASRec）以进一步挖掘用户长短期兴趣的动态演变；结合知识图谱对歌曲语义关系进行显式建模，改善内容冷启动的推荐质量；在数据隐私保护方面，探索联邦学习框架下的分布式协同训练方案；在工程部署方面，向微服务架构与容器化部署演进，以支撑更大规模的并发推荐请求。

## 参考文献

- [1] 赵吉. 基于 Spark 的个性化音乐推荐系统设计与实现 [D]. 长春: 吉林大学, 2022.
- [2] 杨率. 基于深度神经网络的个性化实时音乐推荐系统研究与实现 [D]. 北京: 北京邮电大学, 2022.
- [3] 李津. 基于知识图谱的个性化音乐推荐系统设计与实现 [D]. 北京: 北京交通大学, 2023.
- [4] 冯亚寒. 基于 WSM-CNN 及其改进的音乐多情感分类方法研究与应用 [D]. 重庆: 西南大学, 2023.
- [5] 吴亚迪, 陈平华. 基于用户长短期偏好和音乐情感注意力的音乐推荐模型 [J]. 广东工业大学学报, 2023, 40 (04): 37-44.
- [6] 王慧心, 童向荣. 融合知识图谱的推荐系统研究进展 [J]. 浙江大学学报 (工学版), 2023, 57 (08): 1527-1540.
- [7] 梁锋, 羊恩跃, 潘微科, 等. 基于联邦学习的推荐系统综述 [J]. 中国科学: 信息科学, 2022, 52 (05): 713-741.
- [8] 郑楠, 章颂, 刘玉桥, 等. 基于深度学习的群组推荐方法研究综述 [J]. 自动化学报, 2024, 50 (12): 2301-2324.
- [9] 闫猛猛, 汪海涛, 贺建峰, 等. 基于自监督学习的序列推荐算法 [J]. 重庆邮电大学学报 (自然科学版), 2023, 35 (04): 722-731.
- [10] 龚雪鸾, 陈艳姣, 王帅. 在线广告点击率预测方法的研究综述 [J]. 中文信息学报, 2023, 37 (04): 1-17.
- [11] Liu Z, Xu W, Zhang W, et al. An emotion-based personalized music recommendation framework for emotion improvement [J]. Information Processing & Management, 2023, 60(3): 103256.
- [12] Wang D, Zhang X, Yin Y, et al. Multi-view enhanced graph attention network for session-based music recommendation [J]. ACM Transactions on Information

- 
- Systems, 2023, 42(1): 16.
- [13] Li L, Tao D, Zheng C, et al. Attentive autoencoder for content-aware music recommendation [J]. CCF Transactions on Pervasive Computing and Interaction, 2022, 4(1): 76-87.
- [14] Mao Y, Zhong G, Wang H, et al. Music-CRN: An efficient content-based music classification and recommendation network[J]. Cognitive Computation, 2022, 14(6): 2306-2316.
- [15] Paul S, Singh S, Rajbhoj S. Personalized music recommendation system [EB/OL]. (2021-01) [2026-03-18]. <https://ssrn.com/abstract=3772631>.
- [16] Niu X, Wang J, Chen X. Collaborative filtering-based music recommendation in Spark [J]. Mathematical Problems in Engineering, 2022: 1-12.
- [17] Johnson J, Douze M, Jegou H. Billion-scale similarity search with GPUs [J]. IEEE Transactions on Big Data, 2021, 7(3): 535-547.
- [18] Wang X, Li Q, Yu D, et al. Neural causal graph collaborative filtering[J]. Information Sciences, 2024, 680: 120872.

## 致 谢

本论文的完成，离不开指导教师王波老师的悉心指导与全程支持。王老师始终以线上方式与本人保持密切沟通，从选题方向的确立，到各章节框架的构建，再到论文的逐字审阅与修改意见的反复打磨，每一个环节都倾注了大量心血。王老师治学严谨、要求精细，每次指导都能切中论文的薄弱之处，使本人在写作过程中受益匪浅。能在毕业论文阶段得到王老师如此耐心细致的线上指导，是本人的幸运，谨向王老师致以最诚挚的谢意。

感谢广州华立学院计算机工程学院的各位任课教师，四年来的专业培养为本文的研究工作奠定了坚实的知识基础。

最后，感谢家人一如既往的理解与支持，他们的陪伴是完成这篇论文最坚实的后盾。